

COMPENDIO PERSONAL · JORGEAGG

Compendio de instalación Gentoo Linux AMD64

53 capítulos en español, elaborados capítulo a capítulo junto con una instalación real del Gentoo Handbook oficial — con diagramas, badges de riesgo y bloques "para qué sirve".

AMD64 multilib

OpenRC

GPT + UEFI

GRUB

dracut

es_MX.UTF-8

Basado en el Gentoo AMD64 Handbook oficial

Índice

I. FUNDAMENTOS

01 Introducción

II. MEDIO DE INSTALACIÓN

- 03 Medios de instalación de Gentoo
- 04 Descargando
- 05 Grabar el medio de arranque
- 06 Arranque del medio de instalación

III. RED DURANTE LA INSTALACIÓN (MEDIO LIVE)

- 07 Configuración automática de la red
- 09 Configuración Inalámbrica (WiFi)
- 10 Configuración específica de aplicaciones
- 11 Conceptos básicos de Internet e IP
- 12 Configuración manual de red con IP estática

IV. PREPARANDO EL DISCO

- 02 Particionado del disco
- 08 Preparando los discos
- 13 Introducción a los dispositivos de bloque
- 14 Diseñando un esquema de particiones
- 15 Particionar el disco con GPT para UEFI
- 16 Particionando el disco con MBR para BIOS / Legacy Boot
- 17 Creación de sistemas de archivos
- 18 Montar la partición raíz

V. SISTEMA BASE (STAGE3)

- 19 Eligiendo el Stage File
- 20 Descargando el archivo stage
- 21 Instalando el archivo stage
- 22 Configurando las opciones de compilación
- 23 Referencias Informativo

VI. CHROOT Y PORTAGE

- 24 Chrooting
- 25 Configurando Portage
- 26 Opcional: Usar systemd como sistema y gestor de servicios

VII. CONFIGURACIÓN REGIONAL Y HARDWARE

- 27 Zona horaria (Timezone)
- 28 Configurar los locales
- 29 Referencias Sin comandos
- 30 30. Instalando Firmware y Microcódigo
- 31 sys-kernel/installkernel

VIII. EL KERNEL

- 32 Selección del Kernel

IX. FSTAB Y RED PERMANENTE

- 33 Información del sistema de archivos

34	Información de red
----	--------------------

X. CONFIGURACIÓN INICIAL DEL SISTEMA Y SERVICIOS

35	System Information
----	--------------------

36	System Logger
----	---------------

37	Cron Daemon
----	-------------

38	Indexado de archivos
----	----------------------

39	Acceso remoto por shell (SSH)
----	-------------------------------

40	Shell Completion (Bash)
----	-------------------------

41	Sincronización de tiempo
----	--------------------------

42	Herramientas de Sistemas de Archivos
----	--------------------------------------

XI. BOOTLOADER

43	Seleccionar e instalar el Bootloader
----	--------------------------------------

44	Bootloader — GRUB y alternativas
----	----------------------------------

45-46	Bootloaders alternativos
-------	--------------------------

47	Otras alternativas de bootloader No aplica a tu instalación
----	---

XII. FINALIZANDO LA INSTALACIÓN

48	Reiniciar el sistema
----	----------------------

49	Mantenimiento de Portage
----	--------------------------

50	Administración de usuarios
----	----------------------------

51	Limpieza de disco
----	-------------------

52	¿A dónde ir desde aquí?
----	-------------------------

53	Reflexiones finales Informativo
----	---------------------------------

APÉNDICE

A	Gestión de paquetes post-instalación
---	--------------------------------------

Introducción

La filosofía detrás de Gentoo, antes de tocar el teclado

En este capítulo:

- Bienvenida a Gentoo
- Los tres pilares de diseño
- Cómo está organizada la instalación
- Opciones de instalación
- Qué hacer si algo sale mal

Bienvenida

CONCEPTUAL

Gentoo es un sistema operativo libre basado en Linux, pensado para ser **optimizado y personalizado** para casi cualquier equipo o propósito. A diferencia de otras distribuciones que entregan un sistema ya armado y "cerrado", Gentoo no esconde nada de lo que ocurre por debajo: cada pieza del sistema es visible y ajustable por quien lo instala.

PARA ENTENDERLO FÁCIL

La mayoría de las distribuciones de Linux son como comprar un mueble ya armado en una tienda: funciona, pero viene con las decisiones de diseño de alguien más. Gentoo es más parecido a comprar la madera y las herramientas: toma más tiempo armarlo, pero el resultado está hecho exactamente a la medida de quien lo construyó.

Los tres pilares de diseño

Todo el proyecto Gentoo se sostiene sobre tres principios que se repiten a lo largo del manual.

🔓 Apertura (Openness)

Las herramientas principales de Gentoo están construidas con lenguajes de programación simples y accesibles: **Portage** (el sistema que instala y administra programas) está escrito en Python, y los **ebuilds** (las "recetas" que le dicen a Portage cómo instalar cada programa) están escritos en bash. Cualquier persona puede leerlos, modificarlos y proponer mejoras.

Por diseño, los programas se instalan **compilando su código fuente** (es decir, se construyen desde cero en la propia máquina), aunque también existe la opción de usar paquetes ya compilados. La configuración del sistema se hace mediante archivos de texto simples, no interfaces ocultas.

Elección (Choice)

Gentoo evita imponer decisiones. A lo largo de la instalación se elige, entre otras cosas:

- El sistema de inicio: **OpenRC** o **systemd**
- Cómo particionar el disco y qué sistema de archivos usar
- Un "perfil" base del sistema
- Qué características activar o quitar por programa mediante las **USE flags**
- Bootloader, gestor de red, entorno de escritorio, y más

CONCEPTO: USE FLAGS

Son como interruptores que le dicen a un programa qué funciones incluir al compilarlo. Por ejemplo, agregar `ssl` compila un programa con soporte de cifrado SSL, mientras que `-X` (con el signo menos) le quita el soporte para entornos gráficos.

Poder (Power)

Al construirse desde el código fuente, Gentoo puede adaptarse a nuevas arquitecturas de procesador y **afinar cada programa instalado** para sacarle el máximo provecho al hardware específico de cada máquina.

Este ajuste fino se controla principalmente desde un solo archivo:

`/etc/portage/make.conf`, que define las opciones de compilación que se aplican a todo el sistema.

Cómo está organizada la instalación

El manual no es una única receta rígida; está pensado como un conjunto de pasos que se pueden adaptar según las necesidades de cada persona. En términos generales, los pasos se clasifican así:

TIPO DE PASO	QUÉ SIGNIFICA
Sugerido	Recomendado para la mayoría de las instalaciones

Opcional	Depende de las necesidades particulares del usuario
Obsoleto	Se mantiene por compatibilidad, pero ya no se recomienda
Con alternativas	Tiene un valor por defecto, pero se documentan otras opciones

A TENER EN CUENTA

No todos los pasos del manual aplican a todas las instalaciones. Parte del proceso consiste en decidir, capítulo a capítulo, cuáles pasos son necesarios para el sistema que se quiere construir.

Opciones de instalación para Gentoo

El capítulo menciona que existen distintas vías para llegar a un sistema Gentoo funcional (por ejemplo, distintos medios de instalación y distintos puntos de partida), y que el resto del manual va guiando esas decisiones paso a paso.

Problemas (Troubles)

Esta sección final del capítulo funciona como una guía de referencia para cuando algo no sale como se espera durante la instalación, con enlaces a los canales de soporte oficiales de la comunidad.

▼ Canales de soporte de la comunidad Gentoo

- **IRC:** canal `#gentoo` en la red Libera Chat (accesible también vía webchat)
- **Foros oficiales** de Gentoo
- **Listas de correo**
- **Sistema de reporte de bugs**

Estos mismos canales se documentan con más detalle en el Capítulo 52 ("Where to go from here") al final del manual.

En resumen

IDEA CENTRAL DEL CAPÍTULO

El Capítulo 1 no instala nada todavía. Es la carta de presentación de Gentoo: un sistema que prioriza la **transparencia**, la **libertad de elección** y el **rendimiento a medida**, a cambio de un mayor involucramiento del usuario en su configuración.

Compendio Gentoo en español · Capítulo 1: Introducción
Basado en el Gentoo AMD64 Handbook oficial

Capítulo 3 — Medios de instalación de Gentoo

Elegir con qué imagen vas a arrancar el instalador, y qué es un stage file.

Para qué sirve este capítulo: antes de instalar Gentoo necesitas arrancar una computadora desde algún medio (USB, CD, etc). Aquí eliges qué tipo de imagen usar para eso.

3.1 — CD de instalación mínimo (Minimal Installation CD)

SIN RIESGO

Es una imagen pequeña y autosuficiente. Al arrancar, detecta el hardware automáticamente y carga los drivers necesarios. Todo se hace desde la terminal.

Requisito	Mínimo 140 MB de RAM
Interfaz	Solo terminal (texto)
Nombre típico	<code>install-amd64-minimal-<fecha>.iso</code>

3.2 — Gentoo LiveGUI

SIN RIESGO

Versión más cómoda: trae un escritorio KDE completo, con navegador gráfico para leer el manual mientras instalas en otra ventana de terminal. También tiene mejor soporte WiFi (con applet de NetworkManager visual).

Requisito	Mínimo 2 GB de RAM
Interfaz	Escritorio gráfico (KDE)
Ventaja	Mejor soporte de drivers WiFi

▼ ¿Cuál elegir?

Si tu equipo es limitado en recursos o prefieres trabajar 100% en terminal, usa el **Minimal CD**. Si tienes suficiente RAM y quieres comodidad (navegador para leer el manual mientras instalas), usa el **LiveGUI**. Ambos instalan exactamente el mismo Gentoo — la diferencia es solo el entorno desde el que trabajas.

3.3 — ¿Qué son los "stage files"?

SIN RIESGO

Un **stage file** es un archivo comprimido que sirve como "semilla" del sistema: es el sistema base mínimo desde el cual se construye todo Gentoo. Se descarga desde `releases/amd64/autobuilds/` en los mirrors oficiales.

Nota histórica: antes existían stages 1 y 2, pero quedaron obsoletos. Hoy solo se usa **stage3**.

Información adicional — ¿Qué combinaciones de stage3 existen?

Contenido relacionado, útil para decidir qué descargar más adelante

El stage3 se elige combinando dos decisiones independientes:

Categoría	Opción A	Opción B
Sistema de inicio (init)	OpenRC — nativo de Gentoo, basado en dependencias	systemd — moderno, usado por la mayoría de distros
Arquitectura de librerías	Multilib — 64 bits + compatibilidad 32 bits	No-multilib — 64 bits puro

Recomendación para principiantes: `OpenRC + multilib` (o `systemd + multilib` si prefieres systemd). Evita **no-multilib** al empezar — migrar de ahí a multilib después es muy difícil.

▼ ¿Cuándo se configura el profile?

El stage3 ya trae un profile activado por defecto — no hay que hacer nada aquí. Más adelante, ya dentro del sistema (chroot) y con Portage sincronizado, se puede revisar o cambiar con:

Comando	Para qué sirve	Riesgo
<code>eselect profile list</code>	Muestra los profiles disponibles	bajo
<code>eselect profile set N</code>	Cambia al profile número N	medio

Capítulo 4 — Descargando

Manual de instalación de Gentoo (AMD64) · Resumen sencillo

Este capítulo trata de dos cosas: **conseguir** el archivo de instalación (la imagen ISO) y **comprobar** que ese archivo es real y no está dañado antes de usarlo. Todavía no se instala nada — es la etapa de preparación.

4.1 — Obtener el medio de instalación

Se descarga la imagen `.iso` desde los servidores espejo ("mirrors") de Gentoo. Junto a ella aparecen otros archivos que la acompañan:

Archivo	Para qué sirve
<code>.iso</code>	El medio de instalación en sí (lo que se graba en el USB).
<code>.iso.asc</code>	Firma digital del ISO — sirve para comprobar que es auténtico.
<code>.CONTENTS.gz</code>	Lista comprimida de todo lo que trae el ISO (drivers, firmware, etc.).
<code>.DIGESTS</code>	Checksums (huellas) del archivo en varios algoritmos (SHA256, SHA512, BLAKE2B).
<code>.sha256</code>	Checksum SHA256 firmado con PGP (no siempre disponible).

Navegar los mirrors manualmente

Si no se usa el enlace directo de descarga, el recorrido típico dentro del mirror es:

1. Elegir un mirror cercano.
2. Entrar a `releases/`
3. Entrar a la arquitectura, por ejemplo `amd64/`
4. Entrar a `autobuilds/`
5. Elegir `current-install-amd64-minimal/` y descargar el `.iso`

También se puede usar un navegador de texto como `links` o `lynx` desde una terminal, útil en entornos sin interfaz gráfica.

4.2 — Verificar los archivos descargados

RECOMENDADO

Esto confirma dos cosas: que el archivo **no se corrompió** durante la descarga y que **de verdad**

lo publicó el equipo de Gentoo (no fue alterado por un tercero).

Idea clave: el archivo .asc es una "firma" hecha con la clave privada de Gentoo. Solo Gentoo puede crearla, pero cualquiera puede comprobarla con su clave pública. Si la firma coincide, el archivo es legítimo.

En Windows

Se usa un programa como **GPG4Win**. Se importan las claves públicas de Gentoo (disponibles en su página oficial de firmas) y luego se verifica el .iso contra su .asc.

En Linux

Se usa gnupg. El flujo básico:

```
# 1. Importar la clave pública de Gentoo
gpg --keyserver htps://keys.gentoo.org --recv-keys 13EBDBEDE7A12775DFDB1BABB572E0E2D182916

# 2. Verificar la firma del ISO
gpg --verify install-amd64-minimal-<fecha>.iso.asc install-amd64-minimal-<fecha>.iso
```

Si todo está bien, aparece Good signature from... Es importante comparar el *fingerprint* (huella) mostrado con el publicado en la web oficial, para confirmar que la clave usada es realmente de Gentoo y no una falsificación.

Alternativa: verificar solo el checksum (sin GPG)

Si no se quiere usar GPG, también se puede comprobar la integridad del archivo (que no esté corrupto) comparando su huella con la del archivo .DIGESTS o .sha256, usando herramientas como sha256sum u openssl.

```
sha256sum --check install-amd64-minimal-<fecha>.iso.sha256
```

Diferencia importante: el checksum solo detecta descargas dañadas; no garantiza que el archivo sea auténtico. Para eso hace falta la firma GPG.

Tabla comparativa: checksum vs. firma GPG

Método	¿Qué comprueba?	¿Detecta manipulación maliciosa?
Checksum (SHA256, etc.)	Que el archivo llegó completo y sin errores	✗ No
Firma GPG (.asc)	Que el archivo lo publicó realmente	✓ Sí

Recomendación práctica: si es la primera vez, basta con el checksum para descartar una descarga corrupta. Para mayor seguridad (por ejemplo, en una red pública o si el archivo se bajó de un mirror poco conocido), conviene verificar también la firma GPG.

¿Qué sigue después de este capítulo?

Una vez descargado y verificado el ISO, el siguiente paso (Capítulo 5) es **escribirlo en un medio de arranque** — usualmente un USB con dd en Linux, o herramientas como Rufus en Windows (evitando programas como Ventoy, que el manual advierte que pueden causar fallos de arranque).

Capítulo 5 — Grabar el medio de arranque

Manual de instalación de Gentoo AMD64 · Resumen simplificado

En este capítulo:

- Idea general
- 5.1 Crear un USB booteable
- 5.2 Grabar en disco (CD/DVD)
- Extra: verificar y comparar métodos

Idea general

Ya tienes el archivo `.iso` de Gentoo descargado, pero un ISO por sí solo no arranca una computadora. Este capítulo explica cómo "volcar" ese ISO a un USB o disco físico para que la máquina pueda iniciar desde ahí.

5.1 Crear un USB booteable

Advertencia: No uses herramientas como **Ventoy** para escribir el USB — puede provocar fallos de arranque con el medio de Gentoo.

▼ Paso 1 — Identificar el dispositivo USB riesgo bajo

Antes de escribir nada, hay que saber qué nombre tiene tu USB en el sistema (ej. `/dev/sdd`).

```
root # dmesg
```

```
root # lsblk
```

Para qué sirve: `dmesg` muestra el hardware detectado recientemente (útil justo después de conectar el USB). `lsblk` lista todos los discos y particiones del sistema para confirmar cuál es el correcto por su tamaño.

▼ Paso 2 — Escribir el ISO con `dd` riesgo alto

```
root # dd if=install-amd64-minimal-<fecha>.iso of=/dev/sdd bs=4096 status=progress && sync
```

Parte	Significado
<code>if=</code>	archivo de entrada (el ISO descargado)
<code>of=</code>	dispositivo de salida (tu USB) — todo su contenido se borra
<code>bs=4096</code>	tamaño de bloque, acelera la escritura

`status=progress`

muestra el avance en tiempo real

`sync`

fuerza a que todo quede realmente escrito antes de retirar el USB

Cuidado: usa el dispositivo base (`sdd`), no una partición (`sdd1`). Si te equivocas de `of=`, borras el disco incorrecto — no hay "deshacer".

Para qué sirve: `dd` copia el ISO byte por byte al USB, replicando la estructura de arranque (particiones GPT incluidas) tal cual la trae la imagen oficial.

▼ Alternativa en Windows

riesgo bajo

Se usa el programa gráfico **win32diskimager**, que cumple la misma función que `dd` pero con interfaz visual.

Advertencia del manual: descárgalo únicamente desde su página oficial en SourceForge, para evitar versiones alteradas.

5.2 Grabar en disco (CD/DVD)

▼ En Linux —

`cdrecord`

riesgo medio

```
user $ cdrecord dev=/dev/sr0 install-amd64-minimal-20141204.iso
```

Para qué sirve: graba el ISO como imagen booteable en un disco físico. `/dev/sr0` es la primera unidad óptica del sistema.

Si prefieres interfaz gráfica, **K3B** hace lo mismo desde su menú "Grabar imagen de CD".

▼ En Windows 7 o superior

riesgo bajo

No necesitas programas extra: clic derecho sobre el archivo ISO → "**Grabar imagen de disco**".

Extra: comparación rápida de métodos

Medio	Sistema	Herramienta	Nivel
USB	Linux	<code>dd</code>	alto
USB	Windows	win32diskimager	bajo
CD/DVD	Linux	<code>cdrecord</code> / K3B	medio
CD/DVD	Windows 7+	Explorador de archivos	bajo

Tip (aporte adicional): antes de grabar, verifica la integridad del ISO comparando su checksum

(`sha256sum install-amd64-minimal-<fecha>.iso`) contra el valor publicado en la página de descargas de Gentoo. Un ISO corrupto puede causar errores extraños durante la instalación que son difíciles de diagnosticar después.

Tip (aporte adicional): el USB es preferible al CD/DVD hoy en día — es más rápido, reutilizable, y casi todo hardware moderno arranca desde USB sin problema.

Capítulo 6 — Arranque del medio de instalación

Cómo bootear tu USB/CD de Gentoo y qué hacer si tu hardware necesita ayuda extra.

¿Qué logramos en este capítulo?

Ya tienes el medio de instalación grabado. Aquí aprendes a **arrancar la computadora desde ese medio** en lugar del disco interno, y a pasarle "pistas" al kernel si tu hardware es antiguo, raro, o simplemente no se detecta bien solo.

🔧 **Para la mayoría de los equipos modernos:** basta con cambiar el orden de arranque y presionar `Enter`. Todo lo demás en este capítulo es "por si acaso".

Paso a paso

- 1 **Entra al firmware de la placa madre.** Reinicia la PC e inmediatamente presiona la tecla correspondiente (`DEL`, `F1`, `F10` o `ESC`, según el fabricante). Si no sabes cuál es, busca en internet el modelo exacto de tu placa.

Para qué sirve: sin esto, la PC nunca "verá" tu USB/CD y arrancará el disco interno vacío.

- 2 **Cambia el orden de arranque** para que el USB/CD tenga prioridad sobre el disco interno.

⚠ Si tu placa usa **UEFI**, verifica que el medio arranque en **modo UEFI**, no en modo BIOS/legacy. Elegir el modo equivocado aquí causa problemas más adelante al instalar el gestor de arranque (GRUB).

- 3 **Guarda cambios y reinicia.** Aparecerá un menú tipo GRUB con varias entradas de arranque.
- 4 **Elige el kernel.** Con `Enter` arrancas con la opción por defecto (`gentoo`) — es lo que necesita casi todo el mundo. Solo si tu hardware da problemas, presiona `e` para editar la línea y agregar parámetros (ver siguiente sección).

Para qué sirve: `F1` muestra los kernels disponibles y `F2` las opciones de arranque, por si necesitas consultarlas en el momento.

- 5 **Espera el login automático.** El sistema te deja como usuario `root` directamente en el Minimal CD. En el **LiveGUI**, abre una terminal y ejecuta `sudo -i` para volverte superusuario (la contraseña de root viene deshabilitada por defecto).

Kernels disponibles

Kernel	Uso
<code>gentoo</code>	Opción por defecto. Sirve para prácticamente todo el hardware moderno. Recomendado
<code>gentoo-nofb</code>	Igual, pero sin framebuffer (pantalla gráfica en consola). Útil si la pantalla se ve rota.

Cómo editar la línea de arranque

Resalta la entrada deseada, presiona **e**, escribe la(s) opción(es) al final de la línea, separadas por espacio, y arranca con **Ctrl + X** o **F10**.

```
linux /gentoo nodhcp nosmp
```

Para qué sirve: este cambio es *temporal* — solo aplica a ese arranque, no modifica nada en disco.

Orden de evaluación: el sistema revisa primero las opciones **no*** y después las **do***. Si combinas ambas para el mismo tema, el orden en que las escribas puede "sobrescribirse" entre sí.

Opciones de hardware

Energía y arquitectura (ACPI, APM, APIC, SMP)

Solo si hay problemas

Opción	Qué hace	Ejemplo
acpi=on	Activa ACPI (gestión moderna de energía/hardware) y arranca acpid .	Placa nueva con hardware mal detectado
acpi=off	Desactiva ACPI por completo. También desactiva Hyperthreading.	Equipo muy antiguo que se cuelga al iniciar
doapm	Activa soporte APM (gestión de energía previa a ACPI). Requiere acpi=off .	acpi=off doapm
noapic	Desactiva el controlador avanzado de interrupciones (placas modernas).	Hardware antiguo con conflictos de IRQ
noapic	Desactiva el APIC local en kernels de un solo procesador.	—
nosmp	Desactiva multiprocesamiento simétrico, para aislar bugs de drivers/multinúcleo.	Placa con bug conocido en modo multiprocesador

Almacenamiento y buses (SCSI, SATA, IDE, USB, PCMCIA)

Opción	Qué hace	Ejemplo
doscsi	Activa soporte SCSI. También necesario para arrancar desde USB, que usa este subsistema.	El instalador no ve tu pendrive
dopcmcia	Activa soporte para tarjetas PCMCIA/Cardbus.	Laptop vieja con

		módem/red por PCMCIA
<code>ide=nodma</code>	Desactiva DMA en IDE. Ayuda si el lector de CD da errores de lectura.	—
<code>nosata</code>	Desactiva módulos Serial ATA.	Problemas con el subsistema SATA
<code>nousb</code>	Desactiva la carga automática de módulos USB.	Depurar fallos relacionados a USB
<code>slowusb</code>	Agrega pausas extra para lectores USB lentos.	Unidad óptica USB antigua
<code>sda=stroke</code>	Permite particionar un disco completo aunque el BIOS sea muy antiguo. Sustituye <code>sda</code> por el disco real.	—
<code>nodmraid</code> / <code>dmraid=X</code>	Desactiva o configura el RAID por software del kernel (device-mapper).	<code>dmraid="-i -a no"</code>

Red y consola remota

Opción	Qué hace	Ejemplo
<code>nodhcp</code>	Desactiva la búsqueda automática de IP por DHCP.	Red que solo usa IP fija
<code>nonfs</code>	Desactiva el arranque de portmap/nfsmount.	—
<code>console=X</code>	Configura acceso por consola serial (instalar sin monitor). Formato: dispositivo + parámetros de conexión.	<code>console=ttyS0,9600,8,n,1</code>
<code>dosshd</code> + <code>passwd=X</code>	Arranca sshd en el boot con la contraseña de root indicada. Ideal para instalación remota	<code>dosshd passwd=miclave123</code>

Para qué sirve la combinación `dosshd` + `passwd` : te permite conectarte por SSH desde otra máquina y hacer toda la instalación sin teclado ni monitor conectados.

Volúmenes lógicos (LVM)

Opción	Qué hace	Ejemplo
<code>dolvm</code>	Activa soporte del kernel para LVM (Logical Volume Manager), por si tu esquema de disco usa volúmenes lógicos en vez de particiones tradicionales.	<code>gentoo</code> <code>dolvm</code>

Para qué sirve: LVM te permite redimensionar "particiones" (volúmenes) después, sin las limitaciones rígidas del particionado clásico. No aplica a tu instalación actual (GPT simple), pero es bueno saber que existe.

Depuración y otras opciones (debug, doload, nox, scandelay...)

Opción	Qué hace	Ejemplo
<code>debug</code>	Muestra mucha información de depuración en pantalla.	—
<code>docache</code>	Copia todo el CD a RAM, permitiendo desmontar y montar otro disco. Requiere el doble de RAM que el tamaño del CD.	—
<code>doload=X</code> / <code>noload=X</code>	Fuerza cargar (o evitar cargar) un módulo específico del kernel.	<code>doload=e1000e</code>
<code>nodetect</code>	Desactiva toda autodetección de hardware y DHCP, para depurar por qué falla el arranque.	—
<code>nofirewire</code>	Desactiva módulos Firewire.	—
<code>nogpm</code>	Desactiva el mouse en consola.	—
<code>nohotplug</code>	Desactiva scripts de hotplug/coldplug al arrancar.	—
<code>nokeymap</code>	Desactiva selección de teclado no-US.	—
<code>nosound</code>	Desactiva sonido y ajuste de volumen.	—
<code>nox</code>	En un LiveCD con X, evita que arranque el entorno gráfico y deja la línea de comandos.	—
<code>scandelay</code> / <code>scandelay=X</code>	Pausa de 10 s (o los segundos que definas) para dar tiempo a dispositivos lentos en inicializar.	<code>scandelay=20</code>

Para enriquecer: cosas que vale la pena saber

▣ **Teclado no-US:** si tu teclado no es inglés/US, presiona `Alt` + `F1` apenas veas la barra de progreso, para activar el modo detallado y elegir tu distribución. Si no seleccionas nada en 10 segundos, se usa el layout US por defecto.

✂ **Consolas virtuales:** una vez dentro del entorno live, puedes moverte entre varias consolas con `Alt` + `F2`, `F3`, `F4`, y volver a la principal con `Alt` + `F1`. Útil para tener una consola con documentación abierta y otra ejecutando comandos.

⌚ **Timeout de 15 segundos:** si no eliges nada en el menú de GRUB, el medio de instalación arranca el disco interno por defecto. Esto es intencional: permite reiniciar remotamente y que la máquina vuelva sola al sistema ya instalado, sin tener que quitar el USB/CD a mano.

Capítulo 7 — Configuración automática de la red

Handbook de Gentoo AMD64 · Instalación · Networking (parte 1)

En una frase: antes de instalar nada hace falta internet en el sistema live. Este capítulo revisa si la red ya funciona sola (lo más común) y cómo comprobarlo.

¿Por qué existe este paso?

Todo lo que sigue en la instalación (descargar el stage3, sincronizar Portage, compilar el kernel) necesita salir a internet. Antes de tocar cualquier configuración manual, el Handbook te pide comprobar si ya tienes conexión, porque en la mayoría de los casos **no hay que configurar nada**.

"A lo mejor ya funciona" CONCEPTO

Si tu equipo está conectado por cable Ethernet a un router con DHCP —el caso típico en casa o en la oficina— es muy probable que la red ya haya quedado configurada sola al arrancar el medio de instalación.

Para qué sirve saber esto: te ahorra tiempo. Si tienes cable y router normal, salta directo a la sección de "Comprobar la conexión" antes de intentar configurar nada a mano.

DHCP CONCEPTO

DHCP (*Dynamic Host Configuration Protocol*) es el protocolo que le entrega automáticamente a tu equipo todo lo que necesita para hablar en la red: dirección IP, máscara de red, ruta por defecto, servidores DNS, etc.

Para que funcione tiene que existir un servidor DHCP en la misma red local — normalmente es el propio router. Por defecto, el medio live de Gentoo usa **NetworkManager**, que intenta conectarse por DHCP automáticamente. Si eso no pasa, lo primero es revisar el cable físico.

Comprobar la conexión (Testing the network)

Tres chequeos, de lo más básico a lo más específico:

▼ 1. ¿Hay una ruta por defecto? BAJO RIESGO

```
root # ip route
default via 192.168.0.1 dev enp1s0
```

Para qué sirve: la ruta por defecto es la "puerta de salida" hacia internet. Si el comando no muestra ninguna línea que empiece con `default`, no hay salida a internet configurada — ese es el problema a resolver primero.

▼ 2. ¿Hay conectividad básica? BAJO RIESGO

```
root # ping -c 3 1.1.1.1
```

Para qué sirve: confirma que hay salida real a internet. Se usa una **IP directa** (no un nombre de dominio) a propósito, para descartar de entrada cualquier problema de DNS y enfocarse solo en si hay conexión física/lógica.

▼ 3. ¿Funciona DNS + HTTPS? **BAJO RIESGO**

```
root # curl --location gentoo.org --output /dev/null
```

Para qué sirve: prueba en un solo paso que el nombre de dominio se resuelve (DNS) y que se puede descargar por HTTPS, que es justo lo que necesitará Portage más adelante.

Cómo interpretar los resultados

- **Todo funciona** → continuar directo con la preparación de discos.
- **ping funciona pero curl falla** → el problema es de DNS (se resuelve configurando `/etc/resolv.conf`, visto en un capítulo posterior).
- **Nada funciona** → hay que revisar la información de las interfaces de red, probar con `nmtui`, o configurar la red manualmente (próximos capítulos).

Para enriquecer: ¿por qué probar con IP antes que con dominio?

Es una técnica útil fuera de la instalación también: separar capas del problema. Si el `ping` a una IP falla, el problema está en la capa de red (cable, router, DHCP). Si el `ping` a la IP funciona pero `curl` a un dominio falla, el problema está aislado a DNS. Este tipo de diagnóstico por capas se aplica en casi cualquier troubleshooting de red, no solo en Gentoo.

Configuración Inalámbrica (WiFi)

Cómo conectar tu equipo a una red WiFi desde el medio de instalación

Si tu máquina se conecta a internet por **cable Ethernet**, este capítulo no aplica: el DHCP ya resolvió la conexión automáticamente. Este capítulo es solo para cuando dependes de **WiFi** para instalar Gentoo. Existen tres formas de lograrlo, de la más simple a la más manual.

NetworkManager **FÁCIL**

Menú interactivo. Recomendado para casi todos los casos.

net-setup **ALTERNATIVA**

Asistente por preguntas. Se usa cuando no hay NetworkManager.

Manual **AVANZADO**

Control total con wpa_supplicant / iwd. Para redes abiertas o casos especiales.

0 Antes de empezar: comprobar la tarjeta (opcional)

SIN RIESGO

Verifica que el kernel detecta tu tarjeta inalámbrica antes de configurar nada.

```
root # iw dev wlp9s0 info
```

Para qué sirve: confirma que el driver del WiFi está cargado. Si el nombre de tu interfaz no es wlp9s0 (puede ser wlan0 o ra0), revisa el nombre correcto con `ip link`.

1 Opción A — NetworkManager (recomendada)

SIN RIESGO

Paso único: ejecuta el asistente de texto y sigue el menú.

```
root # nmtui
```

Para qué sirve: abre una interfaz de menús donde eliges tu red y escribes la contraseña, sin comandos adicionales. Si usas la LiveGUI (versión con escritorio), basta con hacer clic en el ícono de red de la barra

de tareas.

2 Opción B — net-setup

Úsala solo si NetworkManager no está disponible en tu medio de instalación.

SIN RIESGO

Paso 1: detener el cliente DHCP activo para evitar conflictos.

```
root # killall dhcpcd
```

Para qué sirve: libera la interfaz de red para que net-setup pueda configurarla sin interferencias.

SIN RIESGO

Paso 2: lanzar el asistente sobre tu interfaz.

```
root # net-setup enp1s0
```

Para qué sirve: hace preguntas paso a paso (nombre de red, contraseña) y configura automáticamente wpa_supplicant o una IP fija según tus respuestas.

3 Opción C — Configuración manual

Para usuarios avanzados o redes abiertas sin contraseña.

SIN RIESGO

Paso 1: activar la interfaz inalámbrica.

```
root # ip link set dev wlp9s0 up
```

Para qué sirve: "enciende" la tarjeta a nivel de kernel; sin esto no puede detectar ni asociarse a ninguna red.

SIN RIESGO

Paso 2: para redes con contraseña (la mayoría), usar un daemon como wpa_supplicant o iwd para autenticarte. Para redes abiertas, con activar la interfaz suele bastar para asociarte al ESSID.

Para qué sirve: estos programas manejan el protocolo de seguridad WiFi (WPA2/WPA3). El handbook los explica a fondo en el capítulo de "Redes inalámbricas" aparte.

▼  **Comparación rápida de los tres métodos**

Método	Cuándo usarlo	Requiere contraseña manual
NetworkManager (nmtui)	Caso general, primera opción	La pide el menú
net-setup	Si no hay NetworkManager (ej. HPPA)	La pide el asistente
Manual (wpa_supplicant/iwd)	Redes abiertas o control total	Configuración directa

☆☆ Para enriquecer: cosas útiles que no siempre se mencionan

- **Verifica siempre después de conectar.** Haz `ping gentoo.org` (o cualquier dominio) para confirmar que hay internet real, no solo asociación WiFi.
- **El nombre de la interfaz cambia entre equipos.** Gracias al esquema de nombres predecibles de systemd, puede ser `wlp2s0`, `wlp3s0`, etc. — nunca asumas que será igual al de otra máquina.
- **Esta configuración es temporal.** Todo lo hecho aquí es solo para el medio de instalación (LiveCD/USB). Una vez dentro del sistema instalado, deberás configurar la red de nuevo con **netifrc** u otra herramienta permanente (ya cubierto en tu instalación).
- **Si algo falla**, el propio handbook recomienda volver a la configuración manual de red como respaldo — es la opción con más control cuando los asistentes no detectan bien el hardware.

Capítulo 10 — Configuración específica de aplicaciones

La "caja de herramientas" de red para cuando `nmtui` no basta: proxies web, ADSL por PPPoE y VPN por PPTP. Solo aplica si tu conexión no funciona con DHCP normal.

Capítulo original: 10 — Application specific configuration

Subsecciones: 10.1 - 10.3

¿CUÁNDO LEER ESTE CAPÍTULO?

Solo si tu red **no** se conectó sola al arrancar el medio de instalación. Si ya tienes internet (por ejemplo, con `nmtui` o por cable con DHCP), puedes saltarte todo este capítulo.

10.1 · Proxies web

Un **proxy** es un intermediario entre tu equipo e internet — común en redes de oficinas, universidades o instituciones. Si tu salida a internet pasa por uno, Portage (el gestor de paquetes) no sabrá llegar a los servidores de descarga a menos que se lo indiques.

PARA QUÉ SIRVE

Definir variables de entorno le dice a Portage y a otras herramientas de descarga (como el navegador de texto `links`) por dónde deben salir a internet.

Variable	Protocolo que cubre	Quién la usa
<code>http_proxy</code>	HTTP	Portage y <code>links</code>
<code>https_proxy</code>	HTTPS	Solo <code>links</code>
<code>ftp_proxy</code>	FTP	Portage y <code>links</code>
<code>RSYNC_PROXY</code>	rsync (sincronía del árbol de Portage)	Solo Portage

```
root # export http_proxy="http://proxy.gentoo.org:8080"
root # export https_proxy="http://proxy.gentoo.org:8080"
root # export ftp_proxy="ftp://proxy.gentoo.org:8080"
root # export RSYNC_PROXY="proxy.gentoo.org:8080"
```

▼ Si el proxy pide usuario y contraseña

Se incrustan directamente en la URL, antes del host:

```
root # export http_proxy="http://usuario:contraseña@proxy.gentoo.org:8080"
```

RIESGO MEDIO La contraseña queda visible en el historial de la terminal y en la lista de procesos mientras la variable está definida. Solo hazlo en un entorno que confíes (como el medio de instalación, que es temporal).

DATO CLAVE

Estas variables solo existen mientras dura la sesión de la terminal — se pierden si reinicias o cierras la ventana. No es un ajuste permanente todavía; eso se configura más adelante, ya con el sistema instalado.

10.2 · ADSL con PPPoE

Si tu proveedor de internet usa **ADSL** y requiere autenticarte con usuario/contraseña sobre Ethernet (protocolo **PPPoE**), el medio de instalación incluye un script guiado que hace casi todo el trabajo.

PARA QUÉ SIRVE

`pppoe-setup` genera automáticamente la configuración necesaria en `/etc/ppp/` para que tu conexión ADSL funcione, sin tener que editar archivos a mano.

```
root # pppoe-setup
root # pppoe-start
```

Durante el asistente te preguntará por:

- La interfaz Ethernet conectada al módem
- Usuario y contraseña del PPPoE
- IPs de servidores DNS
- Si quieres activar un firewall básico

▼ Si la conexión falla

- Revisa que las credenciales sean correctas en `/etc/ppp/pap-secrets` o `/etc/ppp/chap-secrets`.
- Confirma que elegiste la interfaz Ethernet correcta (la que está físicamente conectada al módem).

10.3 · VPN con PPTP

PPTP es un protocolo de VPN más antiguo. Si tu proveedor o tu red exige conectarte por PPTP para tener salida a internet, se usa el paquete `pptpclient`.

PARA QUÉ SIRVE

Establece un túnel cifrado hacia un servidor remoto; tu tráfico de red pasa por ese túnel en vez de

salir directo.

Configuración en tres pasos:

```
root # nano /etc/ppp/chap-secrets # usuario y contraseña
root # nano /etc/ppp/options.pptp # opciones adicionales, si aplica
root # pptp <IP del servidor>
```

NOTA DE SEGURIDAD

RIESGO MEDIO PPTP es un protocolo considerado inseguro para hoy en día (tiene vulnerabilidades conocidas de cifrado). Es útil como salida rápida de emergencia durante la instalación, pero no se recomienda como VPN permanente en producción — para eso existen alternativas modernas como WireGuard u OpenVPN.

En resumen

Escenario	Comando principal	Cuándo usarlo
Proxy corporativo	<code>export http_proxy=...</code>	Red institucional/oficina con proxy obligatorio
ADSL residencial	<code>pppoe-setup</code>	Módem ADSL con autenticación por usuario/contraseña
VPN de emergencia	<code>pptp <servidor></code>	Red que solo permite salida a través de un túnel PPTP

CONEXIÓN CON EL RESTO DEL CAPÍTULO

Si nada de esto aplica a tu caso — es decir, tu red simplemente no toma IP por DHCP — el siguiente paso no es este capítulo sino el **Capítulo 12** del manual: configuración manual y estática de IP (asignar dirección, ruta por defecto y DNS a mano).

Capítulo 11 — Conceptos básicos de Internet e IP

Teoría de redes necesaria para entender (y arreglar a mano) la conexión durante la instalación.

TEÓRICO

RIESGO: NINGUNO

NO REQUIERE COMANDOS OBLIGATORIOS

Para qué sirve este capítulo: si la red no se configuró sola (DHCP, NetworkManager, etc.), este capítulo te da el vocabulario y la lógica para configurarla tú mismo a mano en el siguiente capítulo.

1. Interfaces y direcciones

Una **interfaz** es simplemente el nombre lógico que el sistema le da a un dispositivo de red (tu tarjeta ethernet o WiFi). Para poder hablar con otros equipos necesita al menos una dirección IP asignada, aunque puede tener varias a la vez (por ejemplo IPv4 e IPv6 juntas).

⚠ Las IPs se pueden poner "a mano" libremente, así que si dos equipos usan la misma por error, hay un **conflicto de direcciones**. Por eso se prefiere que se asignen solas mediante:

Protocolo	Se usa en	Qué hace
DHCP	IPv4	Un servidor de la red le reparte una IP libre automáticamente
SLAAC	IPv6	El propio dispositivo se autoconfigura una dirección sin servidor

2. Redes y CIDR

Una vez que tienes una dirección, el sistema necesita saber qué otras direcciones están "cerca" de ti, es decir, en tu misma red local. Eso se expresa con la notación **CIDR**: un número después de una barra que indica el tamaño de la red.

EJEMPLO

192.168.0.2/24 se traduce así:

Tu dirección	192.168.0.2
Red a la que perteneces	192.168.0.0
Tamaño útil	254 equipos (192.168.0.1 – 192.168.0.254)
Dirección de broadcast	192.168.0.255

▼ ¿Cómo se calcula el tamaño de una red? (fórmula)

Tamaño = $2^{(32 - \text{CIDR})}$, y luego se le restan 2 direcciones (la de red y la de broadcast, que no se pueden usar en un equipo normal).

Los valores CIDR más comunes son /24 (254 equipos, típico de una red doméstica) y /32 (un solo equipo).

3. Internet: el router y la puerta de enlace

Para hablar con algo *fuera* de tu red local necesitas un **router**, que reenvía tu tráfico hacia afuera. La dirección de ese router dentro de tu red se llama **default route** o **gateway** (puerta de enlace). Por convención suele ser la primera o la última IP disponible de la red (ej. 192.168.0.1).

Interfaz con IP + CIDR → Habla con la red local → Envía lo externo al gateway → Internet

4. El sistema de nombres de dominio (DNS)

Recordar direcciones IP es incómodo, así que el **DNS** traduce nombres de dominio (como gentoo.org) a la IP real. En Linux, los servidores DNS que se van a usar se definen en el archivo:

```
/etc/resolv.conf
```

Dato útil: muchos routers también hacen de servidor DNS. Usar uno local (el del router o uno propio) suele acelerar las consultas gracias al caché, y mejora algo la privacidad.

Comandos de verificación

Aunque el capítulo es teórico, se apoya en dos comandos ya vistos para comprobar que todo lo anterior funciona:

```
root # ping -c 3 1.1.1.1
```

Para qué sirve: confirma que hay conectividad básica a Internet, probando contra una IP directa (sin depender del DNS).

```
root # curl --location gentoo.org --output /dev/null
```

Para qué sirve: prueba dos cosas a la vez: que el DNS resuelve el nombre y que la salida HTTPS funciona.

▼ + Extra para enriquecer: cómo leer un error rápido

Síntoma	Diagnóstico probable
ping 1.1.1.1 falla	No hay conectividad de red real: revisa cable/WiFi, interfaz e IP

ping 1.1.1.1 funciona, pero curl gentoo.org falla	El DNS no está resolviendo — revisa /etc/resolv.conf
Ambos funcionan	La red está lista, puedes continuar con la instalación

▼ + Extra: IPv4 vs IPv6, en una frase

IPv4 usa direcciones tipo 192.168.0.2 (4 números) y suele configurarse con DHCP; IPv6 usa direcciones más largas tipo fe80::ea40:f2ff... y normalmente se autoconfigura sola con SLAAC — casi nunca hace falta tocarla a mano.

Capítulo 12: Configuración manual de red con IP estática

Handbook de Gentoo AMD64 — Sección de instalación / redes

Este capítulo aplica en la práctica lo visto en la teoría de redes (interfaces, CIDR, rutas y DNS). Aquí se configura **a mano** una IP fija, la ruta de salida a internet y el servidor DNS, todo desde el **medio de instalación en vivo** — es decir, es temporal, sirve solo para tener internet mientras instalas. La configuración permanente del sistema final se hace más adelante (con `netifrc` o `systemd-networkd`).

1. IP local → 2. Ruta por defecto → 3. DNS → Internet funcionando

12.1 Configurar la dirección IP de la interfaz

Le asignas una dirección IP a tu tarjeta de red (aquí `enp1s0`) usando el comando `ip`, parte del paquete `sys-apps/iproute2`.

```
root # ip address add 192.168.0.2/24 dev enp1s0
```

Para qué sirve: le da identidad a tu equipo dentro de la red local. Sin esto, nadie puede "verte" en la red.

RIESGO BAJO — es completamente reversible, solo se pierde al reiniciar (vive en memoria, no se guarda en disco).

Cuidado: la IP se pone a mano, así que puede chocar con la de otro dispositivo ya conectado a la red (conflicto de IP). Revisa que el número que elijas esté libre.

Atajo: `ip a` es lo mismo que `ip address`.

12.2 Configurar la ruta por defecto (gateway)

Con solo el paso anterior, tu equipo ya puede hablar con otros dispositivos de **la misma red local**, pero no con internet. Falta decirle cuál es la "puerta de salida" (normalmente tu router).

```
root # ip route add default via 192.168.0.1
```

Para qué sirve: le dice al sistema "todo el tráfico que no sea para mi red local, mándalo por aquí" — es la puerta hacia internet.

Para revisar qué rutas están activas:

```
root # ip route
```

Atajo: `ip r` es lo mismo que `ip route`.

12.3 Configuración de DNS

El DNS traduce nombres como `gentoo.org` a direcciones IP. En Linux se define en `/etc/resolv.conf`.

```
root # nano /etc/resolv.conf
```

Para qué sirve: abre el archivo donde defines qué servidores DNS usar, para poder navegar por nombre y no solo por IP.

Dentro se agregan líneas `nameserver` (se consultan en el orden en que aparecen):

Proveedor	Contenido del archivo
Cloudflare	<code>nameserver 1.1.1.1</code> <code>nameserver 1.0.0.1</code>
Quad9	<code>nameserver 9.9.9.9</code> <code>nameserver 149.112.112.112</code>

Cuidado: si `dhcpcd` está corriendo, va a sobrescribir tus cambios a `resolv.conf` automáticamente. Verifica primero con `ps x | grep dhcpcd`.

Para confirmar que todo funciona:

```
root # ping -c 3 gentoo.org
```

Para qué sirve: si responde, tienes IP, ruta y DNS funcionando al mismo tiempo — luz verde para seguir con la partición de discos.

Extra: diagnóstico rápido si algo falla

El handbook lo menciona disperso en capítulos anteriores, pero conviene tenerlo junto como checklist de diagnóstico:

Comando	Qué revisa
<code>ping -c 3 1.1.1.1</code>	Conectividad básica a internet (por IP, sin depender de DNS)
<code>curl --location gentoo.org --output /dev/null</code>	Acceso HTTPS + que el DNS resuelva nombres
<code>ip link</code>	Si la interfaz de red existe y está activa (UP)

Lógica de diagnóstico: si el ping a `1.1.1.1` funciona pero `curl` a `gentoo.org` falla, el problema es DNS, no la conexión. Aísla el problema por partes en vez de adivinar.

▼ ¿Por qué esta config no sobrevive un reinicio?

Los comandos `ip address` e `ip route` modifican la configuración de red en memoria (kernel), no escriben nada en disco. Es ideal para el entorno live de instalación, pero para que la IP estática persista en el sistema ya instalado, se necesita configurarla en `/etc/conf.d/net` (con `netifrc` en OpenRC) o en un archivo `.network` (con `systemd-networkd`), pasos que se ven en capítulos posteriores del proceso de instalación.

▼ Repaso relámpago de CIDR (¿por qué /24?)

`/24` equivale a la máscara `255.255.255.0` y define cuántos equipos caben en la red: 254 direcciones usables. Es el tamaño más común para redes domésticas y de laboratorio. La fórmula general es $2^{(32-\text{CIDR})} - 2$.

Capítulo 2: Particionado del disco

Cómo dividir el disco antes de instalar Gentoo, y qué son UEFI, BIOS, GPT y MBR.

Conceptos base

MODERNO

UEFI / EFI

El firmware que enciende la máquina antes del sistema operativo. Busca un archivo de arranque dentro de una partición especial (ESP). Es el estándar actual.

LEGADO

BIOS

El firmware antiguo. No lee particiones ESP; arranca leyendo directamente el primer sector del disco (MBR). Común en equipos pre-2010.

MODERNO

GPT

Tabla de particiones moderna. Sin límite práctico de particiones, soporta discos enormes (>2 TiB). Se usa junto con UEFI.

LEGADO

MBR / DOS

Tabla de particiones antigua. Máximo 4 particiones primarias, límite de 2 TiB. Se usa junto con BIOS.

En resumen: UEFI+GPT es la pareja moderna recomendada. BIOS+MBR es la pareja legada, solo para hardware muy viejo o casos especiales (como algunas nubes tipo AWS).

Esquema de particiones recomendado (GPT + UEFI)

Partición	Punto de montaje	Sistema de archivos	Para qué sirve
/dev/sda1	/efi	vfat (FAT)	ESP: aquí el firmware UEFI encuentra el gestor de arranque
/dev/sda2	—	swap	Memoria virtual: extiende la RAM usando disco
/dev/sda3	/	ext4 / xfs / etc.	Raíz: contiene todo el sistema Gentoo instalado

/dev/sda — disco completo

ESP
/efi · 1G

swap
sin montar · 4G

root
/ · resto del disco

Nota sobre BIOS boot partition

Solo se necesita si el disco usa GPT pero el equipo arranca en modo BIOS legado con GRUB2. No aplica si se usa UEFI puro (nuestro caso) ni con tabla MBR.

Comandos de `fdisk`

Se abre la herramienta apuntando al disco:

```
fdisk /dev/sda
```

Para qué sirve: entra al modo interactivo de fdisk para ver, crear o borrar particiones del disco indicado.

Tecla	Acción
<code>p</code>	Ver la tabla de particiones actual
<code>g</code>	Crear tabla nueva tipo GPT (borra lo existente)
<code>o</code>	Crear tabla nueva tipo MBR/DOS (borra lo existente)
<code>n</code>	Crear una partición nueva
<code>d</code>	Borrar una partición
<code>t</code>	Cambiar el tipo de una partición
<code>a</code>	Marcar como booteable (solo en MBR)
<code>w</code>	Guardar los cambios en el disco
<code>q</code>	Salir sin guardar nada

Ver ejemplo completo paso a paso

```
fdisk /dev/sda
```

```
n → 1 → Enter → +1G      # partición 1: ESP (arranque)
t → 1                      # tipo: EFI System
```

```
n → 2 → Enter → +4G      # partición 2: swap
t → 2 → 19                # tipo: Linux swap
```

```
n → 3 → Enter → Enter    # partición 3: root, usa el resto del disco
t → 3 → 23                # tipo: Linux root (x86-64), opcional
```

w

guardar y salir

Riesgo: medio — hasta presionar **w** nada se escribe en el disco; se puede salir con **q** sin dañar nada.

En una frase:

Se divide el disco en 3 partes — una pequeña para el arranque (ESP), una para memoria virtual (swap), y una grande para el sistema (root) — usando **fdisk** con tabla GPT porque el equipo usa UEFI.

Capítulo 8 · Preparando los discos

Sistemas de archivos, activación de swap, y particionado GPT/UEFI vs MBR/BIOS

Ya tienes el espacio dividido en particiones (visto en el capítulo 2). Ahora toca dos cosas: **darle formato** a cada partición con un sistema de archivos, y **activar** la swap. También cubrimos aquí, como referencia, cómo se ve el particionado completo tanto en **GPT/UEFI** como en el esquema clásico **MBR/BIOS**, y una alternativa a `fdisk` : `cfdisk` .

🔗 **Para qué sirve este capítulo:** sin sistema de archivos, una partición es solo espacio vacío e inutilizable. Este paso convierte esas particiones en lugares donde Linux realmente puede guardar archivos.

1. ¿Qué es un sistema de archivos?

Es la forma en que el sistema organiza y guarda los datos dentro de una partición: cómo nombra archivos, cómo controla permisos, cómo sabe qué bloques del disco pertenecen a cada archivo. Crear la partición (con `fdisk` o `cfdisk`) es como delimitar un terreno; formatearla con `mkfs` es como construir los cimientos y las calles adentro.

Sistema	Comando	Uso típico
XFS	<code>mkfs.xfs</code>	Recomendado por el Handbook para <code>/</code> (root)
ext4	<code>mkfs.ext4</code>	Muy estable, ampliamente soportado
VFAT (FAT32)	<code>mkfs.vfat</code>	Obligatorio para la ESP en UEFI
btrfs	<code>mkfs.btrfs</code>	Snapshots, RAID integrado, compresión
F2FS	<code>mkfs.f2fs</code>	Pensado para USB / microSD (memoria flash)
ZFS	<code>zpool create</code>	Gestión avanzada de redundancia integrada

⚠ **Advertencia:** cualquier comando `mkfs` borra por completo lo que hubiera en esa partición. Verifica el nombre del dispositivo (`/dev/sdaX`) antes de ejecutar.

2. Particionado GPT + UEFI (tu configuración)

Riesgo medio — modifica la tabla de particiones

Partición	Punto de montaje	Sistema	Descripción
-----------	------------------	---------	-------------

/dev/sda1	/efi	vfat	EFI System Partition (ESP)
/dev/sda2	—	swap	Partición swap
/dev/sda3	/	xfs	Partición raíz

Formatear con mkfs

```
# ESP: siempre FAT32, obligatorio para que el firmware UEFI encuentre el bootloader
root # mkfs.vfat -F 32 /dev/sda1

# Raíz con XFS (recomendado)
root # mkfs.xfs /dev/sda3
```

🔔 **Para qué sirve la ESP:** es la partición donde UEFI busca el gestor de arranque (GRUB). Si no está en FAT32, el firmware simplemente no la reconoce y el sistema no arranca.

▼ ¿Necesito una BIOS boot partition en modo UEFI?

No. La **BIOS boot partition** (1-2 MB) solo es necesaria cuando el disco tiene tabla GPT pero el sistema arranca en modo **legacy BIOS** (mezcla poco común). En UEFI puro, o en MBR clásico, no se usa.

3. Particionado MBR + BIOS legacy (referencia)

Riesgo medio — modifica la tabla de particiones

Si el equipo es anterior a ~2010 o se necesita compatibilidad legacy, este es el esquema equivalente usando MBR/DOS en vez de GPT.

Partición	Punto de montaje	Sistema	Descripción
/dev/sda1	/boot	xfs	Partición de arranque (marcada * bootable)
/dev/sda2	—	swap	Partición swap
/dev/sda3	/	xfs	Partición raíz

Creación con fdisk (resumen)

```
root # fdisk /dev/sda
# crear tabla MBR nueva (borra todo lo existente)
Command (m for help): o
# nueva partición primaria /boot de 1G, marcada bootable con 'a'
Command (m for help): n
```

```
Command (m for help): a
# nueva partición swap de 4G, tipo 82
Command (m for help): n
Command (m for help): t
# partición raíz con el resto del espacio
Command (m for help): n
# guardar cambios
Command (m for help): w
```

Formatear (MBR/BIOS)

```
# /boot con cualquier sistema soportado por GRUB, XFS en este ejemplo
root # mkfs.xfs /dev/sda1

# raíz
root # mkfs.xfs /dev/sda3
```

Diferencia clave: en GPT/UEFI el bootloader vive en una partición FAT32 (`/efi`) que el firmware lee directamente. En MBR/BIOS, GRUB se instala en el **sector de arranque** del disco mismo, y `/boot` solo guarda el kernel y los archivos de configuración — por eso no necesita ser FAT32.

4. Alternativa a fdisk: cfdisk

Riesgo medio

`cfdisk` hace lo mismo que `fdisk` (crear/borrar/formatear particiones) pero con una interfaz visual de menús, en vez de comandos de una letra. Es más amigable para quien recién empieza.

🔗 **Para qué sirve:** reduce el riesgo de errores al escribir el comando equivocado, porque todas las acciones se seleccionan con las flechas del teclado y se confirman explícitamente.

```
root # cfdisk /dev/sda
```

Dentro de `cfdisk` , navega con las flechas `↑` `↓` y selecciona acciones con `Enter`:

- **[New]** — crea una partición nueva en el espacio libre (pide el tamaño, ej. `1G`)
- **[Type]** — cambia el tipo de partición (ej. "EFI System" o "Linux swap")
- **[Bootable]** — marca la partición como booteable (solo en MBR)
- **[Delete]** — elimina la partición seleccionada
- **[Write]** — guarda los cambios al disco (pide confirmar escribiendo `yes`)
- **[Quit]** — sale sin guardar si aún no se ha escrito

▼ fdisk vs cfdisk — ¿cuál usar?

	fdisk	cfdisk
Interfaz	Comandos de una letra (n, d, w...)	Menús visuales con flechas
Curva de aprendizaje	Requiere memorizar comandos	Más intuitivo
Scripting	Más fácil de automatizar	Pensado para uso interactivo
Disponibilidad	Siempre presente en el live media	Incluido en <code>sys-apps/util-linux</code> , también disponible

Ambos producen el mismo resultado final. El Handbook usa `fdisk` por defecto, pero `cfdisk` es perfectamente válido si te sientes más cómodo viendo la tabla de particiones de forma gráfica.

5. Activar la partición swap

Riesgo bajo — reversible

El swap no se "formatea" con `mkfs`; se inicializa con `mkswap` y luego se activa con `swapon`.

```
root # mkswap /dev/sda2
root # swapon /dev/sda2
```

🕒 **Para qué sirve el swap:** es espacio en disco que el kernel usa como "memoria RAM extra" cuando la RAM real se satura, moviendo ahí páginas de memoria poco usadas. Es más lento que la RAM, pero evita que el sistema se quede sin memoria.

6. ¿Qué son los inodos? (concepto clave)

Un **inodo** es la estructura donde el sistema de archivos guarda toda la información de un archivo *excepto* su nombre y contenido: dueño, permisos, fechas, y en qué bloques del disco están sus datos.

- El **contenido** del archivo son los datos reales en el disco.
- El **inodo** es la "ficha" con los metadatos del archivo.
- El **nombre** del archivo solo apunta a ese inodo.

Por qué importa: cada sistema de archivos tiene un número limitado de inodos, fijado al crearlo. Si se agotan (típico con muchísimos archivos pequeños), no se pueden crear archivos nuevos aunque sobre espacio en disco.

Particiones ext4 pequeñas

```
# reserva más inodos de lo normal – útil en particiones menores a 8 GiB
root # mkfs.ext4 -T small /dev/<dispositivo>
```

Resumen del capítulo

Paso	Comando	Riesgo
Formatear ESP (UEFI)	<code>mkfs.vfat -F 32 /dev/sda1</code>	⚠ medio
Formatear /boot (BIOS)	<code>mkfs.xfs /dev/sda1</code>	⚠ medio
Formatear raíz	<code>mkfs.xfs /dev/sda3</code>	⚠ medio
Activar swap	<code>mkswap + swapon /dev/sda2</code>	⚠ bajo
Particionar (alternativa visual)	<code>cgdisk /dev/sda</code>	⚠ medio

Capítulo 13 — Introducción a los dispositivos de bloque

El paso previo a particionar: cómo Linux nombra los discos y qué formatos de tabla de particiones existen.

13.1 · Dispositivos de bloque

Un **dispositivo de bloque** es la forma en que Linux representa un disco: una capa que permite leer y escribir en "bloques" de 4096 bytes sin importar la tecnología física (SATA, NVMe, USB, etc.). Cada tipo de conexión recibe un nombre distinto en `/dev/`.

Tipo de disco	Nombre en <code>/dev/</code>	Notas
SATA, SAS, SCSI, USB	<code>/dev/sda</code>	El más común. 1ª partición: <code>/dev/sda1</code>
NVMe (SSD por PCIe)	<code>/dev/nvme0n1</code>	Más rápido. 1ª partición: <code>/dev/nvme0n1p1</code>
MMC / eMMC / SD	<code>/dev/mmcblk0</code>	No recomendado para el sistema base
VirtIO (VM / QEMU)	<code>/dev/vda</code>	Solo en máquinas virtuales

 **Para qué sirve saber esto**

Te permite reconocer de inmediato, con solo ver `lsblk` o `fdisk -l`, qué disco es cuál antes de tocar algo — clave para no particionar el disco equivocado.

13.2 · Tablas de particiones: GPT vs MBR

Un disco casi nunca se usa "en bruto"; se divide en particiones. La tabla de particiones es el índice que describe dónde empieza y termina cada una.

	GPT	MBR / DOS
Año	Moderno	1983 (legacy)
Límite de particiones	Prácticamente ninguno	4 primarias (o 3 + extendida)
Tamaño máx. de disco	~8 ZiB	2 TiB
Copia de respaldo de la tabla	Sí, al final del disco	No — un solo punto de falla
Requiere para UEFI	Sí, casi obligatorio	No compatible

 **Ojo con el dual-boot**

Si usas GPT en una PC configurada en modo BIOS (no UEFI), Windows se niega a arrancar desde ahí. Verifica el modo de tu firmware antes de elegir.

Recomendación del Handbook: usar **GPT** siempre que se pueda. MBR solo tiene sentido en hardware

anterior a 2010 o algunos entornos de nube virtualizados.

13.3 · Almacenamiento avanzado

El medio de instalación también soporta, aunque no los cubre en detalle: `Btrfs` , `ZFS` , `LVM` y cifrado del disco raíz. Se mencionan como referencia para quien quiera ir más allá del esquema básico.

13.4 · Esquema de particionado por defecto

Partición	Sistema de archivos	Tamaño	Uso
<code>/dev/sda1</code>	fat32 (UEFI) / xfs (BIOS)	1 GiB	ESP o arranque
<code>/dev/sda2</code>	linux-swap	RAM × 2	Swap
<code>/dev/sda3</code>	xfs	Resto del disco	Raíz <code>/</code>

Herramientas: fdisk y parted

El capítulo solo las menciona; el uso real viene en el capítulo 15. `fdisk` funciona con MBR y GPT y es la que usa el Handbook. `parted` es una alternativa, más orientada a GPT desde el inicio.

Comandos internos de fdisk (modo interactivo)

Tecla	Acción	Riesgo
<code>p</code>	Imprime la tabla de particiones actual	BAJO
<code>n</code>	Crea una nueva partición	BAJO (en memoria)
<code>d</code>	Marca una partición para borrar	MEDIO
<code>t</code>	Cambia el tipo de partición	BAJO
<code>g</code>	Crea tabla GPT nueva (borra todo)	MEDIO (reversible con <code>q</code>)
<code>o</code>	Crea tabla MBR nueva (borra todo)	MEDIO (reversible con <code>q</code>)
<code>i</code>	Muestra info de una partición: UUID, Type-UUID	BAJO
<code>w</code>	Escribe los cambios al disco de verdad y sale	ALTO — irreversible
<code>q</code>	Salte sin guardar nada	BAJO — botón de pánico



Nada se guarda hasta `w`

Todo lo que haces con `n` , `d` , `t` , `g` u `o` vive solo en memoria. Si te equivocas, sal con `q` y el disco real queda intacto.

Ejemplo: crear el esquema GPT/UEFI paso a paso

-

```

root # fdisk /dev/sda

Command (m for help): g          # tabla GPT nueva

Command (m for help): n          # partición ESP
Partition number (1-128, default 1): 1
First sector: [Enter]
Last sector: +1G

Command (m for help): t          # marcarla como EFI System
Partition number: 1
Type: 1

Command (m for help): n          # swap
...
Command (m for help): n          # root
...

Command (m for help): p          # revisar antes de guardar
Command (m for help): w          # ¡guardar y salir!

```

Ver UUIDs

Dentro de `fdisk`, la tecla `i` muestra el UUID de una partición individual (solo en GPT). Pero para ver **todos** los discos de un jalón, el Handbook usa `blkid` fuera de `fdisk`:

```

root # blkid
/dev/sda1: PARTUUID="c12a7328-f81f-11d2-ba4b-00a0c93ec93b"
/dev/sda2: PARTUUID="0657fd6d-a4ab-43c4-84e5-0933c84b4f4f"
/dev/sda3: PARTUUID="1cdf763a-5b4c-4dbf-99db-a056c504e8b2"

```

Tipo de identificador	¿Qué identifica?	¿Cuándo cambia?
PARTUUID	La partición en sí	Solo si la recreas
UUID (filesystem)	El contenido / sistema de archivos	Al reformatear con <code>mkfs</code>

Para qué sirve esto

Estos identificadores son los que vas a usar más adelante en `/etc/fstab` en vez de `/dev/sda1`, porque no cambian aunque el orden de detección de discos cambie entre reinicios.

Capítulo 14 — Diseñando un esquema de particiones

Manual de Gentoo AMD64 · Fase de planeación (sin comandos ejecutables todavía)

Para qué sirve este capítulo: antes de tocar `fdisk`, hay que decidir cuántas particiones crear, de qué tamaño, y si el equipo necesita una ESP o una partición de arranque BIOS. Los comandos reales de creación vienen en los capítulos 15 (GPT/UEFI) y 16 (MBR/BIOS).

1. ¿Cuántas particiones y de qué tamaño?

No hay una fórmula fija; depende del uso del equipo:

- **Muchos usuarios** → separar `/home` (seguridad y backups más simples).
- **Servidor de correo** → separar `/var` (ahí viven los correos).
- **Servidor de juegos** → separar `/opt` (ahí suele instalarse ese software).

`/usr` y `/var` conviene dejarlos con espacio holgado: `/usr` guarda casi todas las aplicaciones y las fuentes del kernel; `/var` guarda el repositorio de ebuilds (~650 MiB) más las cachés de descargas y paquetes binarios, que crecen con el tiempo.

Ventajas de separar en varias particiones	Desventajas
Puedes elegir el mejor sistema de archivos para cada una	Si calculas mal los tamaños, una se llena mientras otra sobra
Un proceso descontrolado no llena todo el disco	Separar <code>/usr</code> exige <code>initramfs</code> para montarlo antes del arranque
Chequeos de disco más rápidos en paralelo	MBR + SCSI/SATA limita a 15 particiones (GPT no tiene ese límite)
Puedes montar como solo-lectura, <code>nosuid</code> , <code>noexec</code>	Más piezas que administrar

Recomendación para principiantes: no separes `/usr` en su propia partición — complica el arranque innecesariamente.

2. ¿Qué hay del espacio de swap?

El swap es espacio en disco que el kernel usa cuando la RAM se satura: mueve ahí páginas de memoria poco usadas (*swap out*) y las trae de vuelta cuando se necesitan (*swap in*, más lento que leer RAM directamente).

RAM instalada	Con suspensión	Con hibernación
2 GB o menos	4 GB	4 GB
2 a 8 GB	= cantidad de RAM	2 × RAM

8 a 64 GB	8 GB mín. - 16 GB máx.	1.5 × RAM
64 GB o más	8 GB mínimo	No recomendada

- Si quieres **hibernar**, el swap debe ser $\geq$ que la RAM total (ahí se guarda toda la memoria).
- Con varios discos, un swap por disco permite lectura/escritura en paralelo.
- Entre HDD y SSD, el swap rinde mejor en el SSD.
- Alternativa: un **archivo** de swap en vez de partición, útil con poco espacio.

3. ¿Qué es la ESP (EFI System Partition)?

Para qué sirve: es donde el firmware UEFI busca el cargador de arranque (bootloader) y el kernel. Sin ella, el equipo no puede arrancar en modo UEFI.

Obligatoria si el sistema arranca por **UEFI**. Debe formatearse en **FAT** (normalmente FAT32) — si no, el firmware no la reconoce y el sistema no arranca. No se necesita si arrancas en modo BIOS/Legacy.

4. ¿Qué es la partición de arranque BIOS?

Partición diminuta (1–2 MB) donde GRUB2 guarda datos que no caben en el espacio normal asignado. Solo aplica en un caso muy específico:

- Disco con tabla **GPT**, y
- Firmware arrancando en modo **BIOS/MBR legacy** (no UEFI).

No se necesita en modo UEFI ni con tabla MBR/Legacy DOS.

Extra: zram, un vecino útil del swap

Para qué sirve: es una forma de swap que vive comprimido dentro de la RAM en lugar de en disco — mucho más rápido, a cambio de usar algo de CPU.

Cómo funciona

El kernel crea un dispositivo virtual (`/dev/zram0`) dentro de la propia RAM. Los datos que normalmente irían a disco se comprimen (relación típica 2:1 o 3:1) y se guardan ahí mismo en memoria. Resultado: "escribir a swap" y "leer de swap" son operaciones de RAM, no de disco.

El detalle importante para instalaciones que compilan mucho

Compilar paquetes grandes puede consumir varios GB de RAM. Si en ese momento el sistema necesita usar zram, la CPU tiene que comprimir datos *al mismo tiempo* que está compilando — compite por el mismo recurso. Por eso se recomienda:

- Un zram **pequeño** ($\approx$ 512 MB) con **prioridad alta**, para absorber la presión normal de uso.
- Un swap en disco más grande como respaldo, para cuando la compilación realmente necesita más

memoria.

zram no sirve para hibernar

La hibernación necesita guardar toda la RAM en un almacenamiento que sobreviva sin energía. zram vive dentro de la RAM misma, así que se pierde al apagar el equipo. Para hibernar se necesita sí o sí una partición o archivo de swap real en disco.

▼ Ver cómo se usa en otras distros (fuera de Gentoo)

zram es parte del kernel de Linux, no algo exclusivo de Gentoo. Se usa ampliamente en:

- **Android** y **Chrome OS** — estándar desde hace años en equipos con poca RAM.
- **Fedora** (desde la versión 33) y **Ubuntu** (desde 22.04) — habilitado por defecto en vez de una partición de swap tradicional.

En distros con systemd suele configurarse con el paquete `zram-generator`:

```
# /etc/systemd/zram-generator.conf
[zram0]
zram-size = min(ram / 2, 4096)
compression-algorithm = zstd
```

O de forma manual, sin systemd:

```
modprobe zram num_devices=1
echo zstd > /sys/block/zram0/comp_algorithm
echo 4G > /sys/block/zram0/disksize
mkswap /dev/zram0
swapon -p 100 /dev/zram0
```

En una distro binaria (donde no compilas tu propio software) zram casi no tiene contras: solo absorbe la presión de memoria del uso diario. El caso de Gentoo es la excepción, porque tu propia máquina compila el sistema.

Capítulo 15 · Particionar el disco con GPT para UEFI

Guía práctica con `fdisk` para crear la ESP, la swap y la partición raíz — Manual de instalación de Gentoo AMD64

¿Qué se hace en este capítulo?

Aquí dejamos la teoría atrás y ejecutamos los comandos reales para dividir el disco en tres particiones usando `fdisk`, que funciona como un pequeño programa interactivo: lo abres, respondes preguntas de una letra, y al final confirmas para escribir los cambios de verdad.

Partición	Punto de montaje	Sistema de archivos	Función
<code>/dev/sda1</code>	<code>/efi</code>	vfat (FAT32)	ESP — donde el firmware UEFI busca el arrancador
<code>/dev/sda2</code>	—	swap	Memoria de intercambio en disco
<code>/dev/sda3</code>	<code>/</code>	xfs (o el que elijas)	Raíz del sistema

⚠ **Antes de empezar**

Todo lo que se escribe dentro de `fdisk` queda solo en memoria hasta presionar `w`. Pero una vez que guardas, cualquier dato previo en esas particiones se pierde. Confirma que `/dev/sda` es el disco correcto (usa `lsblk` antes de entrar) — sobre todo si hay más de un disco conectado.

Comandos paso a paso

RIESGO BAJO

```
root # fdisk /dev/sda
```

Para qué sirve: abre el disco en modo interactivo. No modifica nada todavía; es seguro entrar y salir sin guardar.

RIESGO BAJO

```
Command (m for help): p
```

Para qué sirve: imprime la tabla de particiones actual (tamaño del disco, tipo de disklabel, particiones existentes). Úsalo para verificar antes y después de cada cambio.

RIESGO MEDIO


```
Command (m for help): g
```

Para qué sirve: crea una tabla de particiones GPT nueva y vacía. Si el disco ya tiene una GPT que quieres conservar, en vez de esto borra particiones una por una con `d`.

RIESGO MEDIO

```
Command (m for help): n
Partition number (1-128, default 1): 1
First sector: [Enter]
Last sector: +1G
Command (m for help): t
Partition type or alias: 1
```

Para qué sirve: crea la partición 1 (1 GiB) y la marca como *EFI System*. Esta es la ESP; sin marcarla correctamente el firmware UEFI no encontrará el arrancador.

RIESGO MEDIO

```
Command (m for help): n
Partition number (2-128, default 2): 2
First sector: [Enter]
Last sector: +4G
Command (m for help): t
Partition type or alias: 19
```

Para qué sirve: crea la partición 2 como swap (aquí 4 GiB de ejemplo — ajusta el tamaño a tu RAM y necesidades). El tipo `19` es el alias de "Linux swap" en `fdisk` moderno.

RIESGO MEDIO

```
Command (m for help): n
Partition number (3-128, default 3): 3
First sector: [Enter]
Last sector: [Enter]
```

Para qué sirve: crea la partición 3 usando todo el espacio restante del disco — será la raíz (`/`). Dejar el "last sector" en blanco le dice a `fdisk` que use todo lo que sobra.

RIESGO ALTO

```
Command (m for help): w
```

Para qué sirve: escribe la tabla de particiones al disco físico y sale. Es el único paso que realmente modifica el disco — hasta este momento puedes salir con `q` sin guardar nada.

▼ ¿Qué es cada tecla que usamos dentro de fdisk?

- **p** — imprimir (print) la tabla de particiones actual
- **g** — crear un disklabel GPT nuevo
- **n** — nueva partición (new)
- **d** — borrar partición (delete)
- **t** — cambiar el tipo de una partición
- **w** — escribir cambios y salir (write)
- **q** — salir sin guardar nada (quit)
- **m** — mostrar la ayuda con todos los comandos

▼ ¿Por qué el tipo de la partición raíz no es obligatorio?

El manual aclara que marcar la partición 3 como "Linux root (x86-64)" (tipo **23**) es opcional. El sistema arranca igual si queda como "Linux filesystem" genérico. Ese tipo específico solo es necesario si usas un bootloader que lo requiere para identificar la raíz sin depender de `/etc/fstab` — por ejemplo `systemd-boot`. Como en tu instalación usarás **GRUB**, no es indispensable, pero no hace daño marcarlo igual.

▼ Alternativa: parted (no cubierta en el manual, pero existe)

parted es otra herramienta de particionado disponible en el medio de instalación, capaz de manejar GPT desde sus inicios. El manual prefiere **fdisk** por ser más estándar y estar disponible en casi cualquier entorno Linux, pero si alguna vez ves guías que usan **parted**, es simplemente una alternativa con la misma finalidad.

📖 Verificación recomendada (no es parte oficial del capítulo)

Antes de continuar con el siguiente capítulo (crear los sistemas de archivos), es buena práctica confirmar que las tres particiones quedaron como se esperaba:

```
root # lsblk /dev/sda
root # blkid
```

lsblk muestra el árbol de particiones y tamaños; **blkid** muestra los UUID y tipos de sistema de archivos (aunque en este punto todavía no tienen filesystem, solo tipo de partición).

→ Qué sigue

Con las particiones creadas pero todavía vacías, el siguiente paso (Capítulo 17 del manual) es formatearlas con **mkfs.vfat**, **mkswap** y **mkfs.xfs** (u otro filesystem elegido) para que realmente puedan almacenar datos.

personal de estudio.

Capítulo 16 — Particionando el disco con MBR para BIOS / Legacy Boot

Gentoo Handbook AMD64 · Resumen simplificado

Es la versión "clásica" del particionado, para equipos que arrancan en modo **BIOS/legacy** en vez de UEFI. Usa la tabla de particiones **MBR/DOS** en lugar de GPT. La lógica es la misma que en GPT (Cap. 15): una partición de arranque, una de swap y una raíz — solo cambia la herramienta interna que usa `fdisk` y que no existe una ESP.

¿Cuándo se usa MBR en vez de GPT?

Solo si tu placa madre es de antes de 2010 o no soporta UEFI, o si estás en un entorno de nube que aún depende de BIOS legacy (por ejemplo, algunas instancias de AWS). Fuera de esos casos, GPT es lo recomendado.

Limitaciones de MBR:

- No puede direccionar discos de más de 2 TiB.
- Máximo 4 particiones primarias (a menos que uses una extendida).
- No tiene copia de respaldo de la tabla de particiones: si se daña, se pierde todo.

Esquema de particiones

Partición	Punto de montaje	Sistema de archivos	Función
<code>/dev/sda1</code>	<code>/boot</code>	xfs	Arranque (reemplaza a la ESP de GPT)
<code>/dev/sda2</code>	—	swap	Memoria de intercambio
<code>/dev/sda3</code>	<code>/</code>	xfs	Raíz del sistema

Diferencia clave con GPT: aquí no existe la ESP (FAT32). En su lugar, `/boot` cumple ese papel y puede formatearse con cualquier sistema de archivos que tu bootloader soporte (por ejemplo XFS).

Pasos con `fdisk`

1. Abrir la herramienta sobre el disco

BAJO RIESGO

Para qué sirve: entra al modo interactivo de fdisk; todavía no modifica nada.

```
fdisk /dev/sda
```

2. Ver la tabla actual

BAJO RIESGO

Para qué sirve: muestra qué particiones existen antes de tocar nada.

Command (m for help): p

3. Crear tabla MBR nueva (borra todo)

ALTO RIESGO

*Para qué sirve: elimina cualquier partición previa y crea una etiqueta MBR limpia. Si prefieres conservar particiones existentes, usa **d** para borrarlas una por una en vez de esto.*

Command (m for help): o

4. Crear partición /boot (1 GB)

RIESGO MEDIO

Para qué sirve: reserva el espacio donde vivirá el kernel y el bootloader.

Command (m for help): n
Partition type: p (primaria)
Partition number: 1
First sector: 2048 (Enter, valor por defecto)
Last sector: +1G

5. Crear partición swap (4 GB)

RIESGO MEDIO

Para qué sirve: crea el espacio de intercambio y luego le asigna el tipo correcto (código 19 = Linux swap).

Command (m for help): n
Partition number: 2
First sector: (Enter)
Last sector: +4G

Command (m for help): t
Partition number: 2
Partition type: 19

6. Crear partición raíz (resto del disco)

RIESGO MEDIO

Para qué sirve: usa todo el espacio restante para el sistema principal.

Command (m for help): n
Partition number: 3
First sector: (Enter)
Last sector: (Enter, usa el resto del disco)

7. Guardar cambios y salir

ALTO RIESGO

*Para qué sirve: escribe la tabla de particiones al disco de forma definitiva. Antes de este paso, todo es reversible con **q**.*

Command (m for help): w

Un adelanto de lo que viene en el siguiente tramo del capítulo (creación de sistemas de archivos):

Formatear /boot y la raíz con XFS

RIESGO MEDIO

Para qué sirve: prepara las particiones para poder guardar archivos. Borra cualquier dato existente en ellas.

```
mkfs.xfs -c options=/usr/share/xfspgms/mkfs/lts_6.18.conf /dev/sda1
mkfs.xfs -c options=/usr/share/xfspgms/mkfs/lts_6.18.conf /dev/sda3
```

Activar el swap

BAJO RIESGO

Para qué sirve: inicializa la partición 2 como memoria de intercambio (se detalla en el capítulo siguiente).

```
mkswap /dev/sda2
swapon /dev/sda2
```

▼ ⚙️ Comparación rápida: MBR vs GPT

	MBR	GPT
Límite de tamaño de disco	2 TiB	Prácticamente ilimitado
Máx. particiones primarias	4 (o más con extendida)	128 por defecto
Backup de la tabla	No	Sí (copia al final del disco)
Partición de arranque	/boot normal	ESP en FAT32
Modo de firmware	BIOS / Legacy	UEFI

▼ ⚙️ ¿Por qué no lo necesitas en tu instalación?

Tu configuración usa GPT + UEFI (Capítulo 15), no MBR. Este capítulo existe como referencia por si en el futuro instalas Gentoo en hardware antiguo o en una VM que solo soporta BIOS legacy.

Capítulo 17 · Creación de sistemas de archivos

Manual de Gentoo AMD64 — resumen simplificado

Ya tienes las particiones creadas. Ahora toca darles formato: decidir cómo se van a organizar los datos dentro de cada una. Es como comprar un librero (la partición) y elegir cómo vas a acomodar los libros dentro (el sistema de archivos).

Sistemas de archivos disponibles

Sistema	¿Para qué sirve?
XFS	Recomendado por Gentoo como opción moderna de uso general. Soporta reflinks y Copy-on-Write, ideal si compilas mucho software.
ext4	Clásico y confiable, pero sin funciones modernas como reflinks.
VFAT (FAT32)	Obligatorio para la partición EFI en sistemas UEFI. No soporta permisos de Linux.
btrfs	Snapshots, autoreparación, compresión, RAID integrado. RAID 5/6 aún no es seguro.
F2FS	Pensado para memorias flash (USB, microSD).
NTFS	Solo para interoperar con Windows, no como raíz de Gentoo.
ZFS	Enfocado en integridad de datos y administración simple del almacenamiento.

▼ ¿Qué son los reflinks? (bonus)

Una copia de archivo casi instantánea que no ocupa espacio extra al crearse: el nuevo archivo apunta a los mismos datos que el original. Solo cuando modificas uno de los dos, el sistema copia (en ese momento) la parte que cambió — esto se llama **Copy on Write**. Es distinto a un *hard link*, que comparte los datos para siempre: el reflink se separa automáticamente en cuanto algo cambia, comportándose como una copia real pero con el ahorro de espacio de un enlace.

Comandos para formatear

Para qué sirve: cada sistema de archivos tiene su propio comando `mkfs.*` que "instala" ese formato en la partición.

Sistema	Comando
XFS	<code>mkfs.xfs</code>
ext4	<code>mkfs.ext4</code>
VFAT	<code>mkfs.vfat</code>
btrfs	<code>mkfs.btrfs</code>

F2FS	<code>mkfs.f2fs</code>
NTFS	<code>mkfs.ntfs</code>
ZFS	<code>zpool create ...</code>

Cuidado: cualquier comando `mkfs` borra todos los datos existentes en esa partición. No hay marcha atrás.

Partición EFI MEDIO

Debe ser FAT32 obligatoriamente si usas UEFI:

```
root # mkfs.vfat -F 32 /dev/sda1
```

Partición raíz con XFS ALTO

```
root # mkfs.xfs -c options=/usr/share/xfsprogs/mkfs/lts_6.18.conf /dev/sda3
```

ext4 en particiones pequeñas (<8 GiB) BAJO

Para qué sirve: la opción `-T small` cuadruplica el número de inodos disponibles (de 1 cada 16kB a 1 cada 4kB), útil si esa partición va a guardar muchos archivos pequeños.

```
root # mkfs.ext4 -T small /dev/<device>
```

🔥 Activar el swap

El swap no lleva un sistema de archivos como tal: se inicializa con `mkswap` y se activa con `swapon`.

```
root # mkswap /dev/sda2
root # swapon /dev/sda2
```

Para qué sirve: esta activación manual solo es necesaria ahora, en el entorno live. Tras instalar el sistema y reiniciar, el swap se activa solo si está bien definido en `/etc/fstab`.

▼ ¿Qué es exactamente el swap? (bonus)

Es espacio en disco que el sistema usa como memoria RAM de respaldo cuando la RAM real se llena. Es más lento que la RAM, pero evita que el sistema se caiga por falta de memoria. También se usa para la hibernación (guardar el estado de la RAM en disco al apagar).

✓ Resumen rápido

1. Elige el sistema de archivos según cada partición (XFS para raíz, FAT32 obligatorio para EFI).

2. Formatea cada partición con su `mkfs.*` correspondiente.
3. Si el ext4 va en una partición chica, usa `-T small` .
4. Inicializa y activa el swap con `mkswap` y `swapon` .

Montar la partición raíz

Conectar tu disco preparado con el sistema de instalación

Idea central: hasta ahora solo creaste y formateaste particiones, pero no las has "puesto en uso". Montar significa decirle al sistema: "todo lo que se guarde en esta carpeta, en realidad se guarda físicamente en esta partición del disco". Es como pegar una etiqueta de puerta ("Bodega 3") y luego conectar un túnel real hacia esa bodega.

1. Crear el punto de montaje

El punto recomendado para la raíz de Gentoo es `/mnt/gentoo`. Esta carpeta vive en el sistema **live** (el USB/CD desde el que instalas), no en tu disco todavía. Algunos entornos live no la traen creada:

```
mkdir --parents /mnt/gentoo
```

Para qué sirve: crea la carpeta (y cualquier carpeta intermedia que falte) donde luego "conectarás" el disco.

2. Montar la partición raíz

Con la carpeta lista, se conecta la partición real del disco (por ejemplo `/dev/sda3`) a esa carpeta:

```
mount /dev/sda3 /mnt/gentoo
```

Para qué sirve: desde este momento, todo lo que se copie dentro de `/mnt/gentoo` se escribe de verdad en el disco duro, no en la memoria RAM del live.

3. Solo si usas UEFI: preparar la carpeta para la ESP

Se crea la carpeta donde más adelante (capítulo 24, ya dentro del chroot) se montará la partición EFI. **Aquí todavía no se monta**, solo se prepara el espacio:

```
mkdir --parents /mnt/gentoo/efi
```

Para qué sirve: deja lista la carpeta "gancho". El `mount` real de la ESP viene después, justo antes de instalar el bootloader — no se necesita antes.

▼ **Nota opcional: si separaste `/tmp` en su propia partición** **BAJO RIESGO**

No es obligatorio para principiantes, pero si decidiste tener una partición aparte para

`/tmp` , después de montarla hay que ajustar sus permisos para que cualquier usuario pueda escribir temporalmente ahí:

```
chmod 1777 /mnt/gentoo/tmp
```

Lo mismo aplica si separaste `/var/tmp` . El permiso `1777` es el mismo que usa `/tmp` en cualquier sistema Linux normal: todos pueden escribir, pero solo el dueño de cada archivo puede borrarlo.

▼ `-p` VS `--parents` : ¿hay diferencia? SOLO REFERENCIA

Ninguna. `-p` es la forma corta de `--parents` ; hacen exactamente lo mismo. Sin esa opción, `mkdir` falla si la carpeta padre no existe todavía (por ejemplo, intentar crear `/mnt/gentoo/efi` sin que `/mnt/gentoo` exista antes). Con `-p` / `--parents` , se crean todas las carpetas intermedias necesarias sin quejarse.

▼ ¿Qué pasaría si no montaras la partición? IMPORTANTE ENTENDER

Si escribieras algo en `/mnt/gentoo` **antes** de montar la partición, esos datos quedarían guardados solo en la memoria RAM del sistema live — y desaparecerían al reiniciar o apagar. El comando `mount` es lo que hace que esa carpeta deje de ser "temporal" y pase a representar un espacio real y persistente en el disco duro.

En resumen

- Este capítulo solo hace una cosa: preparar `/mnt/gentoo` y conectar ahí la partición raíz con `mount` .
- La carpeta `/mnt/gentoo` vive en el sistema live; `mount` es el "túnel" hacia la partición real del disco.
- Para UEFI, solo se crea la carpeta `/mnt/gentoo/efi` aquí — el montaje real de la ESP se hace más adelante, en el capítulo 24.
- Siguiente paso: descargar y extraer el archivo stage dentro de `/mnt/gentoo` (capítulo 19-21), que gracias a este montaje quedará grabado en el disco de verdad.

Capítulo 19 — Eligiendo el Stage File

Compendio de instalación Gentoo · Sección: Base del sistema

En una frase: aquí decides el "molde" con el que va a nacer tu sistema Gentoo — qué sistema de init usará y si soportará software de 32 bits.

§¿Qué es un Stage File?

Es un archivo comprimido que contiene un sistema Gentoo casi completo, ya preparado por el equipo de Gentoo. Lo vas a descomprimir dentro de tu partición raíz y a partir de ahí construyes todo lo demás. No es más que el punto de partida — no hay comandos en este capítulo, solo una decisión.

¿Por qué importa elegir bien? CONTEXTO

Porque cambiar de opción *después* de instalado es posible pero complicado, sobre todo pasar de "sin soporte 32 bits" a "con soporte 32 bits". Vale la pena decidir con calma ahora.

§Eje 1: Sistema de init

El sistema de init es el proceso que arranca todos los servicios del sistema justo después de que carga el kernel.

Opción	Qué es	Ideal para
OpenRC	Sistema de init nativo y original de Gentoo. Basado en dependencias entre servicios.	Quien busca simplicidad y el enfoque tradicional de Gentoo
systemd	Init moderno usado por la mayoría de distros Linux actuales (Ubuntu, Fedora, Arch, etc.)	Quien ya conoce systemd de otras distros

§Eje 2: Soporte de arquitectura

Opción	Qué hace	Riesgo
Multilib	Usa librerías de 64 bits siempre que puede, y cae a 32 bits solo cuando algo lo necesita (ej. software o juegos viejos)	RECOMENDADO
No-multilib	Sistema 100% de 64 bits, sin ninguna librería de 32 bits	AVANZADO

⚠ **Advertencia del manual:** migrar de no-multilib a multilib después requiere conocimiento profundo de Gentoo y del toolchain de bajo nivel. Si eres principiante, evita no-multilib salvo que tengas una razón concreta para necesitarlo.

§Tip que enriquece este capítulo: el flavor "desktop"

Además de `init` y `multilib`, algunos stage files incluyen la palabra `desktop` en el nombre. Estos vienen con paquetes como `llvm` y `rust-bin` preinstalados y ajustes de USE flags pensados para acelerar la instalación si vas a usar entorno gráfico. Si tu plan final incluye escritorio (GNOME, KDE, etc.), es útil tenerlo en mente aunque la elección real de perfil ocurre más adelante.

▼ ¿Cómo se relaciona esto con el perfil (profile) de Portage?

El stage file está *basado* en un perfil de Portage, pero no son lo mismo. El stage file es el punto de partida binario; el perfil (que seleccionas más adelante con `eselect profile`) define configuraciones por defecto como USE flags y versiones de paquetes permitidas. Elegir bien el stage ahora hace que el perfil correspondiente ya venga alineado, evitándote trabajo extra.

🔧 Tu configuración

Según lo definido en el Capítulo 3, tu combinación es:

OPENRC

MULTILIB

Esto significa que en el siguiente capítulo (20) vas a buscar un stage file **sin** la palabra `systemd` ni `nomultilib` en el nombre — el estándar `amd64` multilib con OpenRC.

Capítulo 20 — Descargando el archivo stage

Ajustar el reloj del sistema, obtener el stage file y verificar que llegó íntegro y sin alteraciones.

PARA QUÉ SIRVE ESTE CAPÍTULO

Antes de instalar el sistema base necesitas dos cosas: que el reloj del equipo esté sincronizado (o las descargas por HTTPS pueden fallar) y descargar el archivo `stage3` correcto — la "semilla" de tu futuro sistema Gentoo. Este capítulo cubre ambos pasos, más la verificación de que el archivo no viene corrupto ni manipulado.

1. Ajustar la fecha y hora

Las descargas usan HTTPS, que depende de certificados con fecha de validez. Si el reloj del live environment está muy desfasado, la descarga puede fallar de forma confusa.



Revisar la hora actual **BAJO RIESGO**

```
root # date
```

Si el resultado está a más de unos minutos de diferencia con la hora real, corrígelo.



Opción automática — chrony **RECOMENDADA**

Sincroniza el reloj vía NTP contra servidores de internet. Es la forma más confiable:

```
root # chronyd -q
```

NOTA

El tráfico NTP estándar no está autenticado — es información pública, no un canal seguro, pero suficiente para este propósito.



Opción manual — sin acceso a NTP **BAJO RIESGO**

Formato: `MMDDhhmmYYYY` (mes, día, hora, minuto, año). Ejemplo, 3 de octubre 13:16 del 2021:

```
root # date 100313162021
```

Se recomienda usar hora **UTC**; la zona horaria real se configura después (Capítulo 27).

2. Descargar el stage file

Ya elegiste el tipo de stage (multilib + OpenRC, en tu caso) en el capítulo anterior. Ahora solo falta bajarlo.

▼ Con navegador gráfico **MÁS SIMPLE**

Copias el enlace desde la [página de descargas de Gentoo](#) y lo pegas en **wget** :

```
root # wget <URL_DEL_STAGE_FILE>
```

▼ Con navegador de texto (sin entorno gráfico)

Puedes navegar los mirrors directamente desde la terminal con **links** o **lynx** :

```
root # links https://www.gentoo.org/downloads/mirrors/
```

Navegas a **releases/amd64/autobuilds/** , eliges el stage correcto y presionas **d** para descargar.

Si necesitas proxy:

```
root # export http_proxy="http://proxy.server.com:port"
```

3. Verificar y validar el archivo **IMPORTANTE**

PARA QUÉ SIRVE

Confirma que el archivo descargado no está corrupto y que realmente lo publicó el equipo de Gentoo, no un tercero que lo alteró en el camino.

Archivo	Qué contiene
.CONTENTS.gz	Lista de todos los archivos dentro del stage.
.DIGESTS	Checksums del archivo en varios algoritmos (SHA256, SHA512, BLAKE2B).
.sha256	Checksum SHA256 firmado con PGP (no siempre disponible).



Verificar la firma criptográfica (GPG)

Primero importa la llave pública de Gentoo (si no lo hiciste ya en el Capítulo 4):

```
root # gpg --import /usr/share/openpgp-keys/gentoo-release.asc
```

Luego verifica la firma:

```
root # gpg --verify stage3-amd64-<release>-<init>.tar.xz.asc stage3-amd64-<release>-<init>.tar.xz
```

Si todo está bien, el resultado incluirá la línea `Good signature from .`



Verificar el checksum (alternativa/complemento)

```
root # sha256sum --check stage3-amd64-<release>-<init>.tar.xz.sha256.verify
```

Debe imprimir `OK`. Si dice `FAILED`, el archivo está corrupto o alterado — descártalo y vuelve a descargar.

SI ALGO FALLA

Nunca continúes la instalación con un stage file que no pasó la verificación. Un archivo corrupto puede causar errores difíciles de diagnosticar mucho más adelante en el proceso.

Contexto extra: ¿por qué existe este paso?

Un **stage file** es un archivo comprimido (`.tar.xz`) que contiene un sistema Gentoo casi completo — la base sobre la que se construye todo lo demás. Se genera con una herramienta interna de Gentoo llamada *Catalyst* y se actualiza con frecuencia, por eso no viene incluido en el medio de instalación y hay que descargarlo aparte.

La verificación con GPG y checksums no es paranoia innecesaria: es el mismo principio que usaste al verificar el ISO de instalación en el Capítulo 4. Es la única garantía de que lo que vas a extraer en `/mnt/gentoo` es exactamente lo que Gentoo publicó.

SIGUIENTE PASO

Con el stage file descargado y verificado, el Capítulo 21 lo extrae dentro de `/mnt/gentoo` con `tar`, y el Capítulo 22 configura las opciones de compilación (`make.conf`) antes de entrar al chroot.

Capítulo 21 — Instalando el archivo stage

Extraer el stage3 en tu partición raíz montada

¿Para qué sirve este paso? El stage3 es un paquete comprimido con un sistema Gentoo mínimo y funcional. Extraerlo en `/mnt/gentoo` es literalmente "sembrar" tu nuevo sistema operativo: todo lo que instales después se construye sobre esta base.

🔧 El comando principal

```
tar xpvf stage3-*.tar.xz --xattrs-include='*.*' --numeric-owner -C /mnt/gentoo
```

RIESGO MEDIO — si falla o se interrumpe, se puede repetir sin problema (a diferencia de instalar el bootloader)

▼ Ver qué hace cada bandera

Bandera	Qué hace
<code>x</code>	Extract — extrae el contenido del archivo
<code>p</code>	Preserva los permisos originales de cada archivo
<code>v</code>	Verbose — muestra cada archivo mientras se extrae
<code>f</code>	Indica que a continuación va el nombre del archivo
<code>--xattrs-include='*.*'</code>	Conserva atributos extendidos (metadatos adicionales de algunos archivos)
<code>--numeric-owner</code>	Mantiene los IDs numéricos de usuario/grupo tal cual, evitando líos de permisos
<code>-C /mnt/gentoo</code>	Cambia de directorio antes de extraer, para que todo caiga en la partición raíz

🔧 El truco del punto de montaje

`/mnt/gentoo` no es "un disco", es una **puerta de entrada** a tu partición real (por ejemplo `/dev/sda3`), gracias al `mount` que hiciste en el capítulo 18.

Cuando `tar` escribe con `-C /mnt/gentoo`, el kernel redirige esa escritura al disco físico montado ahí. Si no hubiera nada montado, todo se perdería al reiniciar, porque el entorno live vive en RAM.

✓ Verificación rápida

Después de extraer, puedes confirmar que el stage se desempaquetó correctamente:

```
ls /mnt/gentoo
```

Deberías ver carpetas como `bin`, `etc`, `usr`, `var`, etc. — la estructura típica de un sistema Linux.

Nota: El `*` en `stage3-*.tar.xz` es un comodín para no escribir el nombre completo (que incluye arquitectura, fecha e init system, ej. `stage3-amd64-openrc-20240107T170309Z.tar.xz`).

→ Siguiente paso

Con el sistema base ya extraído, el **capítulo 22** configura las variables de compilación (`CFLAGS`, `MAKEOPTS`, etc.) en `make.conf` antes de entrar al chroot.

← Cap. 20: Descargando el stage file

Cap. 22: Configurando opciones de compilación →

Capítulo 22 — Configurando las opciones de compilación

Qué son las variables de compilación, para qué sirven y cómo configurarlas correctamente en `make.conf`.

¿Dónde se configura todo esto?

Todo lo de este capítulo va en un solo archivo: `/mnt/gentoo/etc/portage/make.conf` (dentro del chroot, simplemente `/etc/portage/make.conf`). Este archivo lo lee **Portage** cada vez que compila cualquier paquete, así que los cambios aquí afectan a todo el sistema, no solo a la instalación inicial.

```
root # nano /mnt/gentoo/etc/portage/make.conf
```

Para qué sirve: abre el archivo de configuración principal de Portage con el editor `nano` para poder editar las variables.

1. CFLAGS y CXXFLAGS — compilador de C y C++

Le dicen a **GCC** (el compilador) cómo optimizar el código de cada programa en C (`CFLAGS`) y C++ (`CXXFLAGS`) al momento de compilarlo.

```
COMMON_FLAGS="-march=native -O2 -pipe"
CFLAGS="${COMMON_FLAGS}"
CXXFLAGS="${COMMON_FLAGS}"
```

Para qué sirve: define una sola vez el conjunto de flags (`COMMON_FLAGS`) y lo reutiliza en ambos compiladores, para no repetir código.

Flags disponibles y qué hace cada una

Flag	Qué hace	Recomendación
<code>-march=native</code>	Optimiza el código para el CPU exacto de la máquina donde compilas.	✓ Uso común
<code>-march=x86-64-v2</code> <code>/ v3 / v4</code>	Optimiza para un nivel de instrucciones x86-64 genérico (útil si el binario correrá en otra máquina distinta).	Alternativa a <code>native</code>
<code>-mtune=native</code>	Ajusta solo el "tuning" (orden de instrucciones), sin exigir instrucciones exclusivas del CPU. Más portable que <code>-march</code> .	Opcional
<code>-pipe</code>	Usa memoria RAM en vez de archivos temporales entre etapas del compilador. Compila algo más rápido.	RAM baja: evitar
<code>-fomit-frame-pointer</code>	Libera un registro extra del CPU al no guardar el "frame pointer".	Dificulta depuración

Niveles de optimización -O (letra O, no cero)

Flag	Qué significa
-00	Sin optimizar. Compila rápido, el programa corre lento. Útil solo para depurar.
-01	Optimizaciones básicas, seguras.
-02	Recomendado por el handbook. Buen balance velocidad/estabilidad.
-03	Máxima optimización de velocidad. Puede causar bugs en algunos paquetes si se usa en todo el sistema.
-0s	Optimiza para tamaño del binario, no velocidad. Útil en discos/RAM muy limitados.
-0g	Optimización mínima pensada para depurar con buena información de debug.

Advertencia del handbook: no metas flags experimentales aquí. Demasiada optimización puede hacer que los programas se comporten mal (crasheen o, peor, funcionen mal silenciosamente). Los valores por defecto que trae el stage3 ya son suficientemente buenos si no quieres tocar nada.

2. RUSTFLAGS — compilador de Rust

Es **independiente** de `CFLAGS` / `CXXFLAGS` : cada variable solo afecta al compilador de su propio lenguaje. Rust ya optimiza a nivel 3 por defecto, así que lo único que vale la pena añadir es la optimización para tu CPU específico:

```
RUSTFLAGS="${RUSTFLAGS} -C target-cpu=native"
```

Para qué sirve: el `${RUSTFLAGS}` al inicio conserva cualquier valor previo (por ejemplo, del profile de Gentoo) y le agrega `-C target-cpu=native` al final, en vez de sobrescribirlo.

```
root # rustc -C target-cpu=help
```

Para qué sirve: muestra la lista de CPUs que Rust reconoce, y cuál sería seleccionado con `native` en tu máquina.

Si dejas **solo** `RUSTFLAGS` sin tocar `CFLAGS` / `CXXFLAGS` , el sistema compila sin problema: los paquetes en Rust quedan optimizados a tu CPU, y los de C/C++ usan los valores por defecto del stage3.

3. MAKEOPTS — compilación en paralelo

Controla cuántos procesos de compilación corren **al mismo tiempo**.

```
MAKEOPTS="-j4 -l5"
```

Flag	Qué controla
-j4	Techo fijo: máximo 4 compilaciones en paralelo.

-l5

Freno dinámico: si el *load average* (carga del sistema) llega a 5, no lanza más procesos, aunque `-j` deje margen.

Regla práctica: usa el menor valor entre el número de hilos (`nproc`) de tu CPU, o tu RAM total dividida entre 2 GiB.

Sobre la memoria: no existe un flag de `MAKEOPTS` que limite RAM directamente (no hay un `-mMemoria`). La memoria se controla *indirectamente* bajando `-j` : menos procesos en paralelo = menos memoria usada a la vez. Regla del handbook: al menos 2 GiB de RAM por cada job (`-j6` requiere mínimo 12 GiB).

▼ ¿Y si necesito un límite duro de memoria? Avanzado

Portage/ `make` no lo ofrecen de forma nativa. Se lograría con herramientas externas del sistema como `cgroups` o `systemd-run --scope -p MemoryMax=...` , fuera del alcance de este capítulo.

Resumen: qué escribir al final

```
COMMON_FLAGS="-march=native -O2 -pipe"
CFLAGS="${COMMON_FLAGS}"
CXXFLAGS="${COMMON_FLAGS}"
RUSTFLAGS="${RUSTFLAGS} -C target-cpu=native"
MAKEOPTS="-j4 -l5"
```

Guarda con `Ctrl+O` y sal con `Ctrl+X` (en `nano`). Con esto, el capítulo 22 queda listo para pasar al capítulo 24 (Chrooting) — el 23 son solo referencias.

Capítulo 23 — Referencias

INFORMATIVO

Es el capítulo más corto del manual. No introduce comandos ni pasos nuevos: son las fuentes técnicas que respaldan lo explicado en el capítulo 22 sobre `MAKEOPTS`.

¿Qué hay aquí?

Dos enlaces a commits del repositorio de Portage en Gitweb, citados como respaldo de una afirmación puntual del capítulo anterior.

- **[1]** Commit que documenta que, a partir de Portage `3.0.31`, si `MAKEOPTS` se deja sin definir, Portage detecta automáticamente el número de hilos de la CPU (vía `nproc`) para el valor de trabajos paralelos (`-j`).
[gitweb.gentoo.org — commit 5d2af56...](https://gitweb.gentoo.org/commit/5d2af56...)
- **[2]** Commit que extiende ese comportamiento automático al valor de carga del sistema (`-l`, load-average) a partir de Portage `3.0.53`.
[gitweb.gentoo.org — commit de7be7f...](https://gitweb.gentoo.org/commit/de7be7f...)

¿Para qué sirve este capítulo? Le da trazabilidad técnica a lo dicho antes: no es una promesa vaga de "Portage es inteligente", sino que hay un commit real en el código fuente que implementa exactamente ese comportamiento.

Recordatorio rápido: ¿qué significa esto en la práctica?

REPASO

Si en tu `/etc/portage/make.conf` no defines `MAKEOPTS`, Portage no se queda sin instrucciones — usa automáticamente todos los hilos de tu CPU para compilar en paralelo (`nproc`) y ajusta el límite de carga del sistema en consecuencia.

El riesgo es que esto no considera tu RAM disponible. Por eso, si tu equipo tiene poca memoria en relación a sus núcleos, conviene fijar manualmente algo como:

```
MAKEOPTS="-j4 -l5"
```

Regla práctica: usa el **menor** valor entre el número de hilos de tu CPU y $(\text{RAM total} \div 2 \text{ GiB})$. Cada trabajo paralelo (`-j`) puede consumir hasta ~ 2 GiB de RAM en compilaciones pesadas (kernel, navegadores, etc.).

Dato adicional (no está en el manual): ¿por qué Gentoo cita commits en vez de documentación formal?

Gentoo es un proyecto muy transparente en cuanto a su desarrollo: gran parte de sus decisiones de comportamiento se documentan directamente en los mensajes de commit de

Portage, en lugar de en changelogs separados. Es una práctica común en proyectos de código abierto maduros — el código y su historial *son* la documentación autoritativa, y el wiki simplemente enlaza hacia esa fuente primaria en vez de duplicarla.

← Cap. 22: Configuración de compilación

Cap. 24: Chrooting →

Cap. 24 — Chrooting

Hasta ahora todo lo hiciste "desde afuera": particionaste discos y extrajiste los archivos del sistema base (stage3) mientras seguías parado en el medio de instalación (USB/CD en vivo). En este capítulo **entras** a ese sistema nuevo para trabajar directamente dentro de él, como si ya estuviera instalado. A esto se le llama **chroot** (*change root* — "cambiar la raíz").

🌐 24.1 — Copiar la información de DNS

Antes de entrar, copia el archivo que le dice a tu sistema qué servidores DNS usar. Sin esto, el nuevo entorno no tendría internet aunque el cable/wifi funcione.

```
root # cp --dereference /etc/resolv.conf /mnt/gentoo/etc/
```

Para qué sirve: asegura que el sistema nuevo pueda resolver nombres de dominio (como gentoo.org) a direcciones IP. `--dereference` copia el archivo real (no un enlace simbólico roto) por si `/etc/resolv.conf` apunta a otro lugar.

📁 24.2 — Montar los sistemas de archivos necesarios

El nuevo sistema necesita "ver" ciertas carpetas especiales del kernel para funcionar. Estas no se copian, se **enlazan** — son literalmente las mismas que las del sistema exterior.

Carpeta	Qué es	Por qué se necesita
<code>/proc</code>	Sistema de archivos virtual generado al vuelo por el kernel	Da información en tiempo real de procesos en ejecución (PID, memoria, CPU). Muchos programas la consultan para funcionar.
<code>/sys</code>	Similar a <code>/proc</code> , pero más estructurado	Expone información de hardware y dispositivos detectados por el kernel (discos, USB, red, etc.)
<code>/dev</code>	Archivos que representan dispositivos físicos	Sin esto no podrías acceder a discos (<code>/dev/sda1</code>), terminales, ni dispositivos aleatorios (<code>/dev/random</code>)
<code>/run</code>	Almacenamiento temporal en RAM	Guarda archivos de estado que solo importan mientras el sistema está encendido (PIDs de servicios, bloqueos, sockets)

```
root # mount --types proc /proc /mnt/gentoo/proc
root # mount --rbind /sys /mnt/gentoo/sys
root # mount --make-rslave /mnt/gentoo/sys
root # mount --rbind /dev /mnt/gentoo/dev
root # mount --make-rslave /mnt/gentoo/dev
root # mount --bind /run /mnt/gentoo/run
root # mount --make-slave /mnt/gentoo/run
```

▼ ¿Qué hacen `--rbind` y `--make-rslave`?

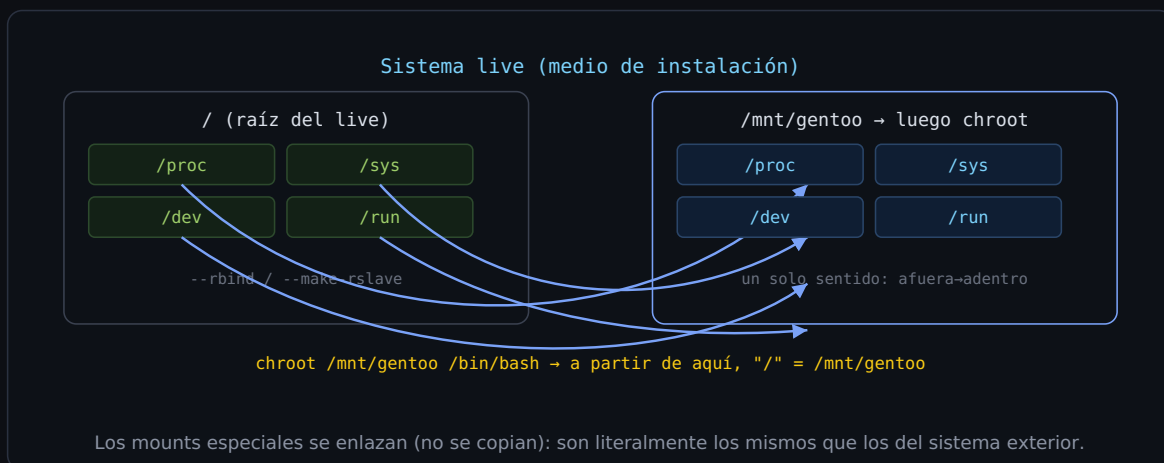
--rbind (bind recursivo): en vez de solo "asomarse" a una carpeta, replica también todos los sub-montajes que tiene adentro (por ejemplo, `/sys/fs/cgroup` suele ser su propio montaje separado). Sin la "r", esos sub-montajes quedarían invisibles dentro del chroot.

--make-rslave: por defecto, un bind mount queda conectado en *ambas direcciones* — lo que pase adentro del chroot también afectaría al sistema exterior. Esta opción lo vuelve de **un solo sentido**: los cambios de afuera se ven adentro (por ejemplo, un USB nuevo), pero lo que hagas adentro del chroot no puede romper el sistema exterior por accidente.

`/run` usa la versión sin "r" (`--bind / --make-rslave`) porque normalmente no tiene sub-montajes complejos anidados.

Si usas el medio oficial de instalación de Gentoo, todo este bloque de montajes (y el paso de copiar DNS) se puede reemplazar con un solo comando:

```
root # arch-chroot /mnt/gentoo
```



24.3 — Entrar al nuevo entorno

Aquí ocurre el chroot real, en tres pasos:

- 1 Cambiar la raíz del sistema de `/` (medio de instalación) a `/mnt/gentoo` (tu sistema nuevo)
- 2 Recargar las variables de entorno del sistema nuevo
- 3 Cambiar el prompt para no perder de vista que estás "adentro"

```
root # chroot /mnt/gentoo /bin/bash
root # source /etc/profile
root # export PS1="(chroot) ${'$'}{PS1}"
```

Para qué sirve: a partir de este punto, cada comando que ejecutes actúa directamente sobre el sistema nuevo, no sobre el medio de instalación. Es importante ver `(chroot)` en tu prompt para no confundir en qué sistema estás parado.

⚠ **Importante — rutas dentro del chroot:** una vez adentro, `/mnt/gentoo` deja de existir como tal. Lo que antes era `/mnt/gentoo/efi` ahora es simplemente `/efi`. Escribir la ruta larga desde dentro del

chroot dará error de "carpeta no encontrada".

Si la instalación se interrumpe después de este punto, puedes retomarla sin volver a particionar: monta la partición raíz, copia el DNS, repite los montajes, y vuelve a entrar al chroot. Muy útil también si necesitas arreglar el bootloader más adelante.

⚙️ 24.4 — Preparando el bootloader

Ya adentro del chroot, hay que montar la partición donde vivirá el bootloader. El comando depende del tipo de firmware de tu equipo:

Tipo de sistema	Partición	Se monta en	Bootloader típico
UEFI (moderno)	ESP, formateada en FAT32	<code>/efi</code>	GRUB (modo UEFI), systemd-boot
BIOS Legacy / MBR	Partición de arranque	<code>/boot</code>	GRUB (modo Legacy)

Sistemas UEFI (GPT)

```
root # mkdir --parents /efi
root # mount /dev/sda1 /efi
```

Para qué sirve: monta la ESP (EFI System Partition) donde el firmware UEFI buscará el bootloader al encender la máquina. El `mkdir` es necesario porque el `stage3` no siempre trae la carpeta `/efi` creada.

Sistemas BIOS Legacy (MBR)

```
root # mount /dev/sda1 /boot
```

Para qué sirve: en un equipo con BIOS heredado (sin UEFI), no existe ESP — el bootloader (GRUB en modo Legacy) se instala directamente en el MBR del disco y sus archivos de arranque viven en `/boot`.

Verificación recomendada tras montar, confirma que quedó correcto antes de seguir:

```
root # mount | grep -E "efi|boot"
```

▼ Comparación rápida: GPT+UEFI vs MBR+BIOS referencia

	GPT + UEFI	MBR + BIOS Legacy
Esquema de partición	GPT	MBR
Partición especial	ESP (FAT32)	No requiere partición dedicada (o partición boot legacy pequeña)
Punto de montaje	<code>/efi</code>	<code>/boot</code>
Límite de	Prácticamente	4 primarias (o 3 + extendida)

particiones	ilimitado	
Discos > 2TB	Soportado	No soportado

Este compendio usa **GPT + UEFI**, que es el estándar recomendado para equipos modernos.

Capítulo 25 — Configurando Portage

Handbook de Gentoo AMD64 · Preparando el gestor de paquetes antes de instalar software real

Contenido

- ¿Qué se logra en este capítulo?
- 1. Descargar el repositorio ebuild
- 2. Noticias de Gentoo
- 3. Elegir el perfil del sistema
- Los perfiles y los sistemas init (OpenRC, systemd y más)
- 4. Paquetes binarios (opcional)
- 5. La variable USE — a fondo
- CPU_FLAGS_X86 y VIDEO_CARDS
- 6. ACCEPT_LICENSE
- 7. El conjunto @world
- Resumen del flujo mínimo

¿Qué se logra en este capítulo?

Hasta el capítulo 24 el sistema es un esqueleto: tiene archivos base pero no sabe nada de software disponible. Aquí se le enseña a **Portage** (el gestor de paquetes de Gentoo) qué paquetes existen, qué perfil de configuración usar, y qué opciones de compilación aplicar por defecto. Es la puesta a punto antes de instalar kernel, bootloader y el resto del sistema.

Contexto importante: Gentoo es una distribución **basada en código fuente**. Cada `emerge` normalmente descarga el código fuente de un paquete y lo **compila en tu propia máquina**, no instala un binario prehecho (salvo que actives paquetes binarios, ver más abajo). Por eso toda esta configuración de Portage importa tanto: define cómo se comporta cada compilación, para siempre, no solo en la instalación inicial.

1. Descargar el repositorio ebuild BAJO RIESGO

Un **ebuild** es la "receta" que le dice a Portage cómo compilar e instalar un paquete. El repositorio ebuild es la colección completa de esas recetas — sin él, `emerge` no sabe que un paquete existe.

```
root # emerge-websync
```

Para qué sirve: descarga un snapshot diario del repositorio vía HTTP/FTP (funciona incluso detrás de firewalls restrictivos) y lo coloca en `/var/db/repos/gentoo/`.

Opcional — Actualizar el repositorio a la última hora (no solo al día)

```
root # emerge --sync
```

Para qué sirve: usa el protocolo `rsync` para traer cambios más recientes (hasta ~1 hora de antigüedad) en vez del snapshot diario de `emerge-webrsync`. Es totalmente opcional en la instalación inicial.

```
root # emerge --sync --quiet
```

Para qué sirve: igual que arriba, pero silenciando la salida — recomendado en terminales lentas (framebuffer, consola serial).

2. Noticias de Gentoo

Al sincronizar, Portage puede avisar de **news items**: comunicados oficiales sobre cambios importantes (por ejemplo, migraciones de perfil que requieren pasos manuales).

```
root # eselect news list
root # eselect news read
```

Para qué sirve: `eselect` es la herramienta genérica de Gentoo para gestionar distintos aspectos del sistema mediante "módulos" (aquí, el módulo `news`). `list` muestra los comunicados pendientes, `read` los muestra completos.

3. Elegir el perfil del sistema IMPACTO MEDIO

Un **perfil** es una plantilla mantenida por los desarrolladores de Gentoo que define valores por defecto de `USE`, `CFLAGS` y otras variables, además de fijar un rango de versiones de paquetes compatibles entre sí.

```
root # eselect profile list
```

Para qué sirve: muestra los perfiles disponibles para tu arquitectura, con un número al lado de cada uno.

```
root # eselect profile set 2
```

Para qué sirve: selecciona el perfil número 2 de la lista anterior como base de configuración del sistema.

Advertencia del propio handbook: al elegir el perfil inicial, usa la misma versión que trae tu stage file (por ejemplo, 23.0). Cambiar de versión de perfil más adelante no es trivial — siempre viene anunciado por un news item con instrucciones de migración.

Los perfiles y los sistemas init (OpenRC, systemd y más)

Algo que **no siempre es obvio** al mirar la lista de perfiles: el nombre del perfil determina qué **sistema init** (el proceso que arranca todo lo demás al encender la máquina) usará tu instalación. Gentoo es de las pocas distros que te deja elegir esto desde el principio.

Sistema init	Cómo se ve en el perfil	Notas
OpenRC	default/linux/amd64/23.0 (sin "systemd" en el nombre)	El init tradicional de Gentoo. Basado en scripts shell y dependencias declaradas simples. Servicios se gestionan con rc-update / rc-service . Es la configuración usada en este proyecto.
systemd	default/linux/amd64/23.0/systemd	Init más extendido en otras distros (Fedora, Arch, Debian). Gestiona servicios, logs (journald) y más como un conjunto integrado. Servicios con systemctl enable/start .
Otros (no cubiertos por perfiles oficiales)	—	Gentoo también permite runit o s6 vía overlays/paquetes de la comunidad, aunque no tienen un perfil oficial dedicado como OpenRC/systemd — requieren configuración manual adicional.

Nota: el capítulo 26 del handbook (opcional) cubre systemd en detalle. Si tu perfil no incluye "systemd" en el nombre, tu sistema queda en OpenRC — que es el caso de esta instalación.

Sub-perfiles no-multilib (solo 64-bit puro)

```
root # eselect profile list
...
[5] default/linux/amd64/23.0/no-multilib
root # eselect profile set 5
```

Para qué sirve: elimina la capacidad de correr binarios de 32-bit en un sistema 64-bit. Útil si nunca vas a necesitar compatibilidad con software antiguo de 32-bit.

4. Paquetes binarios (opcional)

AHORRA TIEMPO

Por defecto Portage compila todo desde código fuente. Si prefieres velocidad sobre optimización a medida, puedes usar binarios ya compilados por Gentoo (el "binhost oficial").

```
FEATURES="${FEATURES} getbinpkg"
FEATURES="${FEATURES} binpkg-request-signature"
```

Para qué sirve: le dice a Portage que, al pedir un paquete, primero pregunte al binhost si existe un binario compatible, y solo compile localmente si no lo encuentra. binpkg-request-signature exige que ese binario venga firmado criptográficamente.

```
root # getuto
```

Para qué sirve: configura el llavero (keyring) necesario para verificar esas firmas antes de instalar cualquier binario.

5. La variable USE — a fondo ALTO IMPACTO

Las **USE flags** son el mecanismo con el que decides qué funcionalidades opcionales se compilan dentro de cada paquete. A diferencia de distros binarias (que suelen incluir "todo por si acaso"), Gentoo te deja decidir tú.

Sintaxis básica

```
USE="ssl -X gnome gtk -kde -qt5"
```

- `ssl` → compila con soporte para cifrado SSL/TLS
- `-X` → el signo **menos** significa "sin esto" (sin servidor gráfico X11)
- `gnome gtk -kde -qt5` → favorece el ecosistema GNOME/GTK, descarta KDE/Qt

¿Por qué importan? Ejemplos concretos

Situación	Sin ajustar la flag	Con la flag ajustada
Servidor sin monitor	Se compilan decenas de paquetes con soporte X11/GUI innecesario ▼	<code>USE="-X"</code> ahorra tiempo de compilación y espacio en disco
Sistema de audio	Un reproductor podría traer PulseAudio aunque tu sistema use ALSA puro	<code>alsa -pulseaudio</code> evita instalar un demonio de audio que nunca arrancará
Seguridad / superficie de ataque	Más código compilado = más código que podría tener vulnerabilidades sin usarse nunca	Desactivar soporte no usado (ej. <code>-ftp -telnet</code> en un cliente) reduce esa superficie
Un solo paquete necesita algo distinto al resto	Cambiar el <code>USE</code> global afectaría a todo el sistema	<code>package.use/</code> permite la excepción solo para ese paquete

Importante: nunca instalas "sin" USE flags — siempre hay un conjunto activo (el del perfil, más lo que agregues en `make.conf`). La pregunta nunca es "¿uso USE flags o no?", sino "¿acepto las que trae el perfil por defecto, o las personalizo?".

¿Dónde se configuran? Global vs. por paquete

Global — afecta a todos los paquetes que reconozcan esa flag:

```
# /etc/portage/make.conf
USE="X gtk -kde alsa -pulseaudio"
```

Por paquete — sobrescribe la regla global, solo para ese paquete:

```
# /etc/portage/package.use/mpv
media-video/mpv pulseaudio -alsa
```

Para qué sirve `package.use/` : evita que `make.conf` crezca sin control. Es un directorio, no un archivo único — Portage lee todo lo que haya dentro, sin importar cómo lo organices (por paquete, por tema, etc.). Sigue la lógica de "regla general en `make.conf` , excepciones puntuales en `package.use/` ".

Advertencia del handbook: nunca uses `USE="-*"` (desactivar todo). Los desarrolladores de cada ebuild eligen valores por defecto para evitar conflictos y errores — desactivarlo todo puede romper el sistema de formas inesperadas.

USE flags de hardware: CPU_FLAGS_X86 y VIDEO_CARDS

Son dos variables especiales, ligadas a tu hardware físico, independientes entre sí.

	CPU_FLAGS_X86	VIDEO_CARDS
Hardware que describe	Tu procesador (CPU)	Tu tarjeta gráfica (GPU)
¿Es obligatorio configurarlo?	No, pero no tiene desventaja hacerlo	Prácticamente sí, si hay GPU dedicada o interfaz gráfica
Qué controla	Rutas de código especiales escritas a mano por cada paquete (cifrado, codecs de video) que aprovechan instrucciones concretas del CPU (AVX2, AES-NI, SSE4.2...)	Qué driver de GPU compilar en Xorg/Mesa/el kernel
Dónde se nota	Software de cómputo pesado: cifrado (OpenSSL), codificación de video (ffmpeg, x264), gráficos (Mesa)	Cualquier cosa que dibuje en pantalla o use aceleración de GPU

No confundir con `-march=native` : `CFLAGS="-march=native"` le dice al **compilador** que optimice automáticamente el código para tu CPU. `CPU_FLAGS_X86` activa rutas de código **manuales** escritas por los desarrolladores del paquete — son mecanismos distintos y complementarios, no sustitutos entre sí.

```
root # emerge --ask --oneshot app-portage/cpuid2cpuflags
root # cpuid2cpuflags
root # echo "*/ * $(cpuid2cpuflags)" > /etc/portage/package.use/00cpu-flags
```

Para qué sirve: `cpuid2cpuflags` detecta automáticamente qué instrucciones soporta tu CPU real (no hay que adivinarlo a mano) y genera el archivo `package.use` correspondiente, aplicado a todo paquete (`*/ *`) que use esa flag.

```
# /etc/portage/package.use/00video_cards
*/ * VIDEO_CARDS: - * amdgpu radeonsi
```

Para qué sirve: primero desactiva (`- *`) los drivers genéricos que trae el perfil por defecto, luego activa solo el tuyo — evita compilar soporte para tarjetas gráficas que no tienes.

Fabricante

Valor VIDEO_CARDS

AMD (reciente)	amdgpu radeonsi
NVIDIA (propietario)	nvidia
NVIDIA (open source)	nouveau
Intel	intel

6. ACCEPT_LICENSE

Controla qué licencias de software tu sistema tiene permitido instalar. Por defecto, solo se permite software agrupado en `@FREE` (libre, según la definición de la FSF/OSI).

```
user $ portageq envvar ACCEPT_LICENSE
@FREE
```

Para qué sirve: consulta el valor actual configurado en el sistema.

```
# /etc/portage/make.conf
ACCEPT_LICENSE="-* @FREE @BINARY-REDISTRIBUTABLE"
```

Para qué sirve: amplía lo permitido — aquí se agregan también licencias que permiten redistribuir binarios, aunque no sean 100% libres.

También se puede ajustar por paquete en `/etc/portage/package.license/`, igual que con `package.use/`.

7. El conjunto @world

Es la lista de **paquetes que instalaste explícitamente** — no las dependencias que se jalaron automáticamente. Vive en `/var/lib/portage/world`.

```
root # emerge --ask --verbose --update --deep --changed-use @world
```

Para qué sirve: actualiza todo lo que pediste explícitamente (y sus dependencias), recompilando lo necesario si cambiaron USE flags o el perfil.

```
root # emerge --ask --pretend --depclean
root # emerge --ask --depclean
```

Para qué sirve: detecta y elimina paquetes que ya nadie necesita (ni tú los pediste, ni son dependencia de algo que sí pediste). `--pretend` primero simula el resultado sin borrar nada — conviene revisarlo antes del `depclean` real.

Comparación con NixOS: `@world` se parece a `environment.systemPackages`, pero no es declarativo ni reproducible — es más un registro que Portage va actualizando según los comandos `emerge` que ejecutes, no un archivo que edites a mano para reconstruir el sistema desde cero.

Resumen del flujo mínimo

- 1 `emerge-webrsync` — obtener el catálogo de paquetes disponibles
- 2 `eselect profile list` / `set N` — elegir perfil (define también OpenRC vs. systemd)
- 3 (Opcional) ajustar `USE` , `CPU_FLAGS_X86` , `VIDEO_CARDS` y `ACCEPT_LICENSE`
- 4 (Opcional) `emerge --ask --verbose --update --deep --changed-use @world`

Capítulo 26

Opcional: Usar `systemd` como sistema y gestor de servicios

En una frase: este capítulo no da comandos — es un aviso de que, desde aquí, el manual mostrará dos caminos (`OpenRC` y `systemd`) cada vez que un paso sea diferente entre ambos.

¿Qué está pasando aquí? Sin comandos

Hasta este punto de la instalación (crear particiones, extraer el `stage3`, entrar al `chroot`, configurar `Portage`) el proceso es **idéntico** sin importar qué `"init system"` vayas a usar. A partir de este capítulo eso cambia: zona horaria, red, logging, cron, servicios de arranque, etc. tienen comandos distintos según el sistema elegido.

Es simplemente el punto donde el manual "bifurca" el camino. No hay que instalar ni ejecutar nada en este capítulo.

¿Qué es un "init system"?

Es el primer proceso que arranca el kernel de Linux (PID 1) y el encargado de levantar todos los demás servicios (red, logs, sshd, etc.) en el orden correcto. Es una pieza fundamental del sistema operativo.

OpenRC vs systemd — comparación rápida

Aspecto	OpenRC tu elección	systemd
Filosofía	Minimalista — hace solo una cosa (gestionar servicios)	"Todo en uno" — también maneja logs, red, temporizadores, montajes
Scripts de arranque	Shell scripts legibles y fáciles de auditar	Archivos de configuración tipo unit (<code>.service</code> , <code>.network</code>)
Comando principal	<code>rc-update</code> , <code>rc-service</code>	<code>systemctl</code>
Logging	Requiere un demonio aparte (ej. <code>sysklogd</code>)	Integrado (<code>journald</code> / <code>journalctl</code>)
Tareas programadas	Requiere <code>cron</code> (ej. <code>cronie</code>)	Integrado (<code>systemd timers</code>)
Consumo de recursos	Más ligero	Algo mayor por sus componentes extra
Uso en otras distros	Casi exclusivo de Gentoo y derivadas	Estándar en Fedora, Debian, Ubuntu, Arch
Recomendado por el manual para principiantes	Sí	Alternativa válida, más pasos propios

Para qué sirve saber esto: como ya elegiste `OpenRC` desde el Capítulo 3, este capítulo confirma que debes seguir la rama "OpenRC" cada vez que el manual presente dos columnas o dos bloques de comandos.

¿Por qué OpenRC para este proyecto?

Contexto

- Es la opción tradicional de Gentoo, con más documentación específica de la distro.
- Los scripts son shell puro, ideales para aprender qué hace cada servicio internamente (encaja con tu enfoque de estudiar cada paso).
- Menos "piezas" moviéndose por debajo del sistema — más fácil de depurar cuando algo falla.

▼ ¿Y si más adelante quiero cambiar a systemd?

Es posible, pero implica reinstalar varios componentes base (stage3 distinto, perfil de Portage distinto, y volver a configurar red, logging y servicios con las herramientas de systemd). No es un simple "cambio de interruptor" — el manual lo trata como una decisión que se toma antes de llegar a este punto, no después.

Zona horaria (Timezone)

Cómo decirle al sistema en qué parte del mundo está — un paso corto pero necesario

⚠ Este paso **no aplica** si tu sistema usa `musl libc` en vez de `glibc`. Si no sabes qué es eso, probablemente usas `glibc` y este capítulo sí te aplica.

¿Qué estamos haciendo aquí?

Linux necesita saber en qué zona horaria vive tu equipo para mostrar la hora local correcta (aunque internamente el reloj del sistema siga funcionando en UTC). Gentoo trae precargada una base de datos con **todas** las zonas horarias del mundo, guardada en `/usr/share/zoneinfo/`. Tu trabajo es simplemente decirle al sistema cuál de esas usar.

¿Para qué sirve esto? El sistema siempre consulta un solo archivo — `/etc/localtime` — para saber la hora local. En vez de copiar un archivo pesado ahí, se crea un **enlace simbólico** (como un acceso directo) que apunta a la zona horaria que elijas. Cambiar de zona horaria en el futuro es tan simple como rehacer ese enlace.

Paso 1 — Ver las zonas disponibles RIESGO BAJO

Listar continentes/regiones

```
root # ls -l /usr/share/zoneinfo
```

Listar ciudades dentro de una región (ejemplo: América)

```
root # ls -l /usr/share/zoneinfo/America/
```

Para qué sirve: solo estás mirando, no cambias nada todavía. Sirve para confirmar el nombre exacto del archivo que necesitas (mayúsculas, guiones bajos, etc.) antes de crear el enlace.

Paso 2 — Crear el enlace simbólico RIESGO MEDIO

Ejemplo con Bruselas (tú usarías tu propia zona)

```
root # ln -sf ../usr/share/zoneinfo/Europe/Brussels /etc/localtime
```

Para qué sirve: este comando es el que realmente configura la zona horaria. `-s` crea un enlace simbólico (no copia el archivo), y `-f` fuerza la sobrescritura si ya existía un enlace ahí. A partir de este momento, cada vez que el sistema pida la hora, seguirá este enlace hasta la zona elegida.

¿Por qué la ruta empieza con

../
?

El enlace se va a crear dentro de `/etc/`. Cuando usas una ruta relativa como `../usr/share/zoneinfo/...`, esa ruta se interpreta **desde la ubicación del enlace** (o sea, desde `/etc/`), no desde tu directorio actual de trabajo. Por eso el `../` "sube" un nivel desde `/etc/` hasta la raíz `/`, y de ahí baja a `/usr/share/zoneinfo/...`

¿Por qué no usar una ruta absoluta como `/usr/share/zoneinfo/...` directamente? También funciona, pero una ruta relativa es más portátil: sigue funcionando incluso si el sistema se monta bajo una raíz alterna (por ejemplo, si estás reparando el sistema desde un live USB y montas todo en `/mnt/gentoo`).

Cuidado con las zonas Etc/GMT*: sus nombres están invertidos por una rareza histórica del estándar POSIX. Etc/GMT-8 en realidad significa GMT+8, no GMT-8. Mejor evítalas por completo y usa siempre una zona geográfica real (ciudad/región), como `America/Monterrey` o `America/Mexico_City`.

Zonas horarias relevantes para México (Tamaulipas)

Zona horaria	Aplica a
America/Matamoros	Tamaulipas (sigue horario de EE. UU./frontera, DST incluido)
America/Monterrey	Zona centro de México (Nuevo León y la mayoría del país)
America/Mexico_City	Zona centro general (alias más reconocido)

Tamaulipas tiene una particularidad: algunos municipios  fronterizos usan el horario de la frontera (que sí observa horario de verano/DST), por eso existe la entrada específica `America/Matamoros`. Verifica cuál corresponde a tu municipio antes de crear el enlace.

Paso 3 — Verificar (opcional pero recomendado)

Confirmar que el enlace apunta a donde debe

```
root # ls -l /etc/localtime
```

Para qué sirve: te muestra hacia dónde apunta el enlace ahora mismo, así confirmas que quedó bien antes de seguir adelante.

¿Y si uso systemd en vez de OpenRC?

Con systemd existe la herramienta `timedatectl`, que hace lo mismo pero de forma más

guiada:

```
root # timedatectl list-timezones
root # timedatectl set-timezone America/Monterrey
```

Internamente crea el mismo tipo de enlace simbólico relativo en `/etc/localtime`, solo que valida el nombre por ti y evita que te equivoques al escribir la ruta a mano.

Nota: en tu instalación estás usando **OpenRC**, así que el método manual con `ln -sf` es el que te corresponde.

💡 En este capítulo no hay mucho paralelismo útil con NixOS más allá de que, ahí, la zona horaria se declara como una línea de texto en `configuration.nix` (`time.timeZone = "..."`) y el sistema genera el enlace por ti al aplicar la configuración. Aquí en Gentoo, tú creas ese enlace manualmente — mismo resultado, un paso más explícito.

| Resumen rápido

1) Busca tu zona en `/usr/share/zoneinfo/` → 2) crea el enlace con `ln -sf ../usr/share/zoneinfo/TU_ZONA /etc/localtime` → 3) evita las zonas `Etc/GMT*` → 4) verifica con `ls -l /etc/localtime`. Listo, siguiente capítulo: configuración de locales.

Configurar los locales

Idioma, formato de fecha y codificación de caracteres del sistema

Un **locale** no es solo "el idioma". Es el conjunto de reglas que le dicen al sistema cómo mostrar mensajes, cómo ordenar texto alfabéticamente, cómo formatear fechas, horas, números y moneda. Este capítulo tiene dos pasos: generar los locales que quieres tener disponibles y luego elegir cuál usar por defecto.

Nota: este paso no aplica si el sistema usa la librería musl libc. Si no sabes qué es eso, este paso sí te aplica normalmente.

28.1 — Generación de locales

Gentoo trae disponible un archivo con **más de 500 locales posibles**, pero casi nadie los necesita todos. Por eso se genera solo una lista corta, definida en un archivo de configuración.

Editar la lista de locales

RIESGO BAJO

```
root # nano /etc/locale.gen
```

Para qué sirve: abre el archivo donde declaras qué locales quieres tener disponibles en el sistema.

Dentro se escribe una línea por cada locale deseado, con el formato `idioma_PAÍS.codificación`:

/etc/locale.gen — ejemplo

```
en_US.UTF-8 UTF-8
es_MX.UTF-8 UTF-8
```

Para qué sirve: cada línea le dice al generador de locales cuáles compilar. Todo lo que no esté aquí, no se genera.

- Los nombres son **sensibles a mayúsculas y minúsculas**: deben escribirse exactamente como aparecen listados.
- La lista completa de locales soportados está en `/usr/share/i18n/SUPPORTED`.
- Se recomienda tener **al menos un locale en UTF-8**: muchos programas lo requieren para compilar correctamente.

Generar los locales elegidos

RIESGO BAJO

```
root # locale-gen
```

Para qué sirve: lee `/etc/locale.gen` y compila solo esos locales, ahorrando tiempo y espacio frente a generar los 500+ disponibles.

Verificar que quedaron disponibles

RIESGO BAJO

```
root # locale -a
```

Para qué sirve: lista todos los locales ya generados en el sistema, para confirmar que el paso anterior funcionó.

28.2 — Selección de locale

Ya generados, hay que decirle al sistema **cuál usar por defecto**. Para esto se usa `eselect`, la herramienta genérica de Gentoo para gestionar configuraciones del sistema, con su módulo `locale`.

Ver los locales disponibles

RIESGO BAJO

```
root # eselect locale list
```

Para qué sirve: muestra, numerados, todos los locales que ya fueron generados y pueden seleccionarse como el de la variable `LANG`.

▼ ¿Qué significa cada entrada de la lista?

Entrada	Significado
<code>C</code>	Locale neutro/mínimo. Mensajes en inglés genérico, sin acentos, sin formato regional.
<code>C.UTF-8</code>	Igual que <code>C</code> , pero soporta caracteres UTF-8 (acentos, ñ, símbolos).
<code>POSIX</code>	Alias histórico de <code>C</code> , mismo comportamiento predecible.
<code>de_DE.UTF-8</code>	Formato completo: idioma_PAÍS.codificación — aquí, alemán de Alemania en UTF-8.
<code>es_MX.UTF-8</code>	Español de México en UTF-8 — el que aplica a esta instalación.
<code>(free form)</code>	No es una opción numerada; permite escribir a mano un locale que no salió listado.

Seleccionar el locale a usar

RIESGO BAJO

```
root # eselect locale set <NÚMERO>
```

Para qué sirve: fija el locale elegido como el valor de la variable `LANG` del sistema.

Recargar el entorno

RIESGO BAJO

```
root # env-update && source /etc/profile && export PS1="(chroot) ${PS1}"
```

Para qué sirve: aplica el cambio de locale de inmediato en la sesión actual, sin esperar a un reinicio.

▼ Alternativa manual (sin `eselect`)

También se puede fijar el locale editando directamente el archivo de entorno:

```
/etc/env.d/02locale
```

```
LANG="es_MX.UTF-8"  
LC_COLLATE="C.UTF-8"
```

En sistemas con **systemd**, el archivo equivalente es `/etc/locale.conf`, y en vez de `eselect` se puede usar `localectl set-locale` o `localectl list-locales`.

Por qué importa este paso: configurar bien el locale evita advertencias y errores durante compilaciones de software más adelante en la instalación, y asegura que fechas, números y mensajes se muestren correctamente desde el primer arranque.

Resumen rápido

1. **Generar:** editar `/etc/locale.gen` → correr `locale-gen`
2. **Seleccionar:** `eselect locale list` → `eselect locale set N`
3. **Recargar:** `env-update && source /etc/profile`

Fuente: Gentoo Handbook AMD64 — Capítulo 28: Configure locales.

Capítulo 29 — Referencias Sin comandos

Handbook de Gentoo AMD64 · Cierre del capítulo 28 (Configuración de locales)

¿Para qué sirve este capítulo? No es un paso de instalación — es solo la bibliografía que respalda lo explicado en el capítulo de *locales*. Aquí no se ejecuta nada en la terminal; es información de contexto para quien quiera profundizar.

Las dos fuentes citadas

- **Anuncio de paquetes binarios de Gentoo** (dic. 2023)
gentoo.org/news/2023/12/29/Gentoo-binary.html
Anuncia que Gentoo empezó a ofrecer binarios oficiales precompilados (además del modelo tradicional de compilar todo desde código fuente).
- **GLEP 23** — propuesta de mejora sobre licencias
gentoo.org/glep/glep-0023.html
El documento que define cómo Gentoo agrupa las licencias de software y cómo funciona la variable `ACCEPT_LICENSE`.

Contexto extra — ¿por qué importa el GLEP 23?

Adelanto: Vas a usar esta idea más adelante, en la configuración de Portage. GLEP significa *Gentoo Linux Enhancement Proposal* (como un "RFC" interno de Gentoo).

La idea central es simple: Gentoo tiene miles de paquetes, y cada uno puede traer una licencia distinta (GPL, MIT, propietaria, etc.). En vez de que revises licencia por licencia, Gentoo las agrupa en **grupos de licencias** (por ejemplo `@FREE` para software libre). Así puedes decirle al sistema algo como "solo instala software con licencia libre" con una sola línea en `ACCEPT_LICENSE`, en lugar de aprobar cada paquete manualmente.

▼ Analogía con NixOS

Es parecido al flag `allowUnfree` de NixOS: en vez de aprobar paquete por paquete, defines una política general (permitir/rechazar cierto tipo de licencias) y el gestor de paquetes la aplica automáticamente en cada instalación.

Resumen en una frase

Capítulo informativo, sin acción requerida — solo enlaces de respaldo al tema de locales y un adelanto de las licencias que verás en la configuración de Portage.

30. Instalando Firmware y Microcódigo

Capítulo corto: solo hay que instalar los paquetes correctos para que tu hardware funcione.

En una frase: el kernel de Linux sabe *hablarle* a tu hardware, pero muchos dispositivos (WiFi, GPU, e incluso el CPU) necesitan un archivito extra de "instrucciones de fábrica" para arrancar correctamente. Eso es el firmware/microcódigo.

30.1 Firmware RIESGO BAJO

El firmware vive normalmente en un chip de la fábrica del dispositivo, pero muchos fabricantes esperan que el sistema operativo lo cargue en cada arranque. En Gentoo, ese firmware no viene incluido — hay que instalarlo aparte.

30.1.1 Linux Firmware (*sugerido para casi todos*)

Paquete "todo en uno" que trae firmware para la mayoría de tarjetas WiFi, GPUs y otros dispositivos comunes.

```
root # emerge --ask sys-kernel/linux-firmware
```

Para qué sirve: sin esto, tu WiFi o tu GPU pueden simplemente no encender, aunque el driver del kernel esté cargado.

este paquete casi siempre pide
⚠️**Ojo:** aceptar licencias no-libres. Se `/etc/portage/package.license/` o `ACCEPT_LICENSE` en `/etc/portage/make.conf`.
configura en

▼ 30.1.1.1 Carga de Firmware (detalle técnico)

El firmware se carga normalmente cuando el kernel activa el módulo del dispositivo. Si compilas ese módulo **integrado** al kernel (`Y`) en lugar de como módulo separado (`M`), el firmware debe estar disponible vía `CONFIG_EXTRA_FIRMWARE` desde el arranque mismo — si no, la carga puede fallar o complicarse.

Esto conecta directo con la decisión que tomarás en el Capítulo 32 al configurar el kernel: *built-in vs módulo* no es solo preferencia, tiene efectos reales.

30.1.2 SOF Firmware (*solo si tu audio lo requiere*)

Firmware de audio abierto (reemplaza al propietario SST de Intel). Lo necesitan CPUs Intel 10ma gen+ y algunas APUs AMD.

```
root # emerge --ask sys-firmware/sof-firmware
```

30.2 Microcódigo REVISAR TU CPU

No es firmware de un periférico — es firmware **del propio CPU**. Corrige bugs de estabilidad, seguridad o comportamiento a nivel de hardware del procesador.

Fabricante	¿Dónde está?	Comando
AMD	Ya incluido en <code>linux-firmware</code>	<i>nada extra que hacer</i>
Intel	Paquete separado	<code>emerge --ask sys-firmware/intel-microcode</code>

Para qué sirve: el microcódigo parchea el CPU en cada arranque (antes de que el kernel termine de iniciar) contra fallos de hardware — algunos son de estabilidad, otros de seguridad (piensa en vulnerabilidades tipo Spectre/Meltdown, corregidas parcialmente así).

▼ + Contenido extra: cómo verificar qué CPU tienes (no está en el Handbook, pero te ahorra dudas)

Antes de decidir si necesitas `intel-microcode`, confirma tu fabricante desde la terminal chroot o el live media:

```
root # grep "model name" /proc/cpuinfo | head -1
```

Si dice `Intel`, instala `intel-microcode`. Si dice `AMD`, ya quedó cubierto por `linux-firmware`.

▼ + Contenido extra: ¿cómo se activa el microcódigo en el arranque?

El Handbook no lo detalla aquí, pero vale saberlo: el microcódigo se carga de dos formas posibles —

- **Early loading (recomendado):** el bootloader (GRUB) lo mete al `initramfs` antes de que arranque el kernel. Con `dracut` (tu caso) esto es automático una vez instalado el paquete correcto — no necesitas configurar nada extra.
- **Late loading:** el kernel lo aplica ya arrancado, vía `sysfs`. Es menos ideal porque el CPU corre unos segundos sin el parche.

Como ya definiste `dracut` como generador de `initramfs`, el microcódigo se integra solo — es una de las ventajas de esa combinación.

🔪 Resumen: ¿dónde se configura todo?

Qué	Dónde
Instalar paquetes de firmware	Comandos <code>emerge --ask</code> (no hay archivo de config propio)

Aceptar licencias no-libres	<code>/etc/portage/make.conf</code> (<code>ACCEPT_LICENSE</code>) o <code>/etc/portage/package.license/</code>
Firmware built-in al kernel	Opción <code>CONFIG_EXTRA_FIRMWARE</code> en la configuración del kernel (Cap. 32)
Carga temprana de microcódigo	Automática vía <code>dracut</code> al regenerar el <code>initramfs</code> — sin archivo que tocar

Capítulo 31 — sys-kernel/installkernel

Handbook de Gentoo AMD64 · Preparando el "repartidor" del kernel antes de instalarlo

Contenido

¿Qué es installkernel? · ¿Dónde se configura? · 31.1 Elegir el bootloader · 31.2 Initramfs · 31.3 Unified Kernel Image · Configuración recomendada · Preguntas frecuentes

¿Qué es installkernel?

Es el paquete que actúa como **repartidor automático** del kernel. Cuando más adelante emerjas el kernel de verdad (capítulo 32), alguien tiene que:

- Copiar la imagen del kernel a `/boot` o `/efi` (según el bootloader)
- Generar el initramfs si hace falta
- Avisarle al gestor de arranque que hay un kernel nuevo

Ese trabajo lo hace `installkernel`, y su comportamiento se define **antes** de instalar el kernel, mediante USE flags.

Para qué sirve este capítulo: no instala el kernel — solo prepara el mecanismo que lo va a "enganchar" correctamente con tu bootloader e initramfs cuando lo instales en el siguiente capítulo.

¿Dónde se configura?

Todo pasa por archivos dentro de `/etc/portage/package.use/`, que es un **directorio**, no un archivo único. Portage lee todos los archivos que haya adentro y los combina.

Archivo (nombre libre, por convención)	Contenido
<code>/etc/portage/package.use/installkernel</code>	<code>sys-kernel/installkernel</code> <code>grub</code> <code>dracut</code>
<code>/etc/portage/package.use/uki</code>	Flags relacionadas a Unified Kernel Image (opcional)

Si el archivo aparece vacío al abrirlo con `nano`, es normal: lo estás **creando** tú mismo, no viene pre-poblado como `make.conf`.

31.1 Bootloader — elegir el mecanismo de arranque DECISIÓN

Esta es la decisión más importante del capítulo: define **quién** arranca tu kernel al encender la máquina. No instala nada del bootloader todavía (eso es el capítulo 44) — solo le dice a `installkernel` para cuál de ellos debe preparar los archivos.

Opción	Qué es	USE flags	Copia el kernel a
--------	--------	-----------	-------------------

GRUB	El más flexible y documentado. Funciona en BIOS y UEFI. Recomendado si no estás seguro.	<code>grub</code>	<code>/boot</code>
systemd-boot	Más simple, solo UEFI. Funciona con OpenRC o systemd. Buen soporte de Secure Boot.	<code>systemd</code> <code>systemd-boot</code>	<code>/efi</code>
EFI stub	El kernel arranca directo desde el firmware UEFI, sin intermediario. Minimalista pero rígido.	<code>efistub</code> (testing)	<code>/efi</code>
Tradicional (lilo, syslinux)	Layout clásico de <code>/boot</code> , para bootloaders antiguos o no-UEFI.	ninguno de los anteriores	<code>/boot</code>

¿Por qué "solo una selección es necesaria"?

—

Porque cada opción configura `installkernel` para hablar con un bootloader distinto. Elegir GRUB y systemd-boot al mismo tiempo no tiene sentido — son mecanismos que compiten entre sí, no que se complementan. Puedes cambiar de opinión después reinstalando `installkernel` con otras flags.

Para qué sirve la flag `grub` específicamente: hace que cada vez que se instale un kernel nuevo, se ejecute automáticamente `grub-mkconfig` por ti — sin ese flag tendrías que regenerar la configuración de GRUB a mano cada vez.

31.2 Initramfs CASI SIEMPRE NECESARIO

Es un mini sistema de archivos temporal en RAM que carga los módulos necesarios (por ejemplo, el driver de tu disco) **antes** de que el kernel pueda montar tu partición raíz real.

Para qué sirve: sin él, si tu driver de almacenamiento es un módulo (lo normal en un kernel de distribución), el kernel arranca pero no encuentra cómo montar `/`, y el sistema no bootea.

```
# /etc/portage/package.use/installkernel
sys-kernel/installkernel dracut
```

```
emerge --ask sys-kernel/installkernel
```

31.3 Unified Kernel Image (opcional) OPCIONAL

Empaqueta kernel + initramfs + parámetros de arranque en **un solo archivo**. Útil sobre todo para Secure Boot, porque firmas un solo archivo en vez de varios.

¿Debería usarlo en mi primera instalación?

—

No es necesario. Añade complejidad extra y está pensado para casos con Secure Boot activo. Se puede agregar después, una vez que el sistema arranque de forma confiable con

la configuración básica.

✓ Configuración recomendada (GRUB + OpenRC + dracut)

```
# /etc/portage/package.use/installkernel  
sys-kernel/installkernel grub dracut
```

```
emerge --ask sys-kernel/installkernel
```

Este comando no instala el kernel — instala la herramienta que sabrá qué hacer cuando lo emerjas en el capítulo 32.

Preguntas frecuentes

?

¿Y si no configuro nada, funciona igual?

—

`installkernel` se instala de todas formas como dependencia del kernel, pero sin las USE flags correctas puede quedar sin generar initramfs ni avisar al bootloader. El sistema podría quedar instalado sin poder arrancar. No es obligatorio en *todos* los casos, pero en tu configuración (kernel de distribución + GRUB) prácticamente sí lo es.

?

¿Por qué el archivo

```
package.use/installkernel
```

aparece vacío al abrirlo?

—

Porque lo estás creando tú. A diferencia de `make.conf` (que ya viene con contenido del stage3), los archivos dentro de `package.use/` normalmente no existen hasta que tú decides que un paquete necesita una excepción.

?

¿El nombre del archivo dentro de

```
package.use/
```

importa?

—

No. Portage lee **todos** los archivos de esa carpeta y los combina como si fueran uno. El nombre `installkernel` es solo convención para que tú sepas qué contiene cada archivo.

?

¿Puedo elegir GRUB y systemd-boot a la vez "por si acaso"?

—

No tiene sentido — son mecanismos de arranque distintos y competirían entre sí. Elige uno; siempre puedes cambiar después reconfigurando `installkernel`.

?

¿Este capítulo ya instala GRUB?

—

No. Solo prepara `installkernel` para que sepa que, cuando GRUB esté instalado (capítulo 44), debe generar su configuración automáticamente. La instalación real de GRUB en el disco es un paso posterior y de mayor riesgo.

Capítulo 32 — Selección del Kernel

Distribution Kernel · gentoo-sources · USE flags · modprobed-db

El kernel es la capa entre el hardware y todos los programas del sistema. Este capítulo decide **cómo** se va a construir y mantener a lo largo del tiempo — es una de las decisiones con más impacto a futuro en tu instalación.

32.0 El dilema central

Hay dos caminos. Ambos son válidos y reversibles (se puede cambiar de uno a otro después), pero implican filosofías distintas de mantenimiento:

Distribution Kernel

- Portage configura, compila e instala todo
- Se actualiza junto con `@world`
- Recomendado para empezar
- Mínima intervención manual

Configuración manual

- Vos elegís cada opción del kernel
- Usa `sys-kernel/gentoo-sources`
- Control total, más trabajo
- Hay que repetir el proceso en cada actualización

Dato importante: un kernel compilado a mano *no* usa menos RAM que uno de distribución. El kernel de Linux es modular — en ambos casos solo se carga lo que el hardware realmente necesita. La diferencia real está en cuánto control querés tener, no en el rendimiento.

32.1 Distribution Kernel

Es el camino recomendado para la mayoría de las instalaciones. Portage trata al kernel como un paquete más: lo configura con una selección genérica de opciones que cubre casi todo el hardware existente, lo compila (o lo descarga ya compilado) y lo instala.

Opción A — Compilar desde fuente

COMPILA LOCALMENTE · TARDA MÁS

COMANDO

```
root # emerge --ask sys-kernel/gentoo-kernel
```

Descarga el **código fuente** del kernel (con los parches propios de Gentoo aplicados) y lo compila en tu propia máquina, usando las **USE flags** y opciones de **make.conf** que ya definiste. El resultado es un kernel "a tu medida" dentro del margen que permite la configuración genérica de distribución.

Para qué sirve: si tu CPU o hardware es poco común, o si querés que tus USE flags globales (como flags de arquitectura, firmware específico, etc.) influyan en el resultado, esta es la opción. El costo es tiempo: compilar un kernel completo puede tardar bastante según el hardware.

Opción B — Binario precompilado

SOLO DESCARGA · INSTALACIÓN RÁPIDA

COMANDO

```
root # emerge --ask sys-kernel/gentoo-kernel-bin
```

En vez de compilar, Portage **descarga un kernel ya compilado** por el proyecto Gentoo, con una configuración genérica pensada para funcionar en la mayoría del hardware "de fábrica". No consume tiempo de CPU en compilación — es prácticamente instantáneo comparado con la opción A.

Para qué sirve: ideal si priorizás rapidez de instalación, estás en una máquina virtual, un laptop con hardware común, o simplemente no querés esperar una compilación. La contrapartida es menos margen de personalización, porque no pasa por tu **make.conf** de la misma forma.

Ambas opciones son intercambiables en cuanto a mantenimiento: las dos se actualizan igual, vía `emerge -avuDN @world`, y ambas requieren que `installkernel` esté configurado con un generador de initramfs (por ejemplo, Dracut) antes de instalarlas.

32.1.a El USE flag `dist-kernel`

USE FLAG

📌 Se configura en `/etc/portage/make.conf`

/ETC/PORTAGE/MAKE.CONF

```
USE="dist-kernel"
```

Es un interruptor global que le dice a Portage: *"cuando actualices el kernel, hacé también el trabajo de limpieza asociado, sin que yo tenga que acordarme."*

Sin este flag, después de cada actualización del kernel tendrías que correr manualmente:

```
root # emerge --ask @module-rebuild
```

para reconstruir módulos externos (como drivers de NVIDIA o ZFS, que dependen de la versión exacta del kernel) y regenerar el initramfs. Con `dist-kernel` activo, todo eso pasa automáticamente cuando el kernel sube de versión.

Analogía NixOS: es similar a lo que hace `nixos-rebuild switch` al regenerar el `initrd` y actualizar el bootloader en cada nueva generación — automático, sin pasos manuales sueltos. `dist-kernel` le da ese mismo comportamiento a Gentoo, que por defecto es más manual.

32.1.b Mantenimiento y limpieza

ACTUALIZAR TODO EL SISTEMA (INCLUIDO EL KERNEL SI HAY VERSIÓN NUEVA)

```
root # emerge --sync
root # emerge -avuDN @world
```

Para qué sirve: `--sync` baja la lista más reciente de paquetes disponibles (no instala nada todavía). `-avuDN @world` sí actualiza lo instalado. El flag `-a` (`ask`) muestra qué va a cambiar antes de confirmar.

LIMPIAR KERNELS VIEJOS

```
root # emerge --depclean
# o, específicamente:
root # emerge --prune sys-kernel/gentoo-kernel sys-kernel/gentoo-kernel-bin
```

Para qué sirve: libera espacio en disco eliminando versiones antiguas que Portage conserva hasta que se le pide limpiar.

32.2 Configuración manual (alternativa)

Usa `sys-kernel/gentoo-sources` en vez de un kernel de distribución. Acá vos configurás cada opción a mano con herramientas como `make menuconfig`. Hay tres formas de decidir *qué* incluir:

Opción 1 — Modprobed-db (automatiza la detección con el tiempo)

`sys-kernel/modprobed-db` es un demonio que corre en segundo plano (vía cron o un timer de systemd) y va **registrando qué módulos del kernel usa tu sistema a lo largo del tiempo** — no solo en el momento de instalar, sino con cada dispositivo nuevo que conectás.

Ejemplo: si semanas después de instalar Gentoo conectás un control de Xbox, modprobed-db anota ese módulo, y la próxima vez que reconstruyas el kernel, ese soporte ya está incluido automáticamente — sin que vos tengas que acordarte de agregarlo.

```
root # emerge --ask sys-kernel/modprobed-db
```

Para qué sirve: pensado para quien compila el kernel manualmente pero no quiere estar adivinando qué módulos incluir cada vez. Es la opción más "hands-off" dentro del camino manual — se usa normalmente junto con gentoo-kernel-bin como base.

Opción 2 — Proceso asistido (detección puntual, una sola vez)

A diferencia de modprobed-db (que observa con el tiempo), este método hace una **detección instantánea** del hardware presente en el momento de la instalación, usando dos herramientas:

- `lspci` — lista los dispositivos conectados por PCI (tarjeta de red, gráfica, etc.)
- `lsmod` — muestra qué módulos ya está usando el medio de instalación (el "live CD"), lo cual da pistas de qué necesitás activar

```
root # emerge --ask sys-apps/pciutils
root # cd /usr/src/linux
root # make localmodconfig
```

Para qué sirve: `make localmodconfig` autodetecta los módulos que el sistema está usando *ahora mismo* y genera una configuración base con eso. Es un punto de partida razonable si no querés empezar de cero, pero no seguirá aprendiendo después como modprobed-db.

Después de esto, se puede revisar manualmente con una interfaz de texto:

```
root # make nconfig
```

Para qué sirve: abre un menú interactivo para ajustar opciones específicas del kernel a mano, por si la autodetección se saltó algo.

Opción 3 — Configuración manual completa

Sin ayuda de detección automática: se configura cada opción del kernel a mano, una por una. Requiere conocer bien el hardware propio (controladores de disco, tarjeta de red, sistema de archivos, etc.). Es la opción de máximo control, pero también la de mayor curva de aprendizaje y riesgo de olvidar algo esencial (como el driver del disco donde arranca el sistema).

32.2.4 Compilar e instalar (camino manual)

TOMA TIEMPO · NO INTERRUMPIR

COMANDOS

```
root # make -j$(nproc) && make modules_install
root # make install
```

Comando

Para qué sirve

`make -j$(nproc)`

Compila el kernel usando todos los núcleos de CPU disponibles en paralelo (`nproc` los cuenta automáticamente), acelerando la compilación

`make modules_install`

Instala los módulos ya compilados en `/lib/modules/`

`make install`

Copia la imagen del kernel a `/boot/`. Si `installkernel` está presente, delega en él para nombrar el archivo con su versión y, opcionalmente, generar initramfs y actualizar el bootloader

32.x Resumen rápido

Necesito	Uso
Instalar rápido, sin complicaciones	<code>emerge --ask sys-kernel/gentoo-kernel-bin</code>
Instalar con mis propias USE flags	<code>emerge --ask sys-kernel/gentoo-kernel</code>
Que el sistema se auto-mantenga tras cada update	<code>USE="dist-kernel"</code> en <code>make.conf</code>
Control total del kernel	<code>sys-kernel/gentoo-sources</code> + <code>make menuconfig</code>
Que el kernel manual aprenda solo con el tiempo	<code>sys-kernel/modprobed-db</code>
Detección rápida y puntual de mi hardware actual	<code>make localmodconfig</code>

Capítulo 33 · Información del sistema de archivos

Gentoo Handbook AMD64 — GPT + UEFI · OpenRC · Perfil multilib

Contenido

- ¿De qué trata este capítulo?
- 33.1 Etiquetas y UUIDs del sistema de archivos
- 33.2 Etiquetas y UUIDs de partición
- El comando `blkid` a fondo
- 33.3 ¿Qué es fstab?
- 33.4 Creando el archivo fstab
- Extra: comprobando y depurando fstab

¿De qué trata este capítulo?

Este capítulo resuelve una sola pregunta: **¿cómo le digo a Gentoo qué particiones montar, dónde y con qué reglas, cada vez que arranca?** La respuesta vive en un único archivo: `/etc/fstab`. Antes de editarlo hay que entender cómo identificar las particiones de forma confiable (UUIDs), y para eso se usa el comando `blkid`.

📖 Se configura en: `/etc/fstab` — dentro del chroot

33.1 Etiquetas y UUIDs del sistema de archivos

Cuando formateas una partición con `mkfs.xfs`, `mkfs.vfat`, etc., el sistema de archivos resultante recibe automáticamente un **UUID** (identificador único) y, opcionalmente, tú le puedes poner un **LABEL** (nombre legible).

¿Para qué sirve? En vez de escribir `/dev/sda3` en `fstab` — que puede cambiar de nombre si agregas/quitas discos — usas su UUID, que no cambia sin importar el orden en que el kernel detecte los discos.

Ojo: si vuelves a formatear la partición, el UUID *cambia*, porque es una propiedad del sistema de archivos, no del disco físico ni de la partición GPT.

33.2 Etiquetas y UUIDs de partición

Distinto al punto anterior: aquí GPT le asigna directamente a la **partición** (no al sistema de archivos que vive dentro) un `PARTUUID` y opcionalmente un `PARTLABEL`. Como usas GPT, tienes esto disponible de forma nativa.

Identificador	¿A qué pertenece?	¿Cambia si reformateo?
UUID / LABEL	Sistema de archivos (xfs, vfat...)	Sí

PARTUUID / PARTLABEL	La partición GPT en sí	No
----------------------	------------------------	----

Analogía: el `PARTUUID` es el número de serie grabado en la caja fuerte; el `UUID` es el código de la cerradura que pusiste adentro. Si cambias la cerradura (reformateas), la caja sigue siendo la misma caja.

El comando `blkid` a fondo

`blkid` es la herramienta que consulta y muestra ambos tipos de identificadores (de sistema de archivos y de partición) para todos los dispositivos de bloque del sistema. Viene incluido en `sys-apps/util-linux`, ya presente en el stage3.

Uso básico y opciones más útiles

```
# Mostrar todo (recomendado, es lo que verás casi siempre)
blkid

# Consultar un solo dispositivo
blkid /dev/sda3

# Mostrar solo un campo específico (útil en scripts)
blkid -s UUID -o value /dev/sda3
blkid -s PARTUUID -o value /dev/sda1

# Forzar un nuevo escaneo (ignora la caché de blkid)
blkid -g
```

Opción	Qué hace
<code>-s CAMPO</code>	Filtra y muestra solo ese campo (UUID, PARTUUID, LABEL, TYPE...)
<code>-o value</code>	Imprime solo el valor, sin el nombre del campo — ideal para copiar/pegar o scripts
<code>-g</code>	Regenera la caché interna de blkid en <code>/run/blkid</code>
<code>-p</code>	Modo "low-level probe", ignora la caché por completo (más lento pero más confiable)

Salida típica dentro del chroot en tu configuración (GPT + UEFI):

```
/dev/sda1: UUID="A1B2-C3D4" TYPE="vfat" PARTUUID="c12a7328-f81f-11d2-ba4b-00a0c93ec93b"
/dev/sda2: UUID="1234abcd-..." TYPE="swap" PARTUUID="0657fd6d-a4ab-43c4-84e5-0933c84b4f4f"
/dev/sda3: UUID="5678efgh-..." TYPE="xfs" PARTUUID="4f68bce3-e8cd-4db1-96e7-fbcaf984b709"
```

33.3 ¿Qué es `fstab`?

`/etc/fstab` ("filesystem table") es una tabla de texto plano con **6 columnas** separadas por espacios o tabs. Cada línea = una partición a montar.

#	Campo	Explicación sencilla

1	Dispositivo	Qué montar: ruta (<code>/dev/sda1</code>), <code>UUID=</code> , <code>LABEL=</code> o <code>PARTUUID=</code>
2	Punto de montaje	Carpeta donde aparecerá (<code>/</code> , <code>/efi</code> , etc.)
3	Tipo de FS	<code>xfs</code> , <code>vfat</code> , <code>swap</code> ...
4	Opciones	Reglas de montaje, separadas por comas (ver abajo)
5	dump	Casi siempre <code>0</code> (respaldo con la utilidad <code>dump</code> , en desuso)
6	fsck	<code>1</code> para la raíz, <code>2</code> para el resto, <code>0</code> si no se revisa

Ojo: el `/etc/fstab` que trae el stage3 por defecto **no es válido**, es solo una plantilla de ejemplo. Hay que editarlo sí o sí.

```
nano /etc/fstab
```

33.4 Creando el archivo fstab

🔖 Se configura en: **/etc/fstab** · Aplica: al arrancar, cada vez que el kernel monta el sistema de archivos raíz

Como tu esquema es **GPT + UEFI**, la sección que te aplica es **33.4.2 / 33.4.3**, no la de DOS/BIOS heredado.

Ejemplo con rutas de dispositivo (33.4.2)

```
/dev/sda1 /efi vfat umask=0077,tz=UTC 0 2
/dev/sda2 none swap sw 0 0
/dev/sda3 / xfs defaults,noatime 0 1 ▼
```

Ejemplo recomendado con PARTUUID (33.4.3)

```
PARTUUID=c12a7328-f81f-11d2-ba4b-00a0c93ec93b /efi vfat umask=0077,tz=UTC 0 2
PARTUUID=0657fd6d-a4ab-43c4-84e5-0933c84b4f4f none swap sw 0 0
PARTUUID=4f68bce3-e8cd-4db1-96e7-fbcaf984b709 / xfs defaults,noatime 0 1
```

¿Cuál usar? PARTUUID es más robusto: si un día reformateas `/` de xfs a ext4, el `UUID` del sistema de archivos cambia, pero el `PARTUUID` de la partición no. Menos cosas que arreglar después.

Tu caso con OpenRC: el manual menciona que si usas **systemd** con UEFI y PARTUUIDs en formato DPS, te puedes saltar la creación de fstab, porque systemd monta todo solo. Como tú usas **OpenRC**, esto no aplica — el fstab es obligatorio en tu instalación.

Opciones de montaje más comunes explicadas

INFORMATIVO

Opción	Qué hace

<code>defaults</code>	Paquete estándar: <code>rw, suid, dev, exec, auto, nouser, async</code>
<code>noatime</code>	No registra la hora de "último acceso" al leer archivos → menos escrituras, más rendimiento (ideal en SSD)
<code>umask=0077</code>	Permisos restrictivos; necesario en <code>/efi</code> porque FAT32 no entiende permisos Unix
<code>tz=UTC</code>	Le dice al driver de FAT en qué zona horaria interpretar las fechas de archivos
<code>sw</code>	Opción específica de swap (equivalente a "sin opciones especiales")
<code>noauto</code>	No monta automáticamente al arrancar; hay que montarlo a mano
<code>user</code>	Permite a usuarios sin privilegios montar ese dispositivo (típico en <code>/mnt/cdrom</code>)
<code>auto</code>	Deja que el sistema adivine el tipo de FS; útil en medios removibles

No recomendado: la opción `discard` (TRIM en cada borrado) degrada el rendimiento. Es mejor programar `fstrim` periódico con cron o un timer, en vez de ponerlo en `fstab`.

Puente con NixOS: esto equivale a las entradas `fileSystems."/"` en tu `configuration.nix` — la diferencia es que ahí lo declaras en Nix y el sistema lo genera; aquí lo escribes tú mismo, a mano, en formato de tabla plana.

Extra: comprobando y depurando fstab

Esto no viene explícito en el capítulo del handbook, pero es útil añadirlo para tu flujo de trabajo:

Verificar el fstab antes de reiniciar

BAJO RIESGO

¿Para qué sirve? Detectar errores de sintaxis en `/etc/fstab` sin tener que reiniciar y arriesgarte a que el sistema no arranque.

```
# Simula el montaje de todo lo que no esté ya montado, sin montar de verdad
mount -fav

# Monta (de verdad) todo lo definido en fstab que aún no esté montado
mount -a
```

`mount -fav` es la forma segura de probar: `-f` = fake (no monta realmente), `-a` = todas las entradas, `-v` = verbose (te dice qué haría con cada línea).

Errores comunes al escribir fstab

RIESGO MEDIO SI SE IGNORAN

- Usar tabs/espacios inconsistentes no rompe nada (cualquier espacio en blanco sirve como separador), pero mezclar mal el número de columnas sí.

- Olvidar la columna 6 (`fsck`) o ponerle `1` a más de una partición aparte de `/` — solo la raíz debería llevar `1` .
- Copiar un UUID mal (un carácter cambiado) hace que el sistema no encuentre la partición al arrancar → caerá en modo de rescate/emergencia.
- Olvidar `umask=0077` en `/efi` : no rompe el arranque, pero deja el ESP con permisos poco seguros.

Información de red

Cómo dejar configurada la red del sistema ya instalado: nombre del equipo, resolución de nombres local y el método de conexión (DHCP, NetworkManager o netifrc). Enfoque: **OpenRC**.

¿Para qué sirve este capítulo?

💡 Hasta ahora usaste la red del **medio de instalación (live USB)**. Este capítulo configura la red que usará tu Gentoo **ya instalado**, la primera vez que arranque sin el USB. Si te lo saltas, el sistema instalado no tendrá internet al reiniciar.

1. Nombre del equipo (hostname)

RIESGO BAJO · REVERSIBLE

El **hostname** es el nombre que identifica tu máquina en la red local. No es una decisión permanente: se puede cambiar después sin problema.

Dónde se configura: `/etc/hostname`

COMANDO (DENTRO DEL CHROOT)

```
echo tux > /etc/hostname
```

🔄 Reemplaza `tux` por el nombre que quieras darle a tu equipo. Solo letras, números y guiones, sin espacios.

2. Archivo `/etc/hosts`

RIESGO BAJO · REVERSIBLE

`/etc/hosts` es una "libreta de contactos" local: traduce nombres a direcciones IP **sin necesidad de un servidor DNS**. Es lo primero que el sistema consulta antes de preguntarle a internet.

⚠ En sistemas **OpenRC** es obligatorio que el hostname elegido en el paso anterior también aparezca aquí. Si no coincide, pueden aparecer errores o demoras al arrancar servicios.

Dónde se configura: `/etc/hosts`

ARCHIVO `/ETC/HOSTS`

```
# Debe coincidir con /etc/hostname
127.0.0.1    tux.homenetwork tux localhost
```

```
::1          tux.homenetwork tux localhost

# Opcional: otros equipos conocidos de tu red
192.168.0.5 jenny.homenetwork jenny
192.168.0.6 benny.homenetwork benny
```

La línea con `127.0.0.1` es IPv4 "loopback" (la máquina hablándose a sí misma); la de `::1` es lo mismo en IPv6. Ambas deben existir.

3. Elegir el método de conexión

El handbook ofrece varias formas de conectar el sistema instalado. No hay que usar más de una a la vez.

⚠ **No mezcles gestores de red.** Si activas dos al mismo tiempo (por ejemplo `netifrc` y `NetworkManager`), van a pelear por controlar la misma interfaz y la red puede fallar de forma intermitente.

Método	Ideal para	Init compatible
dhcpcd	Ethernet con router doméstico, IP automática	Cualquiera
NetworkManager	WiFi, laptops, escritorio	Cualquiera
netifrc	Configuración clásica y control fino	Solo OpenRC ✓
systemd-networkd	Equivalente de <code>netifrc</code>	Solo systemd

Opción A — DHCP automático con `dhcpcd`

rápido

—

RIESGO BAJO

🔗 La forma más simple: tu router le asigna la IP a tu equipo automáticamente. Úsalo si no necesitas configuración especial.

Dónde se configura: nada que editar — el servicio hace todo solo.

```
emerge --ask net-misc/dhcpcd
rc-update add dhcpcd default
```

`rc-update add dhcpcd default` agrega el servicio al runlevel `default`, el conjunto de servicios que arrancan en cada boot normal.

Opción B — NetworkManager

WiFi / escritorio

—

RIESGO BAJO

🔗 Gestiona Ethernet, WiFi y PPPoE por CLI, TUI o GUI. Preferido si vas a usar un entorno de escritorio.

Dónde se configura: `/etc/portage/make.conf` y `/etc/portage/package.use/`
`/ETC/PORTAGE/MAKE.CONF`

```
USE="${USE} networkmanager"
```

`/ETC/PORTAGE/PACKAGE.USE/NETWORKMANAGER`

```
net-wireless/wpa_supplicant dbus
```

```
emerge --ask net-misc/networkmanager  
rc-update add NetworkManager default
```

Opción C — netifrc

recomendado para tu setup OpenRC

—

RIESGO MEDIO · SI TE EQUIVOCAS DE INTERFAZ, PIERDES RED AL REINICIAR

🔗 La forma "nativa" de OpenRC. Toda la configuración vive en **un solo archivo**: `/etc/conf.d/net`. Te da control total sobre IP estática, rutas, DNS, etc.

Dónde se configura: `/etc/conf.d/net`

Instalar el paquete

```
emerge --ask --noreplace net-misc/netifrc
```

`--noreplace` evita reinstalar si ya está presente, por si viene de fábrica en el stage3.

Configurar la interfaz

```
nano /etc/conf.d/net
```

OPCIÓN 1 — DHCP

```
config_eth0="dhcp"
```

OPCIÓN 2 — IP ESTÁTICA

```
config_eth0="192.168.0.2 netmask 255.255.255.0 brd 192.168.0.255"
```



```
routes_eth0="default via 192.168.0.1"
```

📌 El nombre de tu interfaz real casi nunca es `eth0` hoy en día (suele ser algo como `enp3s0`). Verifícalo antes con `ip a` desde el entorno live, y reemplaza `eth0` por ese nombre en todo el proceso.

Activar la red al arrancar

```
cd /etc/init.d
ln -s net.lo net.eth0
rc-update add net.eth0 default
```

¿Por qué un symlink? `net.lo` es un script genérico de OpenRC que sabe leer `/etc/conf.d/net` y aplicar la configuración según el nombre con el que se le invoque. Al enlazarlo como `net.eth0`, ese mismo script configurará específicamente esa interfaz. Si tienes varias interfaces, repites el enlace para cada una (`net.wlan0`, etc.).

Si te equivocaste de nombre de interfaz tras reiniciar

1. Corrige el nombre en `/etc/conf.d/net`
2. Crea el nuevo enlace simbólico correcto
3. Borra el enlace viejo: `rm /etc/init.d/net.eth0`
4. Agrega el nuevo al runlevel: `rc-update add net.enpXsY default`
5. Quita el viejo: `rc-update del net.eth0 default`

Opción D — systemd-networkd

solo si usas systemd

NO APLICA A TU INSTALACIÓN (USAS OPENRC) · REFERENCIA

🔗 Es el equivalente de netifrc pero para systemd. Se configura con archivos `.network` en estilo INI.

Dónde se configura: `/etc/systemd/network/*.network`
`/ETC/SYSTEMD/NETWORK/50-DHCP.NETWORK`

```
[Match]
Name=en*
```

```
[Network]
DHCP=yes
```

```
systemctl enable systemd-networkd.service
```

4. Conceptos de red que conviene entender

CIDR — ¿qué significa el "/24"?

—

CIDR indica cuántos bits de la dirección IP pertenecen a la **red** y cuántos al **host**. Es la notación tipo `192.168.0.2/24`.

- **/24** → 254 direcciones utilizables (el tamaño más común en redes domésticas)
- **/32** → 1 sola dirección (un host individual)

Fórmula: $2^{(32 - \text{CIDR})} - 2$ direcciones utilizables (se restan la dirección de red y la de broadcast).

Gateway / ruta por defecto

—

El **gateway** (puerta de enlace) es el dispositivo — normalmente tu router — que reenvía tu tráfico hacia fuera de la red local, hacia internet. Es lo que le dice a tu equipo "todo lo que no sea de esta red, mándamelo a mí".

```
# Ver rutas actuales
ip route

# Agregar ruta por defecto manualmente
ip route add default via 192.168.0.1
```

DNS — traducir nombres a IPs

—

El DNS traduce nombres de dominio (`gentoo.org`) a direcciones IP. Se configura en `/etc/resolv.conf`.

/ETC/RESOLV.CONF

```
nameserver 1.1.1.1
nameserver 1.0.0.1
```

⚠ Si `dhcpcd` está corriendo, sobrescribirá este archivo automáticamente y tus cambios manuales no persistirán. Verifica con `ps x | grep dhcpcd`.

1. Interfaz activa

`ip link · ip addr`

¿existe y está UP?



2. IP + CIDR asignada

`ip a · dhcpcd / netifrc`

red local



3. Ruta por defecto (gateway)

`ip route · ping -c3 1.1.1.1`

salida a internet



4. DNS resolviendo

`/etc/resolv.conf · curl gentoo.org`

nombres → IP

Diagnóstico por capas: si un nivel falla, el problema está ahí (no en los de arriba).

5. Verificar que todo funciona

Ver interfaces y direcciones asignadas

`ip a`

Ver tabla de rutas

`ip r`

Probar conectividad y resolución DNS

`ping -c 3 gentoo.org`

Si el `ping` responde, tienes conectividad e internet funcionando correctamente.

Resumen de tu ruta (OpenRC)

1. `echo tux > /etc/hostname`
2. Reflejar el mismo nombre en `/etc/hosts`
3. `emerge --ask --noreplace net-misc/netifrc`
4. Configurar `/etc/conf.d/net` (DHCP o IP estática)
5. Crear el symlink y activar en el runlevel: `rc-update add net.eth0 default`
6. Verificar con `ip a` y `ping`

Capítulo 35 · System Information

Contraseña de root y configuración inicial de arranque (init), antes del primer reinicio.

¿PARA QUÉ SIRVE ESTE CAPÍTULO?

Prepara lo mínimo indispensable para que el sistema sea usable en cuanto reinicies: una forma de entrar (contraseña de root) y cómo se comporta el sistema al arrancar/apagar (teclado, reloj, gestor de servicios).

35.1 Contraseña de root

Se define con un solo comando, sin editar archivos:

```
passwd
```

CÓMO SE USA

Al ejecutarlo te pide escribir la contraseña dos veces (confirmación). No aparece nada en pantalla mientras escribes — es normal, no está congelado.

IMPORTANTE

Sin esto no podrás iniciar sesión al reiniciar: todavía no existe ningún usuario normal, solo `root`. El usuario para el día a día se crea después, en el capítulo 50.

35.2 Init and boot configuration

"Init" es el primer proceso que arranca el sistema y administra los demás servicios. Gentoo te deja elegir cuál usar; tú usas **OpenRC**.

35.2.1 OpenRC TU CONFIGURACIÓN

Son tres archivos de texto que se editan con `nano`. No hay comando de "instalación", solo edición directa.

CONFIGURACIÓN GENERAL

`/etc/rc.conf`

Controla el comportamiento general de OpenRC: verbosidad al arrancar, colores, arranque en paralelo, editor por defecto, etc.

TECLADO

`/etc/conf.d/keymaps`

Define el layout de teclado de la *consola* en modo texto (no aplica a un entorno gráfico).

RELOJ

`/etc/conf.d/hwclock`

Le dice al sistema si el reloj físico de la placa madre está en UTC o en hora local.

SOLO STAGE3 "DESKTOP"

`dbus-uuidgen`

Genera el ID de máquina que usa `dbus`, el "bus" de comunicación entre programas de escritorio.

1. `/etc/rc.conf` — el panel de control

```
nano /etc/rc.conf
```

Viene lleno de comentarios (`#`) explicando cada opción, con valores por defecto razonables. Solo descomenta lo que quieras cambiar.

▼ Ver opciones comunes que suele tocar la gente

OPCIÓN	QUÉ HACE
<code>rc_parallel="YES"</code>	Arranca servicios en paralelo → boot más rápido (puede complicar el diagnóstico si algo falla).
<code>rc_interactive="YES"</code>	Te deja elegir si saltar un servicio que está tardando o fallando durante el arranque.
<code>unicode="YES"</code>	Habilita soporte Unicode en la consola (recomendado para <code>es_MX.UTF-8</code>).

2. `/etc/conf.d/keymaps` — layout de teclado

```
nano /etc/conf.d/keymaps
```

Busca la línea `keymap="us"` y cámbiala. Para español latinoamericano:

```
keymap="la-latin1"
```

CÓMO VER LAS OPCIONES DISPONIBLES

Antes de editar, puedes listar los layouts instalados:

```
ls /usr/share/keymaps/i386/qwerty/
```

El nombre del archivo sin la extensión `.map.gz` es el valor que va en `keymap=`.

RIESGO

Si el keymap es incorrecto, al escribir en consola después del reinicio saldrán símbolos raros (tildes, ñ, signos fuera de lugar). No es grave, solo hay que corregir el archivo y volver a cargar el mapa con `loadkeys`.

3. /etc/conf.d/hwclock — reloj de hardware

```
nano /etc/conf.d/hwclock
```

Busca la línea `clock=`:

VALOR	CUÁNDO USARLO
<code>clock="UTC"</code>	Recomendado si el equipo solo tiene Linux.
<code>clock="local"</code>	Si tienes dual-boot con Windows (que por defecto usa hora local en el reloj de hardware).

CONSECUENCIA DE EQUIVOCARSE

El sistema puede mostrar la hora desfasada varias horas ("clock skew"), lo que puede causar problemas con certificados, sincronización o registros de log con hora incorrecta.

Extra — solo si tu stage3 es tipo "desktop"

```
dbus-uuidgen --ensure=/etc/machine-id
```

Genera un identificador único para la máquina, usado por `dbus`. Si tu stage3 no es desktop, se omite: se genera automáticamente cuando se instale el paquete `dbus` más adelante.

35.2.2 systemd REFERENCIA — NO ES TU CONFIGURACIÓN

Si usaras systemd en vez de OpenRC, en lugar de editar archivos sueltos usarías comandos interactivos:

```
systemd-machine-id-setup  
systemd-firstboot --prompt
```

- `systemd-machine-id-setup` — genera el ID único de la máquina (equivalente al `dbus-uuidgen` de OpenRC).
- `systemd-firstboot --prompt` — asistente interactivo que pregunta locale, timezone, hostname, contraseña de root y shell, todo en un solo paso.

Después se activan los servicios recomendados por defecto:

```
systemctl preset-all --preset-mode=enable-only
```

DIFERENCIA CON LA VARIANTE COMPLETA

`--preset-mode=enable-only` solo activa servicios, sin tocar los que ya desactivaste. La variante sin esa bandera (`systemctl preset-all`) puede sobrescribir configuraciones que ya hayas ajustado durante la instalación, así que se usa con más cuidado.

NOTA DEL MANUAL

Es normal que aparezca una advertencia de fallo al correr este comando desde el LiveCD, porque el propio medio de instalación de Gentoo usa OpenRC, no systemd.

Comparación rápida

TAREA	OPENRC	SYSTEMD
ID de máquina	<code>dbus-uuidgen</code> (solo desktop)	<code>systemd-machine-id-setup</code>
Configuración inicial	Editar 3 archivos a mano	<code>systemd-firstboot --prompt</code> (asistente)
Activar servicios por defecto	Uno por uno, luego con <code>rc-update add</code>	<code>systemctl preset-all</code> (masivo)
Enfoque	Imperativo, archivos de texto plano	Declarativo por unidades + asistentes

ANALOGÍA CON NIXOS

Estos tres archivos de OpenRC equivalen a opciones dispersas que en NixOS declararías juntas en `configuration.nix` (`console.keyMap` , `time.hardwareClockInLocalTime` , etc.). Aquí no hay un solo archivo declarativo: es imperativo y repartido en varios archivos que editas directamente.

Capítulo 36 — System Logger

Elegir y activar el registrador de eventos del sistema (perfil OpenRC)

¿Qué es y por qué el stage3 no trae uno?

El **system logger** es el servicio que guarda en archivos (normalmente dentro de `/var/log/`) todo lo que va pasando en el sistema: errores, advertencias, eventos de arranque, intentos de acceso, etc. Sin él, esos mensajes se pierden apenas se generan.

Gentoo **no incluye ninguno por defecto** en el stage3 porque existen varios paquetes que hacen lo mismo con distinto enfoque, y prefiere que el usuario elija el que le conviene en vez de imponer uno.

Analogía NixOS: es como declarar `services.journald` o cualquier otro logger en `configuration.nix` — nada corre "gratis" sin que lo pidas explícitamente en el sistema.

Opciones disponibles

PAQUETE	PERFIL	ROTACIÓN DE LOGS	NOTAS
app-admin/sysklogd	Principiante RECOMENDADO	Sí (incluida, v2.0+)	Funciona bien "tal cual", sin configurar nada extra.
app-admin/syslog-ng	Avanzado	No — requiere logrotate aparte	Muy potente, pero necesita configuración manual para sacarle provecho.
app-admin/metalog	Avanzado	Configurable	Altamente personalizable, orientado a filtrado fino de mensajes.

PARA QUÉ SIRVE ELEGIR BIEN

Si sólo quieres logs funcionando sin complicarte, `sysklogd` es la elección estándar del handbook para principiantes. Si más adelante necesitas reenviar logs a un servidor central o filtrar mensajes por reglas complejas, ahí es cuando `syslog-ng` o `metalog` valen la pena.

Instalación y activación (sysklogd)

CORRE `ENchroot`, como root

1. Instalar el paquete **BAJO RIESGO**

```
root # emerge --ask app-admin/sysklogd
```


PARA QUÉ SIRVE

Descarga y compila (o instala binario, si configuraste binhost) el logger. El flag `--ask` te muestra qué se va a instalar y pide confirmación antes de proceder — es "emerger", es decir, reversible: si te arrepientes, luego lo desinstalas sin dejar el sistema en mal estado.

2. Activarlo en el arranque BAJO RIESGO

```
root # rc-update add syslogd default
```

PARA QUÉ SIRVE

Instalar el paquete **no** lo hace arrancar solo. `rc-update add` es el comando de OpenRC que registra el servicio en un *runlevel* (aquí, `default`, el que se activa en un arranque normal). Sin este paso, `syslogd` quedaría instalado pero apagado.

Comandos útiles después de instalarlo

COMANDO	QUÉ HACE
<code>rc-service syslogd start</code>	Lo arranca de inmediato (sin esperar el próximo reboot).
<code>rc-service syslogd status</code>	Muestra si está corriendo.
<code>tail -f /var/log/messages</code>	Ver los logs en vivo mientras se generan.
<code>rc-update show</code>	Lista todos los servicios y en qué runlevel están activos.

¿Y si prefiero `syslog-ng` o `metalog`?

El patrón es idéntico, solo cambia el nombre del paquete y del servicio:

```
root # emerge --ask app-admin/syslog-ng
root # rc-update add syslog-ng default
```

ADVERTENCIA

Si eliges `syslog-ng`, instala también `app-admin/logrotate` — de lo contrario el archivo de log crecerá indefinidamente sin rotarse ni comprimirse.

Dónde vive todo esto

BINARIO / **SERVICIO** `/etc/init.d/syslogd`

CONFIGURACIÓN/`etc/syslog.conf`

SALIDA DE LOGS/`var/log/messages`,/`var/log/syslog`

El archivo `/etc/syslog.conf` define *qué* se registra y *dónde*: por ejemplo, puedes mandar solo los mensajes críticos del kernel a un archivo separado. Para un sistema doméstico, la configuración por defecto suele ser suficiente y no hace falta tocarla.

Referencia: si usaras systemd

No aplica a tu instalación (usas OpenRC), pero para completar el panorama: los sistemas con systemd ya traen `systemd-journald` integrado, que cubre casi toda la funcionalidad de un logger tradicional. La mayoría de esas instalaciones se saltan por completo este paso. Para consultar esos logs se usa `journalctl` en vez de leer archivos de texto plano.

Capítulo 37 — Cron Daemon

Automatización de tareas programadas en tu sistema

En una frase

Un **cron daemon** es un programa que corre en segundo plano y ejecuta comandos automáticamente según un horario que tú definas (cada hora, cada día, cada semana...). En Gentoo, con OpenRC, la opción recomendada y más sencilla es `cronie`.

¿Qué problema resuelve?

Hay tareas de mantenimiento que un sistema necesita hacer con regularidad: limpiar archivos temporales, actualizar bases de datos, respaldar información, sincronizar algo, etc. En lugar de acordarte tú de ejecutarlas manualmente, un cron daemon lo hace por ti, con la periodicidad que definas.

¿Para qué sirve? Automatizar tareas repetitivas del sistema sin intervención humana. Es la pieza que le falta a tu servidor/PC para "acordarse" de hacer mantenimiento por sí mismo.

¿Dónde se configura? El horario de las tareas vive en archivos llamados *crontabs*. El más importante para el sistema es `/etc/crontab` (tareas a nivel sistema, ejecutadas por root u otros usuarios especificados en el propio archivo). Cada usuario también puede tener su propio crontab personal, editable con `crontab -e`, y que normalmente se guarda internamente en `/var/spool/cron/crontabs/<usuario>` (la ubicación exacta depende del demonio elegido).

Las 4 opciones disponibles en Gentoo

Todas cumplen la misma función básica (ejecutar comandos programados), pero difieren en filosofía: seguridad, simplicidad o funciones extra.

Demonio	Paquete	Enfoque	¿Cuándo elegirlo?
cronie (default)	<code>sys-process/cronie</code>	El cron "clásico" (basado en Vixie cron) con mejoras de seguridad: soporta PAM y SELinux.	Uso general. Es la opción recomendada por el manual y la más compatible/documentada.
<code>dcron</code>	<code>sys-process/dcron</code>	Minimalista: simple y seguro, solo lo esencial.	Sistemas donde quieres el mínimo software posible corriendo.

fcrn	sys-process/fcron	Extiende cron con capacidades tipo <i>anacron</i> (ejecuta tareas perdidas si el equipo estuvo apagado).	Laptops o equipos que no están encendidos 24/7 y no quieres perder tareas programadas.
bcron	sys-process/bcron	Diseño moderno centrado en seguridad: divide responsabilidades en varios procesos pequeños con comunicación controlada entre ellos.	Entornos donde el aislamiento/seguridad de procesos es prioridad.

Instalación y uso — opción por defecto: cronie

INSTALAR EL PAQUETE

```
root # emerge --ask sys-process/cronie
```

RIESGO BAJO

Qué hace: descarga, compila e instala cronie. `--ask` te muestra qué se va a instalar antes de confirmar.

ACTIVAR EL SERVICIO EN EL ARRANQUE

```
root # rc-update add cronie default
```

RIESGO BAJO — REVERSIBLE

Qué hace: agrega el servicio `cronie` al runlevel `default` de OpenRC, para que arranque automáticamente cada vez que enciendas el sistema. Sin esto, cronie no se ejecutaría a menos que lo inicies manualmente con `rc-service cronie start` cada vez.

¿Y cómo le doy tareas para que ejecute?

Aquí hay dos caminos, según quién debe ejecutar la tarea:

Opción 1 — Crontab del sistema:

```
/etc/crontab
```

—

Se edita directamente como root y permite especificar **qué usuario** ejecuta cada línea. Formato:

```
minuto hora día-mes mes día-semana usuario comando
```

Ejemplo — ejecutar un script cada día a las 3:00 AM como root:

```
0 3 * * * root /usr/local/bin/mi_backup.sh
```

Los campos aceptan `*` (cualquier valor), rangos (`1-5`), listas (`1,15`) y saltos (`*/6` = cada 6 unidades).

Opción 2 — Crontab personal de un usuario:

```
crontab -e
```

—

Cada usuario (incluido root) puede tener su propio crontab, sin necesidad de indicar el usuario en cada línea (ya se sabe de quién es):

```
$ crontab -e      # abre tu crontab personal en el editor
$ crontab -l      # lista tus tareas programadas actuales
$ crontab -r      # elimina tu crontab (¡cuidado, sin confirmación!)
```

RIESGO MEDIO — `CRONTAB -R` **BORRA TODO SIN PREGUNTAR**

Alternativas: pasos de instalación específicos

dcron — requiere inicialización manual

—

```
root # emerge --ask sys-process/dcron
root # crontab /etc/crontab
```

El segundo comando es necesario porque dcron, a diferencia de cronie, no lee `/etc/crontab` automáticamente — hay que "cargarlo" explícitamente la primera vez.

fcron — requiere paso de configuración post-emerge

—

```
root # emerge --ask sys-process/fcron
root # emerge --config sys-process/fcron
```

`emerge --config` ejecuta un script de configuración interactivo que trae el propio paquete (no todos los paquetes lo requieren, pero fcron sí). Esto suele encargarse de crear las estructuras internas que fcron necesita para funcionar correctamente.

bcron — instalación simple, sin pasos extra

–

```
root # emerge --ask sys-process/bcron
```

El manual no indica pasos adicionales de activación para bcron más allá de la instalación estándar.

Caso especial: usuarios de modprobed-db

Si compilaste tu kernel manualmente (en vez de usar `gentoo-kernel` con configuración automática) y usaste la herramienta `modprobed-db` para detectar qué módulos usa tu hardware, aquí se programa que escanee el sistema periódicamente.

AGREGAR A /ETC/CRONTAB

```
# Ejecutar modprobed-db cada 6 horas
0 */6 * * * /usr/bin/modprobed-db store &> /dev/null
```

RIESGO BAJO

Nota: como en tu instalación usas `sys-kernel/gentoo-kernel`, esta sección probablemente no te aplica — es relevante solo para quienes compilan el kernel a mano y usan esa herramienta de detección de módulos.

Verificar que todo funciona

CONFIRMAR QUE EL SERVICIO ESTÁ CORRIENDO

```
root # rc-service crontie status
```

REVISAR LOS LOGS DE EJECUCIÓN DE CRON

```
root # tail -f /var/log/cron.log
```

La ruta exacta del log puede variar según el logger del sistema que instalaste (`sysklogd`, `syslog-ng`, etc. — capítulo 36).

Si usaras systemd (referencia)

`systemd` no necesita un cron daemon aparte: trae **timers** integrados, que cumplen la misma

función (ejecutar servicios en horarios programados) mediante unidades `.timer`. Para la mayoría de instalaciones `systemd`, instalar un cron adicional es innecesario — solo se justifica si necesitas compatibilidad específica con sintaxis `crontab` tradicional.

Resumen rápido

- **Qué es:** un programa que ejecuta comandos en horarios definidos.
- **Recomendado:** `crone` — simple, seguro, compatible.
- **Se configura en:** `/etc/crontab` (sistema) o `crontab -e` (por usuario).
- **Para activarlo en el arranque:** `rc-update add crone default`.
- **Alternativas:** `dcron` (ligero), `fcron` (recupera tareas perdidas), `bcron` (seguridad por diseño).

Capítulo 38 — Indexado de archivos

Instalación de Gentoo · Sección opcional · Herramienta: `mlocate`

¿Para qué sirve este capítulo? Instalar una herramienta que crea un "catálogo" de todos los archivos del sistema, para poder buscarlos por nombre de forma casi instantánea, en vez de recorrer el disco completo cada vez.

38.1 El concepto: índice vs. búsqueda en vivo

Cuando buscas un archivo en Linux tienes dos caminos:

Método	Cómo funciona	Velocidad
<code>find /</code>	Recorre el disco carpeta por carpeta en el momento de la búsqueda	Lento en discos grandes
<code>locate</code>	Consulta una base de datos ya construida de antemano	Casi instantáneo

La base de datos se actualiza periódicamente (normalmente vía `cron`, que ya configuraste en el capítulo 37). Es el mismo comando `locate` que probablemente conoces de Debian/Ubuntu.

38.2 Instalación

RIESGO BAJO — REVERSIBLE

Para qué sirve: descarga e instala el paquete `mlocate`, que incluye el comando de búsqueda (`locate`) y el generador de la base de datos (`updatedb`).

```
root # emerge --ask sys-apps/mlocate
```

38.3 Cómo se usa

Construir/actualizar el índice

Para qué sirve: escanea todo el sistema de archivos y guarda la lista en la base de datos. Sin este paso, `locate` no encuentra nada nuevo.

```
# Ejecutar manualmente cuando lo necesites
root # updatedb
```

En un sistema normal no necesitas correrlo a mano: `cron` lo ejecuta automáticamente todos los días (revisa `/etc/cron.daily/`).

Buscar archivos

```
$ locate nombre_archivo
```

Devuelve todas las rutas que contienen ese texto, aunque no sea el nombre exacto.

38.4 Opciones más útiles de locate

Opción	Qué hace
<code>-i</code>	Ignora mayúsculas/minúsculas
<code>-c</code>	Solo cuenta cuántos resultados hay, no los lista
<code>-l N</code>	Limita la salida a N resultados
<code>-e</code>	Verifica que el archivo aún exista en disco (más lento)
<code>-r</code>	Interpreta el patrón como expresión regular

```
# Ejemplo: buscar archivos .conf sin importar mayúsculas, máximo 10 resultados
$ locate -i -l 10 .conf
```

38.5 Dónde se configura

El archivo de configuración es `/etc/updatedb.conf`. Ahí se ajusta:

- `PRUNE_PATHS` — carpetas que **no** se deben indexar (ej. `/tmp` , `/mnt` , puntos de montaje temporales)
- `PRUNE_BIND_MOUNTS` — si se ignoran los montajes tipo bind (normalmente `"yes"`)
- `PRUNEFS` — tipos de sistema de archivos a excluir (ej. `nfs` , `tmpfs`)

```
root # nano -w /etc/updatedb.conf
```

▼ Nota de seguridad (información adicional, no está en el manual)

La base de datos de `mlocate` se genera respetando permisos: un usuario normal no puede ver rutas de archivos que no tiene permiso de leer. Aun así, si guardas datos sensibles fuera de tu carpeta personal, revisa `PRUNE_PATHS` para excluirlos del índice.

▼ ¿Y si no me interesa buscar archivos por terminal?

Este capítulo es **opcional**. Puedes saltarlo sin que afecte el resto de la instalación — no tiene relación con el arranque, el kernel ni el bootloader. El siguiente paso crítico sigue siendo GRUB (capítulo 44).

Acceso remoto por shell (SSH)

Paso opcional — habilita el servidor SSH para administrar tu equipo desde otra máquina

¿Para qué sirve este capítulo? Configura tu Gentoo para aceptar conexiones remotas por SSH desde el primer arranque, en vez de que solo puedas usarlo sentado frente al teclado/monitor.

🔧 El concepto en simple

El paquete que hace esto posible es `net-misc/openssh`, y ya viene instalado en el sistema base de Gentoo. El problema es que estar instalado no significa que arranque solo: en Gentoo, cada servicio debe **registrarse explícitamente** para iniciar con el sistema.

Este capítulo simplemente le dice a OpenRC: "cuando el sistema arranque, enciende también el servidor SSH (`sshd`)".

⚙️ Dónde se configura

Elemento	Ubicación
Registrar el servicio para que arranque	<code>rc-update</code> (comando, no archivo)
Configuración del servidor SSH	<code>/etc/ssh/sshd_config</code>
Consola serial (caso especial)	<code>/etc/inittab</code>
Llaves del servidor (host keys)	<code>/etc/ssh/</code>

🐞 Activar el servicio RIESGO BAJO

Para qué sirve: agrega `sshd` al runlevel `default`, que es el conjunto de servicios que arrancan en un uso normal del sistema (equivalente a "modo multiusuario").

```
rc-update add sshd default
```

Es exactamente el mismo patrón que usaste con `chronyd` o el logger del sistema: `rc-update add <servicio> <runlevel>`. En NixOS sería el equivalente a `services.openssh.enable = true;`, solo que aquí es un comando en vez de una declaración en un archivo.

Para verificar que quedó registrado:

```
rc-update show | grep sshd
```

Y si quieres probarlo sin reiniciar (útil incluso durante la instalación, dentro del entorno live):

```
rc-service sshd start
```

🔒 Comportamiento por defecto y seguridad

Importante: por defecto `sshd` no permite login como root. Necesitas un usuario normal ya creado (o crear uno) antes de poder conectarte remotamente.

Esto es intencional: reduce el riesgo de que alguien intente forzar el acceso directamente a la cuenta con más privilegios del sistema.

▼ ¿Cómo permitir root por SSH? (no recomendado)

Si por alguna razón necesitas permitirlo (por ejemplo, en un entorno de pruebas aislado), se edita:

```
nano /etc/ssh/sshd_config
```

Y se ajusta la directiva `PermitRootLogin` (valores comunes: `no`, `yes`, `prohibit-password`). Después hay que reiniciar el servicio para que el cambio tome efecto:

```
rc-service sshd restart
```

🔌 Caso especial: consola serial

Esto **solo aplica** si tu equipo es un servidor sin monitor que administras a través de un puerto serie (poco común en un equipo de escritorio). Si no es tu caso, puedes ignorar esta sección.

▼ Configurar agetty en consola serial

```
nano -w /etc/inittab
```

Se descomentan las líneas de la sección `SERIAL CONSOLES` :

```
s0:12345:respawn:/sbin/agetty 9600 ttyS0 vt100
s1:12345:respawn:/sbin/agetty 9600 ttyS1 vt100
```

📖 Referencia: equivalente en systemd

Solo para contraste, ya que tu instalación usa OpenRC — **no necesitas ejecutar esto**:

```
systemctl enable sshd
```

🌟 **Vale la pena saber (no está en el manual, pero es útil)**

La primera conexión mostrará una "huella digital" (fingerprint)

La primera vez que te conectes por SSH a tu Gentoo recién instalado, el cliente SSH te pedirá confirmar la huella del servidor. Es normal — solo verifica que estás conectando al equipo correcto.

Si reinstalas el sistema, tu cliente SSH se quejará

Cuando reinstales Gentoo (mismo disco, mismo hostname/IP), tu computadora cliente pensará que alguien está suplantando al servidor, porque las llaves cambiaron. Hay que borrar la entrada vieja con:

```
ssh-keygen -R <ip-o-hostname>
```

Opciones útiles en `sshd_config`

Directiva	Qué controla
<code>Port</code>	Puerto de escucha (por defecto 22)
<code>PermitRootLogin</code>	Si root puede entrar directamente
<code>PasswordAuthentication</code>	Permitir login con contraseña vs. solo llaves
<code>AllowUsers</code>	Lista blanca de usuarios que pueden conectarse

Cualquier cambio en este archivo requiere `rc-service sshd restart` para aplicarse.

✓ Resumen rápido

Qué hacer

- `rc-update add sshd default`
- Crear un usuario no-root para conectarte
- (Opcional) probar con `rc-service sshd start`

Cuándo saltarlo

- Si vas a trabajar solo desde el teclado/monitor local
- Se puede activar después del primer arranque sin problema

Cap. 40 — Shell Completion (Bash)

OPCIONAL

RIESGO BAJO

Autocompletado inteligente en bash: presionas `Tab` y el shell te sugiere comandos, opciones (`--flags`), rutas y hasta paquetes de Portage. Es pura comodidad — no toca el arranque, el kernel ni nada crítico del sistema.

¿Qué problema resuelve?

Por defecto, bash en Gentoo solo completa nombres de archivos y comandos básicos. No sabe, por ejemplo, qué opciones acepta `emerge` o `systemctl`. El paquete `bash-completion` le agrega a bash "diccionarios" de reglas por comando, para que el `Tab` sea mucho más útil.

▼¿Para qué sirve realmente, en la práctica?

Evita errores de tipeo en comandos largos (nombres de paquetes, flags de `emerge`, subcomandos de `git`, etc.) y ahorra tiempo al no tener que recordar la sintaxis exacta de cada herramienta. Es más notorio cuanto más usas la terminal a diario.

Instalación

```
root # emerge --ask app-shells/bash-completion
```

¿Qué instala este comando?

El paquete `app-shells/bash-completion`, que trae:

- Un framework central que bash carga al iniciar sesión.
- Scripts de completado individuales para cientos de comandos comunes (`tar`, `git`, `ssh`, `rsync`, etc.).
- Reglas específicas para herramientas propias de Gentoo, como `emerge` — por ejemplo, escribir `emerge --a` y presionar `Tab` sugiere `--ask`, `--autounmask`, etc.

¿Dónde se configura y cómo se activa?

No requiere edición manual de archivos para funcionar de forma básica. El paquete se engancha automáticamente al iniciar una sesión de bash interactiva, a través de `/etc/bash/bashrc` (o `/etc/profile.d/`, según la versión). Basta con cerrar y volver a abrir la terminal (o hacer `source /etc/bash/bashrc`) después de instalarlo.

Nota: si estás dentro del entorno `chroot` haciendo la instalación, el autocompletado nuevo aplicará quizando arranques el sistema instalado y abras una sesión de bash normal ahí — no dentro del live environment.

Gestión fina con eselect

Una vez instalado el paquete, puedes activar o desactivar completions por comando individual usando el módulo `bashcomp` de `eselect` — la misma herramienta que ya usaste para `eselect news` o para elegir perfil.

Comando	Qué hace
<code>eselect bashcomp list</code>	Muestra todos los completions disponibles y cuáles están activos/inactivos.
<code>eselect bashcomp enable <nombre></code>	Activa el completion de un comando específico.
<code>eselect bashcomp disable <nombre></code>	Desactiva uno puntual (útil si genera lentitud o conflictos).
<code>eselect bashcomp enable --global</code>	Activa todos los completions disponibles de una vez.

```
# Ejemplo: ver qué completions hay disponibles
root # eselect bashcomp list

# Ejemplo: activar solo el de git
root # eselect bashcomp enable git
```

Analogía con NixOS: es parecido a poner `programs.bash.enableCompletion = true;` en tu `configuration.nix` — activa el mecanismo, pero aquí en Gentoo el paquete se emerge de forma explícita y luego puedes afinar comando por comando con `eselect`, en vez de una sola opción declarativa global.

Verificación rápida

Para confirmar que quedó activo, abre una terminal nueva y prueba:

```
$ emerge --
# Debería listar todas las opciones largas de emerge (--ask, --deep, --update, ...)
```

¿Es necesario para tu instalación?

No. Es completamente independiente de OpenRC, GRUB, dracut o cualquier paso crítico. Puedes instalarlo ahora o después del primer arranque sin ningún impacto en el resto del proceso.

Capítulo 41 — Sincronización de tiempo

Gentoo Handbook · Configuración recomendada: **chrony** + **OpenRC**

¿Por qué importa este paso? Tu sistema necesita saber la hora exacta. Sin ella, la verificación de certificados HTTPS falla, los paquetes descargados pueden parecer "corruptos" por errores de fecha, y los logs quedan desordenados. Se corrige sincronizando contra servidores de internet vía el protocolo **NTP** (Network Time Protocol).

1. El problema que resuelve

Hay dos escenarios donde esto se necesita:

- **Sin RTC funcional** (reloj de hardware de la placa madre): el sistema **debe** fijar la hora en cada arranque, siempre.
- **Con RTC**: igual conviene, porque la pila del RTC puede fallar o el reloj se desfasa ("clock skew") con el tiempo.

2. Opción recomendada: chrony

Es la implementación de NTP que sugiere el handbook. Corre en segundo plano como un servicio (*daemon*) llamado `chronyd`, que consulta servidores de tiempo confiables y ajusta tu reloj de forma gradual y suave, sin saltos bruscos que rompan procesos en ejecución.

⚠ **No confundir:** `chrony` (sincroniza el reloj) vs `cronie` (Cap. 37, ejecuta tareas programadas). Suenan parecido, son cosas distintas.

Instalar chrony

riesgo bajo · reversible

```
root # emerge --ask net-misc/chrony
```

Para qué sirve: descarga e instala el paquete. "Emerger" solo lo agrega a tu sistema, no cambia nada en el arranque todavía.

2.1 Activarlo — OpenRC

tu configuración

Añadir al runlevel default

riesgo bajo

```
root # rc-update add chronyd default
```

Para qué sirve: le dice a OpenRC que arranque `chronyd` automáticamente cada vez que enciendes la máquina, en el runlevel normal (`default`). Es el mismo patrón que ya usaste con `sshd` o `cronie`.

2.2 Activarlo — systemd

Habilitar el servicio

riesgo bajo

```
root # systemctl enable chronyd.service
```

Para qué sirve: equivalente exacto a `rc-update add` pero en el mundo systemd — registra el servicio para que systemd lo lance al iniciar.

▼ Configuración y uso de chrony (para enriquecer)

El archivo de configuración vive en `/etc/chrony/chrony.conf`. Ahí defines qué servidores NTP usar, por ejemplo:

```
pool 2.gentoo.pool.ntp.org iburst
```

Una vez corriendo, se administra con la herramienta de línea de comandos `chronyc` :

Comando	Qué hace
<code>chronyc tracking</code>	Muestra el estado actual de sincronización (desfase, precisión)
<code>chronyc sources</code>	Lista los servidores NTP configurados y su calidad de conexión
<code>chronyc makestep</code>	Fuerza un ajuste inmediato si el desfase es grande

También existe el modo puntual, útil justo durante la instalación (ya lo usaste en el capítulo de configuración inicial):

```
root # chronyd -q
```

Esto sincroniza una sola vez y termina, sin quedar como servicio corriendo.

3. Alternativa: systemd-timesyncd

Solo disponible si usas **systemd** como init system (no aplica a tu instalación con OpenRC). Es un cliente SNTP más simple y liviano que chrony — ya viene instalado por defecto en sistemas systemd, solo hay que activarlo:

Habilitar systemd-timesyncd

riesgo bajo

```
root # systemctl enable systemd-timesyncd.service
```

Para qué sirve: activa el sincronizador ligero integrado en systemd. No requiere instalar ningún paquete aparte.

▼ ¿Dónde se configura systemd-timesyncd?

En `/etc/systemd/timesyncd.conf`, sección `[Time]`, con la directiva `NTP=` para especificar servidores. Se consulta el estado con:

```
$ timedatectl timesync-status
```

4. Comparativa rápida

Aspecto	chrony	systemd-timesyncd
Init system	OpenRC o systemd	Solo systemd
Config	<code>/etc/chrony/chrony.conf</code>	<code>/etc/systemd/timesyncd.conf</code>
Herramienta CLI	<code>chronyc</code>	<code>timedatectl</code>
Precisión / funciones	Más completo (puede servir hora a otros equipos, mejor manejo de redes inestables)	Básico, solo cliente
Instalación	Requiere <code>emerge</code>	Ya incluido con systemd

🔄 **Analogía NixOS:** similar a declarar `services.chrony.enable = true;` en `configuration.nix`. En Gentoo el proceso es imperativo: primero `emerge` instala el paquete, luego `rc-update add` lo "activa" explícitamente para el arranque.

5. Resumen ejecutable (tu caso: OpenRC)

Los dos únicos comandos que necesitas

riesgo bajo

```
root # emerge --ask net-misc/chrony
root # rc-update add chronyd default
```

Capítulo 42 — Herramientas de Sistemas de Archivos

Filesystem tools: qué son, para qué sirven y cómo instalarlas correctamente antes de continuar con el bootloader.

Idea central del capítulo: ya creaste tus particiones y les diste formato (con `mkfs.xfs`, `mkfs.ext4`, etc.). Pero **formatear** no es lo mismo que **mantener**. Este capítulo trata sobre instalar las herramientas de administración de cada filesystem — las que usarás *después*, ya con el sistema arrancado, para revisar, reparar o redimensionar tus particiones.

¿Para qué sirve esto?

Cada tipo de filesystem (XFS, ext4, btrfs, etc.) es en realidad un formato de datos distinto en el disco, y cada uno tiene su propio conjunto de programas para administrarlo: revisar errores, reparar corrupción, agrandar o achicar la partición, ver estadísticas de uso, etc. El kernel sabe *leer y escribir* esos formatos, pero las tareas de mantenimiento las hacen programas de espacio de usuario (userspace), no el kernel.

Si no instalas el paquete correspondiente, tu sistema instalado se quedará **sin herramientas de mantenimiento** para su propio disco — por ejemplo, no podrás correr un `fsck` si algo se corrompe después de un apagón.

Analogía con NixOS: es como declarar `boot.supportedFilesystems` en tu `configuration.nix`. No basta con que el kernel soporte el filesystem — también necesitas los paquetes de utilidades correspondientes instalados para poder administrarlo día a día.

Punto clave: ext4 ya viene incluido

Las herramientas de **ext4** (`sys-fs/e2fsprogs`) ya están instaladas por defecto en cualquier sistema Gentoo, porque forman parte del `@system set` — el conjunto mínimo de paquetes que Portage considera imprescindibles. Si tu partición raíz es ext4, no tienes que hacer nada en este capítulo.

Para cualquier otro filesystem (XFS, btrfs, VFAT, F2FS, NTFS, ZFS), **sí tienes que emergerlo tú mismo**, dentro del chroot, antes de reiniciar.

Tabla de referencia: qué instalar según tu filesystem

--	--	--

Filesystem	Paquete	Herramienta principal de reparación/chequeo
XFS	<code>sys-fs/xfsprogs</code>	<code>xfs_repair</code>
ext4	<code>sys-fs/e2fsprogs</code> <i>(ya instalado)</i>	<code>e2fsck</code>
VFAT (FAT32)	<code>sys-fs/dosfstools</code>	<code>fsck.vfat</code>
btrfs	<code>sys-fs/btrfs-progs</code>	<code>btrfs check</code>
F2FS	<code>sys-fs/f2fs-tools</code>	<code>fsck.f2fs</code>
NTFS	<code>sys-fs/ntfs3g</code>	<code>ntfsfix</code>
ZFS	<code>sys-fs/zfs</code>	<code>zpool scrub</code>

El comando

RIESGO BAJO — ES SOLO INSTALAR PAQUETES, TOTALMENTE REVERSIBLE

El patrón general es siempre el mismo:

```
emerge --ask sys-fs/<paquete-correspondiente>
```

Por ejemplo, si tu raíz es XFS:

```
root # emerge --ask sys-fs/xfsprogs
```

¿En dónde se ejecuta esto?

Todavía estás dentro del **chroot** (el sistema nuevo, no el live environment). No hay ningún archivo de configuración que editar en este capítulo — es puramente instalar el paquete correcto vía Portage.

Profundizando: qué hace cada herramienta y sus opciones más usadas

xfsprogs (XFS) — comandos y opciones útiles

Para qué sirve: XFS es el filesystem recomendado por el handbook para uso general moderno. `xfsprogs` trae todo lo necesario para crearlo, revisarlo y mantenerlo.

- `mkfs.xfs` — crea el filesystem (ya lo usaste en el capítulo de discos).
- `xfs_repair /dev/sdaX` — repara corrupción. **Importante:** la partición debe estar desmontada.
- `xfs_growfs /` — agranda el filesystem *en caliente* (con el sistema montado) después de expandir la partición subyacente. XFS no se puede achicar.

- `xfs_info /` — muestra información técnica del filesystem (tamaño de bloque, geometría, etc.).



e2fsprogs (ext4) — comandos y opciones útiles

Para qué sirve: conjunto de herramientas para ext2/ext3/ext4. Ya viene instalado por defecto.

- `e2fsck -f /dev/sdaX` — revisa y repara. La opción `-f` fuerza el chequeo aunque el filesystem se vea "limpio".
- `resize2fs /dev/sdaX` — agranda o achica el filesystem (a diferencia de XFS, ext4 sí se puede achicar, pero solo desmontado).
- `tune2fs -l /dev/sdaX` — muestra parámetros configurados (UUID, label, intervalos de chequeo automático, etc.).

dosfstools (VFAT/FAT32) — comandos y opciones útiles

Para qué sirve: necesario para administrar tu partición EFI (`/efi`), que obligatoriamente es FAT32 por especificación UEFI.

- `fsck.vfat -a /dev/sdaX` — revisa y repara automáticamente (`-a` = auto-fix sin preguntar).
- `fatlabel /dev/sdaX NOMBRE` — cambia la etiqueta de la partición.

Relevante para tu instalación: ya que usas GPT + UEFI, tu partición `/efi` depende de este paquete. Aunque técnicamente no es obligatorio tenerlo instalado en el sistema final (el kernel puede montarlo sin problema), sí lo necesitas si algún día quieres reparar o redimensionar esa partición.

btrfs-progs, f2fs-tools, ntfs3g, zfs — solo si aplica

Estos no aplican a tu configuración actual (XFS/ext4 + FAT32), pero se dejan documentados para referencia:

- **btrfs-progs:** `btrfs check`, `btrfs balance` (redistribuye datos entre discos), `btrfs subvolume` (crea subvolúmenes tipo snapshot).
- **f2fs-tools:** pensado para memorias flash (USB, microSD). `fsck.f2fs`, `resize.f2fs`.
- **ntfs3g:** `ntfsfix` repara errores básicos; no reemplaza `chkdsk` de Windows para reparaciones profundas.
- **zfs:** administración por *pools*, no por partición individual — `zpool status`, `zpool scrub` (verifica integridad de datos con checksums).

¿Aplica systemd aquí?

No. Este capítulo es agnóstico al sistema de init — `emerge` y las herramientas de filesystem funcionan exactamente igual bajo OpenRC o systemd, porque son utilidades de Portage/kernel, no servicios. La diferencia OpenRC/systemd solo aparece cuando se trata de *automatizar* chequeos (por ejemplo, systemd tiene unidades `fstrim.timer` para SSDs vía `systemctl enable fstrim.timer`; en OpenRC eso se haría con un cronjob).

Extra: recomendación adicional del handbook

Si tu disco es un SSD/NVMe, el handbook recomienda instalar `sys-block/io-scheduler-udev-rules` para que el sistema use el planificador de E/S correcto automáticamente:

```
root # emerge --ask sys-block/io-scheduler-udev-rules
```

Checklist rápido antes de avanzar al capítulo 43

- ✓ ¿Tu raíz es ext4? → no hay nada que hacer.
- ✓ ¿Tu raíz es XFS u otro? → corre `emerge --ask sys-fs/<paquete>`.
- ✓ ¿Vas a mantener tu `/efi` a futuro? → considera `dosfstools` si no lo tienes.
- → Siguiendo paso: capítulo 43-44, selección e instalación del bootloader (GRUB) — el paso más delicado de toda la instalación.

Cap. 43 · Selección del Bootloader (concepto)

Gentoo Handbook AMD64 — resumen enriquecido en español, enfocado en UEFI + OpenRC.

43. Selección del bootloader

¿Para qué sirve? El bootloader es lo primero que se ejecuta tras el firmware (UEFI/BIOS). Sin él, el kernel de Linux nunca se carga en memoria — el equipo se queda "mirando" el firmware sin saber a dónde ir.

Este capítulo no tiene comandos: es la bifurcación de decisión antes de instalar. El manual documenta 3 rutas para UEFI (tu caso) y 1 para BIOS legado.

Vocabulario clave: `emerge` vs `install`

Término	Qué hace	Riesgo
<code>emerge</code>	Compila/descarga el paquete del bootloader vía Portage. Solo lo deja disponible en el sistema.	Bajo reversible
<code>install</code>	El bootloader escribe físicamente en el disco o en la partición EFI, quedando activo para el próximo arranque.	Alto paso crítico

Comparativa de opciones (UEFI)

GRUB DEFAULT

Más maduro, mayor compatibilidad (BIOS + UEFI), detecta otros SO (dual boot con `os-prober`).
Más pesado y con más piezas móviles.

systemd-boot

Chainloader ligero. Funciona con OpenRC o systemd. Buen soporte de Secure Boot.
Requiere `kernel-install` de systemd — kernels van a `/efi` , no a `/boot` .

EFI Stub

El kernel se arranca directo desde el firmware, sin bootloader intermedio.
Minimalista pero rígido; no garantizado en todo firmware UEFI.

Analogía NixOS: en NixOS `nixos-rebuild switch` reconfigura el bootloader automáticamente a partir de `configuration.nix` . En Gentoo separas manualmente: 1) `emerge` el paquete, 2) ejecutas el comando de `install` al disco, 3) generas el archivo de configuración. Mismo concepto, sin la capa declarativa.

📍 Dónde se configura todo: `sys-kernel/installkernel`

Esto es clave: el paquete `installkernel` es el "pegamento" entre instalar un kernel nuevo y que tu bootloader elegido se entere de que existe. Su comportamiento cambia según qué USE flags actives — **ahí es donde realmente "seleccionas" el bootloader**, no solo al emerger GRUB/systemd-boot por separado.

Bootloader	USE flags en <code>installkernel</code>	Dónde va el kernel
GRUB	<code>grub</code> (auto-corre <code>grub-mkconfig</code>)	<code>/boot</code>
systemd-boot	<code>systemd</code> <code>systemd-boot</code>	<code>/efi</code>
EFI Stub	<code>efistub</code>	<code>/efi</code>

Ejemplo para tu setup (GRUB + OpenRC, ya configurado según tus notas previas)

```
sys-kernel/installkernel grub dracut
```

💡 Esta es la razón por la que ya tenías `grub dracut` escrito en `/etc/portage/package.use/` desde antes de llegar a este capítulo: ese flag es el que hace que, cada vez que instales un kernel nuevo, `grub-mkconfig` se ejecute automáticamente.

Capítulo 44 · Bootloader — GRUB y alternativas

Instalación de Gentoo · Configuración: GPT + UEFI · dracut · OpenRC

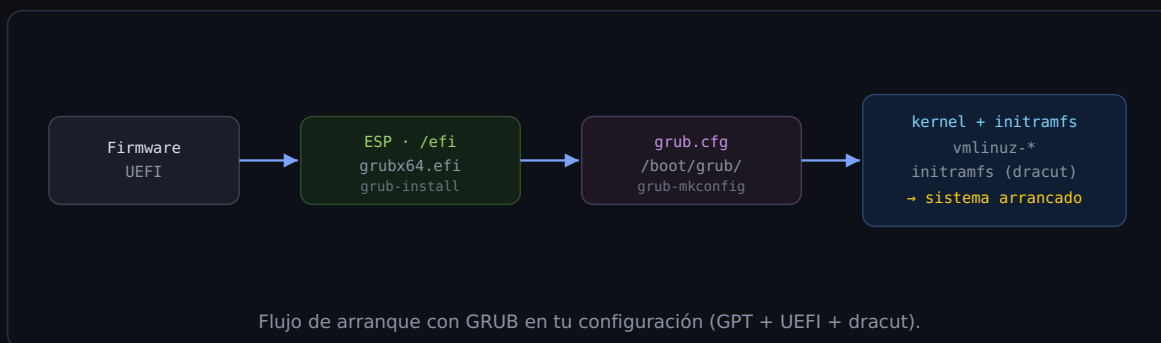
¿Para qué sirve este capítulo?

El **bootloader** es el primer programa que corre después del firmware (UEFI). Su único trabajo es encontrar el kernel de Linux, cargarlo en memoria y arrancarlo con los parámetros correctos. Sin bootloader, el firmware enciende pero no sabe qué sistema operativo cargar.

Dos verbos distintos

Emerge = descargar/compilar el paquete con Portage (reversible, sin riesgo).

Install = escribir el bootloader físicamente en el disco o en la partición EFI (ESP) — esto es lo que lo activa para el próximo arranque.



44.1 — Emerge (instalar el paquete GRUB)

Para qué sirve: descargar y compilar GRUB con las opciones necesarias para tu firmware (UEFI en tu caso).

➤ Se configura en: `/etc/portage/make.conf`

HABILITAR SOPORTE UEFI ANTES DE COMPILAR

RIESGO BAJO

```
echo 'GRUB_PLATFORMS="efi-64"' >> /etc/portage/make.conf
emerge --ask sys-boot/grub
```

Opciones de GRUB_PLATFORMS

No solo existe `efi-64`. GRUB soporta múltiples plataformas de arranque en la misma variable (separadas por espacio): `pc` (BIOS legacy), `efi-32`, `efi-64`, `coreboot`, `ieee1275` (Open Firmware/PowerPC), entre otras. Solo necesitas la que corresponda a tu firmware real.

Si ya compilaste GRUB sin esa variable, se recompila así:

RECOMPILAR CON NUEVAS USE/VARIABLES

RIESGO BAJO

```
emerge --ask --update --newuse --verbose sys-boot/grub
```

44.2 — Install (grabar GRUB en el disco/ESP)

Para qué sirve: copiar los archivos ejecutables de GRUB a la partición donde el firmware los va a buscar al encender la máquina.

Punto de no retorno

Este es el paso más delicado de toda la instalación. Si la partición ESP (`/efi`) no está montada al ejecutar el comando, GRUB puede instalarse en la carpeta equivocada **sin mostrar ningún error**, dejando el sistema sin forma de arrancar.

44.2.1 — Legacy BIOS (no aplica a tu setup)

🔗 Se ejecuta en: `chroot`, con el disco completo como destino

INSTALAR EN EL MBR DEL DISCO

RIESGO ALTO

```
grub-install /dev/sda
```

Se instala en el disco entero (no en una partición), porque BIOS legacy no usa un sistema de archivos para el bootloader.

44.2.2 — UEFI (tu caso)

🔗 Requiere: `/efi` ya montada

INSTALAR EN LA PARTICIÓN EFI

RIESGO ALTO

```
grub-install --efi-directory=/efi
```

Copia los archivos (como `grubx64.efi`) dentro de `/efi/EFI/Gentoo/` . El firmware UEFI guarda una entrada de arranque apuntando a esa ruta.

▼ Firmware UEFI "raro" que solo busca `/EFI/BOOT/`

Algunas placas madre con implementaciones UEFI defectuosas ignoran las entradas de arranque registradas y solo cargan el archivo ubicado en la ruta genérica `/EFI/BOOT/BOOTX64.EFI` . Para ese caso, se usa la opción `--removable` , que fuerza a GRUB a instalarse también ahí:

INSTALACIÓN EN MODO "REMOVABLE"

RIESGO MEDIO

```
grub-install --target=x86_64-efi --efi-directory=/efi --removable
```

▼ Secure Boot (opcional, no lo necesitas por defecto)

Para qué sirve: permitir que el sistema arranque con Secure Boot activo en el firmware, verificando que el bootloader y el kernel estén firmados con una llave de confianza.

El mecanismo usa **shim**, un pequeño pre-cargador ya firmado por Microsoft y aceptado por la mayoría de los firmwares UEFI por defecto. El flujo general:

1. Convertir el certificado de tu llave de firma a formato DER
2. Importarlo a la lista de confianza de shim (pide una contraseña temporal)
3. Registrar shim ante el firmware UEFI como entrada de arranque

CONVERTIR CERTIFICADO **RIESGO BAJO**

```
openssl x509 -in /path/to/kernel_key.pem -inform PEM -out /path/to/kernel_key.der -outform DER
```

IMPORTAR A MOKLIST DE SHIM **RIESGO BAJO**

```
mokutil --import /path/to/kernel_key.der
```

REGISTRAR SHIM EN EL FIRMWARE **RIESGO MEDIO**

```
efibootmgr --create --disk /dev/boot-disk --part boot-partition-id \
--loader '\EFI\Gentoo\shimx64.efi' --label 'GRUB via Shim' --unicode
```

Nota

El proceso no termina hasta reiniciar: al arrancar, shim mostrará MokManager pidiendo la contraseña definida en el paso de importación. Ahí se confirma la llave y recién entonces el firmware confía en tu GRUB firmado.

▼ Depurar errores de grub-install

Si aparece el mensaje `"EFI variables are not supported on this system"`, casi siempre significa que el medio live se arrancó en modo **Legacy/BIOS** en vez de UEFI. La solución habitual es repetir la instalación con `--removable` (ver arriba), ya que sobrescribe la ruta genérica que el firmware sí reconoce en modo legacy accidental.

44.3 — Configure (generar grub.cfg)

Para qué sirve: generar el archivo real que GRUB lee al arrancar: qué kernel cargar, qué initramfs usar, y con qué parámetros.

🔗 Se configura en: `/etc/default/grub` (opciones) · Se genera en: `/boot/grub/grub.cfg` GENERAR LA CONFIGURACIÓN FINAL

RIESGO BAJO

```
grub-mkconfig -o /boot/grub/grub.cfg
```

GRUB escanea `/boot/` automáticamente y detecta:

- El kernel más reciente disponible (`vmlinuz-...`)
- La imagen initramfs correspondiente — en tu caso, la que genera **dracut**

Verificación obligatoria

La salida del comando debe mostrar al menos un `Found linux image` y, por usar dracut, también un `Found initrd image`. Si falta alguno, revisa `/boot/` con `ls` — probablemente algo falló antes en la instalación del kernel.

Personalizar parámetros de arranque (por ejemplo, agregar opciones al kernel) se hace editando la variable `GRUB_CMDLINE_LINUX` en `/etc/default/grub` y volviendo a correr `grub-mkconfig`.

▼ Extra: dual-boot con Windows u otra distro

Para qué sirve: detectar automáticamente otros sistemas operativos instalados en discos conectados y agregarlos al menú de GRUB.

INSTALAR DETECTOR DE OTROS SO **RIESGO BAJO**

```
emerge --ask sys-boot/os-prober
grub-mkconfig -o /boot/grub/grub.cfg
```

Detecta Windows 7/8.1/10 y otras distros de Linux automáticamente.

Alternativas a GRUB

GRUB es la opción por defecto porque es flexible (menú, parámetros editables en caliente, soporte multi-SO), pero en sistemas puramente UEFI existen alternativas más simples. Aquí es donde entra **systemd-boot**, que — a diferencia de GRUB — **no requiere systemd como init**: funciona tanto en sistemas OpenRC como systemd, porque solo necesita el paquete `sys-apps/systemd-utils` (versión mínima sin el init completo).

Bootloader	Qué es	Cuándo conviene
GRUB	Bootloader completo con su propio menú, shell de rescate y soporte para BIOS legacy y UEFI. Puede detectar otros SO (dual-boot) y editar parámetros de arranque interactivamente antes de bootear.	Es el estándar de Gentoo. Ideal si quieres flexibilidad, dual-boot, o si algo falla y necesitas un shell de rescate.
systemd-boot	Un "chainloader" delgado: no tiene menú configurable ni shell propio, solo lista las entradas <code>.conf</code> que le pongas en la ESP y las carga directo. Funciona en OpenRC o systemd.	Sistemas puramente UEFI, sin necesidad de dual-boot complejo. Arranca más rápido y es más simple de mantener. Buen soporte para Secure Boot.
EFI Stub	No es un bootloader separado: el propio kernel de Linux se compila para ser directamente ejecutable por el firmware UEFI (usando la USE flag <code>efistub</code>). El firmware "bootea" el kernel sin intermediarios.	Setups minimalistas donde no quieres ninguna capa extra. Más rígido: cambiar parámetros de arranque requiere recompilar el kernel o usar <code>efibootmgr</code> , no hay menú interactivo.

▼ Comandos: instalar systemd-boot

Para qué sirve: habilitar el chainloader ligero de systemd como alternativa a GRUB, compatible con tu init OpenRC.

🔧 Se configura en: `/etc/portage/package.use/systemd-boot` HABILITAR USE FLAG "BOOT" Y COMPILAR **RIESGO BAJO**

```
# /etc/portage/package.use/systemd-boot
sys-apps/systemd-utils boot

emerge --ask sys-apps/systemd-utils
```

INSTALAR EN LA ESP **RIESGO ALTO**

```
bootctl install
```

Nota para OpenRC

En sistemas con systemd se usa el paquete `sys-apps/systemd` ; en OpenRC (tu caso) se usa `sys-apps/systemd-utils` , que trae solo las herramientas de boot sin instalar el init de systemd.

▼ Comandos: arrancar el kernel como EFI Stub

Para qué sirve: eliminar la capa de bootloader por completo, dejando que el firmware ejecute el kernel directamente.

🔧 Se configura en: `/etc/portage/package.accept_keywords/installkernel(USE flag testing)`

HABILITAR SOPORTE EFI STUB EN EL KERNEL **RIESGO MEDIO**

```
emerge --ask sys-boot/efibootmgr
```

Requiere recompilar el kernel con soporte EFI Stub habilitado y la USE flag `efistub` en `sys-kernel/installkernel` (está bajo keyword *testing*, no es estable). Los parámetros de arranque se "incrustan" en el propio kernel vía `CONFIG_CMDLINE` , o se agregan como entrada de arranque con `efibootmgr` .

Recomendación para tu setup

Dado que ya definiste GRUB + dracut como tu stack, no necesitas cambiar nada — esta sección es solo referencia. GRUB es la opción con más margen de maniobra si algo sale mal durante el arranque (tiene shell de rescate propio), algo valioso mientras sigues aprendiendo Gentoo.

Cap. 45-46 · Bootloaders alternativos

systemd-boot y EFI Stub — la instalación de referencia usa **GRUB**, así que estos capítulos son contexto, no ejecución.

NO APLICA A TU INSTALACIÓN (USAS GRUB)

OPENRC

SYSTEMD

¿Por qué existen 3 formas de arrancar?

Un firmware UEFI ya sabe ejecutar un archivo `.efi` directamente — técnicamente no necesita un "bootloader". Las tres opciones del Handbook son distintos niveles de comodidad sobre esa base:

	GRUB	systemd-boot	EFI Stub
Qué hace	Bootloader completo con menú, detección automática, scripting	Chainloader delgado, solo lee entradas de texto	El kernel es el ejecutable EFI; no hay intermediario
Detecta kernels solo	Sí (<code>grub-mkconfig</code>)	Sí (convención de archivos)	No — se registra cada uno a mano
OpenRC	✓	✓	✓
systemd	✓	✓ (su entorno natural)	✓
Complejidad configuración	Alta	Baja	Media (todo manual)

PARA QUÉ SIRVE SABER ESTO

Sí en el futuro migras a un equipo distinto, cambias a systemd, o simplemente el firmware de una placa da problemas con GRUB, tener claro el panorama te permite elegir la alternativa correcta sin partir de cero.

45 · systemd-boot

Es un cargador minimalista: no tiene lenguaje de configuración propio, solo lee archivos `.conf` por cada entrada de arranque. Funciona igual de bien en OpenRC que en systemd, porque solo necesita el binario y la herramienta `bootctl` — no arrastra el resto del ecosistema systemd (PID 1, journald, etc.).

45.1 Emerge — instalar el paquete correcto

DÓNDE SE CONFIGURA

`/etc/portage/package.use/systemd-boot`

```
FILE /etc/portage/package.use/systemd-boot
sys-apps/systemd boot          # sistemas con systemd
sys-apps/systemd-utils boot    # sistemas con OpenRC
```

```
root # emerge --ask sys-apps/systemd          # systemd
root # emerge --ask sys-apps/systemd-utils    # OpenRC
```

El USE flag `boot` es el interruptor que activa el componente de instalación/gestión de `systemd-boot` dentro de ese paquete. Sin él, el paquete se compila sin esa funcionalidad.

45.2 Instalación — un solo comando

```
root # bootctl install
```

PARA QUÉ SIRVE

Copia el binario `systemd-bootx64.efi` a la ESP (tu `/efi`) y lo registra en el firmware. No existe un paso equivalente a `grub-mkconfig`: las entradas de arranque las genera `installkernel` automáticamente cada vez que instalas un kernel, no `bootctl`.

RIESGO: BAJO, PERO CON CONDICIÓN

La ESP debe estar montada (en `/efi`) antes de correr el comando, o `systemd-boot` no tendrá dónde escribir.

▼ Verificar que la entrada de arranque quedó registrada

```
root # bootctl list
```

Muestra las entradas detectadas. Si el kernel line de arranque parece incorrecto (por ejemplo, usa parámetros del live CD), crea manualmente `/etc/kernel/cmdline` con la línea correcta y reinstala el kernel.

45.3 Secure Boot (opcional)

Dos caminos según qué acepte tu firmware:

Escenario	Qué hacer
El firmware acepta certificados propios	Activar USE flag <code>secureboot</code> globalmente → Portage firma el ejecutable automáticamente, <code>bootctl install</code> instala la versión ya firmada.
El firmware solo confía en Microsoft (caso más común)	Usar shim como intermediario pre-firmado.

▼ Configurar shim paso a paso

```
root # emerge --ask sys-boot/shim sys-boot/mokutil sys-boot/efibootmgr
root # bootctl install --no-variables
root # cp /usr/share/shim/B00TX64.EFI /efi/EFI/systemd/shimx64.efi
```

```
root # cp /usr/share/shim/mmx64.efi /efi/EFI/systemd/mmx64.efi
```

Shim ya está firmado por Microsoft, así que casi cualquier placa lo acepta por defecto. Shim, a su vez, confía en tu certificado propio — hay que "presentárselo":

```
root # openssl x509 -in /path/kernel_key.pem -inform PEM -out /path/kernel_key.der -outform DER
root # mokutil --import /path/kernel_key.der
```

Registrar shim con el firmware (reemplaza `boot-disk` y `boot-partition-id`):

```
root # efibootmgr --create --disk /dev/boot-disk --part boot-partition-id \
--loader '\EFI\systemd\shimx64.efi' \
--label 'Systemd-boot via Shim' \
--unicode '\EFI\systemd\systemd-bootx64.efi'
```

NOTA

La importación no se completa hasta reiniciar. Al arrancar aparecerá **MokManager** pidiendo la contraseña que definiste en `mokutil --import` — ahí confirmas que confías en esa llave, y luego ya puedes activar Secure Boot en el menú UEFI.

46 · EFI Stub — arranque sin bootloader

El enfoque minimalista: el kernel compilado es el ejecutable EFI. No hay menú, no hay configuración intermedia — cada entrada se registra directamente en el firmware.

```
firmware UEFI → lee entrada NVRAM (efibootmgr) → ejecuta bzImage.efi directamente
```

Preparar el USE flag y el kernel

DÓNDE SE CONFIGURA

```
/etc/portage/package.accept_keywords/installkernel y /etc/portage/package.use/installkernel
```

```
FILE /etc/portage/package.accept_keywords/installkernel
sys-kernel/installkernel **
```

```
FILE /etc/portage/package.use/installkernel
sys-kernel/installkernel efistub
```

POR QUÉ HAY QUE ACEPTAR KEYWORDS "TESTING"

EFI Stub no está garantizado en todas las implementaciones UEFI (varían mucho entre fabricantes), por eso el flag `efistub` está enmascarado como inestable — Gentoo te obliga a aceptarlo explícitamente.


```
root # emerge --ask sys-kernel/installkernel
root # mkdir -p /efi/EFI/Gentoo
root # emerge --ask --config sys-kernel/gentoo-kernel{,-bin} # kernels de distro
root # make install # kernel manual
```

▼ Alternativa: copiar el kernel a mano (sin installkernel)

```
root # mkdir -p /efi/EFI/Gentoo
root # cp /boot/vmlinuz-* /efi/EFI/Gentoo/bzImage.efi
```

Registrar la entrada de arranque

```
root # emerge --ask sys-boot/efibootmgr
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" \
  --loader "\EFI\Gentoo\bzImage.efi"
```

PARA QUÉ SIRVE EFIBOOTMGR

No es el bootloader en sí — es la herramienta que edita las **variables EFI** (la lista de entradas de arranque que vive en el firmware, tipo NVRAM). Con GRUB o systemd-boot esto lo hacen ellos solos; aquí lo haces tú manualmente por cada kernel.

Si usas initramfs, se agrega como parámetro extra en la misma entrada:

```
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" \
  --loader "\EFI\Gentoo\bzImage.efi" \
  --unicode "initrd=\EFI\Gentoo\initramfs.img"
```

NOTA IMPORTANTE

El separador de rutas en definiciones UEFI es `\` (backslash), no `/` — es una convención heredada de Windows que el estándar UEFI mantuvo.

46.1 Unified Kernel Image (UKI)

Un UKI empaqueta en **un solo archivo** `.efi` : kernel + initramfs + línea de comandos de arranque. Es la forma más "limpia" de usar EFI Stub, porque reduces todo a un único ejecutable en vez de gestionar kernel e initramfs por separado.

Init	Paquete/flag necesario
systemd	<code>sys-apps/systemd boot</code>
OpenRC	<code>sys-apps/systemd-utils boot kernel-install</code>

Requiere un "stub loader" — actualmente solo existe `systemd-stub`, por eso aparece systemd/systemd-utils aunque estés en OpenRC.

▼ Generar el UKI con dracut o ukify

```
FILE /etc/portage/package.use/uki (con dracut)
sys-kernel/installkernel dracut uki
```

```
FILE /etc/portage/package.use/uki (con ukify)
sys-kernel/installkernel dracut ukify uki
```

dracut puede generar initramfs y UKI. **ukify** solo genera el UKI final — el initramfs lo sigue haciendo dracut por detrás.

```
root # emerge --ask sys-kernel/installkernel
```

Entrada de arranque para el UKI ya generado (queda en `/efi/EFI/Linux/`):

```
root # efibootmgr --create --disk /dev/sda --part 1 --label "gentoo" \
--loader "\EFI\Linux\gentoox.y.z.efi"
```

CON SECURE BOOT ACTIVO

El cmdline queda "grabado" dentro del UKI firmado. El firmware **ignora** cualquier parámetro pasado desde fuera — es intencional, para que nadie altere el arranque sin volver a firmar el archivo.

| En tu instalación (GRUB + dracut)

No ejecutas nada de este capítulo. El único punto de contacto real con tu setup: **dracut** también genera tu initramfs para GRUB — es la misma herramienta, solo que aquí se usa de forma "fusionada" (UKI) en vez de como archivo separado.

Capítulo 47 — Otras alternativas de bootloader

No aplica a tu instalación

GRUB ya seleccionado

Solo referencia

Para qué sirve este capítulo: No instala nada. Es una nota del Handbook que reconoce que GRUB, systemd-boot y EFI Stub no son las únicas opciones de arranque para Linux — solo son las tres que Gentoo documenta en detalle. Si ninguna te sirviera, aquí te dirigen a la [lista completa de bootloaders](#) en la wiki.

¿Por qué existen varios bootloaders?

Un bootloader tiene un solo trabajo real: decirle al hardware dónde está el kernel de Linux y cómo arrancarlo. Pero hay distintas formas de resolver ese problema, dependiendo de cuánto control quieras tener en el momento de arrancar (editar parámetros, elegir entre varios kernels, dual-boot con otro sistema operativo, etc.) contra qué tan simple quieres que sea el proceso.

Las tres opciones que cubre el Handbook representan tres niveles distintos de esa balanza: **control total** (GRUB), **simplicidad con menú** (systemd-boot), y **mínima intervención** (EFI Stub, sin bootloader intermedio).

Comparativa de las tres opciones

1. GRUB Tu elección

Para qué sirve: Es el bootloader "todoterreno". Presenta un menú de arranque real donde puedes elegir entre múltiples kernels instalados, editar parámetros de arranque al vuelo sin reiniciar de nuevo, y detectar otros sistemas operativos instalados (Windows, otras distros) para dual-boot.

Cómo se usa: Se instala una sola vez con `grub-install` (escribe los archivos ejecutables EFI en la partición EFI) y se configura con `grub-mkconfig`, que genera `grub.cfg` escaneando automáticamente qué kernels e initramfs existen en `/boot`.

```
# Instalar en el sistema (una vez)
grub-install --target=x86_64-efi --efi-directory=/efi

# Generar/actualizar configuración (cada vez que cambia el kernel)
grub-mkconfig -o /boot/grub/grub.cfg
```

Opción útil: `app-portage/mirrorselect` no aplica aquí, pero sí `sys-boot/os-prober` — si algún día agregas dual-boot, este paquete detecta otros SO y los agrega al menú automáticamente al volver a correr `grub-mkconfig`.

systemd vs OpenRC: No hay diferencia — GRUB no depende del init system, funciona igual

en ambos.

2. systemd-boot Alternativa simple

Para qué sirve: Es un "chainloader delgado" — en vez de generar lógica compleja como GRUB, simplemente lee archivos de configuración individuales (uno de texto plano por cada entrada de arranque) y los muestra como opciones. No tiene scripting ni detección automática de otros sistemas operativos.

Cómo se usa: Se instala con `bootctl install` (no con `grub-install`) y cada entrada de arranque es un archivo `.conf` separado dentro de `/efi/loader/entries/`, en vez de un único archivo generado como `grub.cfg`.

Aclaración importante: A pesar del nombre, **no requiere systemd como init**.

Funciona igual en máquinas con OpenRC — el nombre viene de que el proyecto systemd lo desarrolla, no de una dependencia técnica.

Cuándo elegirlo en vez de GRUB: Si quieres algo minimalista, con Secure Boot fácil de configurar, y no necesitas edición dinámica de parámetros de arranque ni detección de otros sistemas operativos.

3. EFI Stub Alternativa mínima

Para qué sirve: Elimina el bootloader secundario por completo. El firmware UEFI moderno puede arrancar el kernel de Linux *directamente*, porque el propio kernel se compila para comportarse como una aplicación EFI ("stub" = capa mínima de arranque incrustada en el kernel).

Cómo se usa: No hay un paquete bootloader que instalar. En vez de eso, se activa la USE flag `efistub` en `sys-kernel/installkernel`, se copia el kernel compilado a la partición EFI, y se registra una entrada de arranque en el firmware usando `efibootmgr` (el mismo paquete que usarías para gestionar entradas de GRUB, pero aquí es el único mecanismo de gestión).

```
# Habilitar la USE flag (requiere aceptar keywords de testing)
# /etc/portage/package.use/installkernel
sys-kernel/installkernel efistub

# Registrar la entrada de arranque directamente en el firmware
efibootmgr --create --disk /dev/sda --part 1 \
  --label "gentoo" --loader "\EFI\Gentoo\bzImage.efi"
```

Trade-off clave: Ganas simplicidad (menos piezas que pueden fallar) pero pierdes flexibilidad — cambiar parámetros del kernel implica reconfigurar/recompilar en vez de editar un archivo de texto como en GRUB.

¿Y las que ni siquiera están en el Handbook?

El capítulo 47 apunta a la [lista completa de bootloaders soportados por Gentoo](#), que incluye opciones como **rEFInd** (menú gráfico con detección automática de sistemas, popular en equipos con múltiples SO o Hackintosh), **Limine** (rápido y minimalista, gana popularidad reciente), o **syslinux/extlinux** (más orientado a sistemas Legacy BIOS o embebidos). Ninguno se documenta paso a paso en el Handbook porque no son la ruta "por defecto" ni la más común entre usuarios de Gentoo.

▼ ¿Vale la pena cambiar de GRUB a otra opción?

Para tu caso: no. Ya invertiste en configurar GRUB con dracut y OpenRC, que es la combinación más probada y documentada. Cambiar de bootloader solo tiene sentido si te topas con un problema específico que GRUB no resuelve bien (por ejemplo, quieres Secure Boot muy simple → systemd-boot, o quieres cero capas intermedias → EFI Stub). Fuera de eso, GRUB es la opción más flexible y con mejor soporte de la comunidad.

Dónde se configura — resumen

Elemento	Ubicación	Bootloader
Config. de compilación de GRUB	<code>/etc/portage/make.conf</code> (<code>GRUB_PLATFORMS</code>)	GRUB
Parámetros de arranque (cmdline)	<code>/etc/default/grub</code>	GRUB
Config. generada final	<code>/boot/grub/grub.cfg</code>	GRUB
Entradas de arranque individuales	<code>/efi/loader/entries/*.conf</code>	systemd-boot
USE flag para arranque directo	<code>/etc/portage/package.use/installkernel</code>	EFI Stub
Entradas registradas en firmware	Gestionadas vía <code>efibootmgr</code> (no en disco)	EFI Stub / cualquiera

Capítulo 48 — Reiniciar el sistema

La prueba de fuego: salir del chroot, desmontar todo y arrancar el Gentoo recién instalado

RIESGO: BAJO (COMANDOS NO DESTRUCTIVOS)

APLICA IGUAL EN OPENRC Y SYSTEMD

¿PARA QUÉ SIRVE ESTE CAPÍTULO?

Hasta ahora todo el trabajo (particiones, base del sistema, kernel, bootloader) vivía "dentro" del entorno chroot, montado sobre el live media. Este capítulo cierra esa sesión de forma limpia y reinicia la máquina apuntando al disco real. Es el momento en el que se comprueba si GRUB, el kernel y `/etc/fstab` quedaron bien configurados.

Los 4 comandos, en orden

▼

1

Salir del chroot

```
(chroot) lived # exit
```

Regresa el prompt al entorno live normal. No se puede desmontar una partición mientras la terminal sigue "parada" dentro de ella (dentro del chroot), así que este paso es obligatorio antes de desmontar.

▼

2

Moverte fuera de

```
/mnt/gentoo
```

```
lived~# cd
```

`cd` sin argumentos manda al `$HOME` del usuario actual (normalmente `/root`). Es un detalle práctico: si la terminal seguía posicionada dentro de `/mnt/gentoo/algo`, el desmontaje de abajo fallaría con `target is busy`.

▼

3

Desmontar

```
/dev
```

y sus submontajes

```
lived~# umount -l /mnt/gentoo/dev{/shm,/pts,}
```

- `{/shm,/pts,}` es expansión de llaves de bash: equivale a escribir tres rutas

(`/mnt/gentoo/dev/shm` , `/mnt/gentoo/dev/pts` , `/mnt/gentoo/dev`) en un solo comando.

- `-l` (*lazy unmount*): desconecta el punto de montaje del árbol de archivos de inmediato, pero si algún proceso lo sigue usando, espera a que termine antes de liberarlo por completo "por debajo".

`/dev` se desmonta aparte porque tiene submontajes anidados; hacerlo explícito aquí evita problemas de orden en el paso siguiente.

▼
4

Desmontar todo lo demás

```
livecd~# umount -R /mnt/gentoo
```

`-R` (recursivo) desmonta `/mnt/gentoo` y todo lo que quedó montado debajo: `/proc` , `/sys` , `/run` , la partición raíz y `/efi` . Portage/el kernel se encargan del orden correcto automáticamente.

▼
5

Reiniciar

```
livecd~# reboot
```

El comando final. La máquina se reinicia y, si todo salió bien, debería arrancar el Gentoo recién instalado en vez del live media.

ANTES DE QUE REINICIE

Retira el USB/CD del medio de instalación. Si no lo haces, el firmware puede volver a bootear el live environment en vez del disco, dependiendo del orden de arranque configurado en la UEFI.

Checklist antes de presionar "reboot"

Estos comandos no revisan nada por ti — vale la pena confirmar manualmente antes de desmontar, sobre todo porque una vez reiniciado, corregir un error del bootloader requiere volver a bootear el live media.

- **GRUB instalado:** `grub-install` terminó sin errores y `grub-mkconfig -o /boot/grub/grub.cfg` generó un archivo con entradas de arranque (revisado en el capítulo 44).
- **/etc/fstab correcto:** las UUID o rutas de las particiones raíz, `/efi` y swap coinciden con `blkid` .
- **Kernel e initramfs presentes:** en `/boot` debe existir el kernel y, dado tu setup con `dracut` , el `initramfs-*.img` correspondiente.
- **Contraseña de root establecida:** si no se hizo, quedarás sin forma de iniciar sesión al reiniciar.

- **Hostname configurado:** revisar `/etc/conf.d/hostname` (OpenRC) para que el sistema no arranque como `localhost`.

Diferencia OpenRC vs. systemd en este paso

Ninguna. Los comandos `exit`, `cd`, `umount` y `reboot` son utilidades genéricas de Linux/util-linux, no dependen del sistema de init. La diferencia entre OpenRC y systemd aparece *después* del reinicio: qué servicios arrancan (`rc-update` vs. `systemctl enable`) y cómo depurar fallas de arranque (logs de OpenRC en `/var/log/` vs. `journalctl` en systemd). Como tu instalación usa OpenRC, cualquier problema post-reinicio se investiga con las herramientas clásicas (`dmesg`, `/var/log/rc.log` si está habilitado) en vez de `journalctl`.

Si el sistema no arranca

NO ENTRES EN PÁNICO

Nada de lo hecho en este capítulo es destructivo. Si el arranque falla, vuelve a bootear con el live media, monta la partición raíz, y repite los pasos de "reanudar la instalación": copiar `resolv.conf`, montar `/proc`, `/sys`, `/dev`, `/run` y `/efi`, luego `chroot /mnt/gentoo /bin/bash` para diagnosticar (revisar `grub.cfg`, volver a correr `grub-install`, o regenerar el initramfs con `dracut`).

Dónde se configura / revisa cada cosa

Elemento	Ubicación	Se revisa en
Configuración de GRUB	<code>/boot/grub/grub.cfg</code>	Cap. 44
Tabla de montajes al arrancar	<code>/etc/fstab</code>	Cap. previo a chroot
Hostname	<code>/etc/conf.d/hostname</code> (OpenRC)	Cap. 35
Generador de initramfs	<code>dracut</code> , invocado por <code>installkernel</code>	Config. inicial (USE flags)
Punto de montaje EFI	<code>/efi</code>	Preparación del bootloader

Capítulo 49 — Mantenimiento de Portage

Limpieza de caché: distfiles y binpkgs con `gentoolkit` / `eclean`

Contenido

- 1. El problema: el caché de Portage crece indefinidamente
- 2. Para qué sirve limpiarlo
- 3. Instalar gentoolkit
- 4. `eclean-dist` — limpiar tarballs de código fuente
- 5. `eclean-pkg` — limpiar paquetes binarios
- 6. Modos de protección (seguro vs. profundo)
- 7. Tabla de opciones disponibles
- 8. Archivos de exclusión (proteger paquetes específicos)
- 9. Extra: otras herramientas de gentoolkit útiles
- 10. Dónde se configura — resumen

1. El problema

Cada vez que `emerge` instala o actualiza un paquete, guarda una copia local del trabajo realizado:

Tipo de archivo	Ubicación por defecto	Variable en <code>make.conf</code>
Tarballs de código fuente descargado	<code>/var/cache/distfiles</code>	<code>DISTDIR</code>
Paquetes binarios compilados (binpkgs)	<code>/var/cache/binhost/gentoo</code>	<code>PKGDIR</code>

El detalle clave: **cuando actualizas un paquete, Portage no borra la versión anterior descargada**. Se queda ahí. Si nunca limpias, estos directorios crecen sin límite — se han reportado casos de varios gigabytes acumulados solo en `DISTDIR` tras meses de uso normal.

2. Para qué sirve conservarlos (y por qué limpiarlos igual)

✓ Ventaja de conservarlos

Si necesitas reinstalar o hacer *downgrade* de un paquete, ya tienes el archivo local — no hay que descargarlo ni recompilarlo de nuevo. También sirve como respaldo si algún día el mirror de Gentoo deja de tener disponible esa versión específica.

✗ Desventaja de no limpiar

Versiones antiguas que ya nadie va a volver a instalar simplemente ocupan espacio en disco de forma indefinida, sin ningún beneficio real.

PARA QUÉ SIRVE ECLEAN

Es la herramienta que resuelve ese balance: identifica automáticamente qué archivos ya no corresponden a ningún paquete instalado (o a ningún ebuild disponible, según el modo) y los elimina, dejando intactos los que sí podrías necesitar.

3. Instalar gentoolkit

`gentoolkit` es un paquete que agrupa varias utilidades de administración para sistemas Gentoo. `eclean` es una de ellas (junto con `equery`, `eread`, `revdep-rebuild`, entre otras).

```
root # emerge --ask app-portage/gentoolkit
```

- 1 Portage resuelve las dependencias del paquete `app-portage/gentoolkit`.
- 2 `--ask` muestra el plan de instalación y pide confirmación antes de proceder.
- 3 Se compila (o se instala desde `binpkg` si aplica) y queda disponible en el sistema.
- 4 A partir de aquí, los comandos `eclean-dist` y `eclean-pkg` están disponibles como atajos de `eclean`.

4. `eclean-dist` — limpiar tarballs de código fuente

SEGURO POR DEFECTO

Limpia archivos obsoletos dentro de `DISTDIR` (`/var/cache/distfiles`).

```
root # eclean-dist
```

▼ Proceso paso a paso de qué hace este comando

- 1 Escanea el árbol de ebuilds de Portage (y overlays, si tienes) para saber qué versiones de qué paquetes existen actualmente como ebuilds disponibles.
- 2 Recorre `DISTDIR` comparando cada archivo contra esa lista.
- 3 Por defecto (modo seguro), conserva cualquier tarball que aún corresponda a **algún** ebuild disponible en el repositorio — esté o no instalado ese paquete en tu sistema.
- 4 Borra únicamente los tarballs que ya no corresponden a ningún ebuild existente (paquetes discontinuados o versiones eliminadas del árbol de Portage).
- 5 Muestra un resumen del espacio liberado al terminar.

NOTA SOBRE VELOCIDAD

Este modo por defecto es el más lento de los tres disponibles, porque necesita revisar todo el árbol de Portage completo (incluyendo overlays) para decidir qué proteger.

5. `eclean-pkg` — limpiar paquetes binarios

ÚTIL SOBRE TODO CON BINPKGS

Limpia archivos obsoletos dentro de `PKGDIR` (`/var/cache/binhost/gentoo`).

```
root # eclean-pkg
```

Funciona con la misma lógica que `eclean-dist`, pero sobre paquetes ya compilados en formato binario en vez de código fuente. Es especialmente relevante si usas la característica `FEATURES="buildpkg"` (que genera un binpkg cada vez que compilas algo) o si configuraste un host de binpkgs.

6. Modos de protección

`eclean` tiene tres niveles de "qué tan agresivo" es al limpiar. Se controlan principalmente con la opción `-d` / `--deep`:

Modo por defecto (seguro)

Protege todo lo que corresponda a **cualquier ebuild disponible** en el árbol de Portage, esté instalado o no. Ideal si quieres poder reinstalar cualquier cosa sin volver a descargar. Más lento y deja más archivos.

`--deep` / `-d`

Solo protege archivos que correspondan a paquetes **instalados actualmente**, en su versión exacta. Libera mucho más espacio, pero si desinstalaste algo por error tendrás que volver a descargarlo. Es el modo más rápido.

Un punto intermedio: combinar `--deep` con `-n` / `--package-names` protege **todas** las versiones de los paquetes instalados (no solo la versión exacta actual), en lugar de solo la última.

```
# Modo agresivo: solo conserva lo mínimo para reinstalar lo que ya tienes
root # eclean-dist --deep

# Modo intermedio: conserva todas las versiones de paquetes instalados
root # eclean-pkg --deep --package-names
```

7. Tabla de opciones disponibles

Opciones globales (aplican a ambos comandos)

Opción	Qué hace
<code>-p</code> , <code>--pretend</code>	Solo muestra qué se borraría, sin borrar nada. Siempre recomendable correrlo primero.
<code>-i</code> , <code>--interactive</code>	Pide confirmación antes de cada eliminación.
<code>-d</code> , <code>--deep</code>	Solo conserva lo mínimo necesario para reinstalar los paquetes ya instalados.

<code>-n</code> , <code>--package-names</code>	Con <code>--deep</code> : protege todas las versiones instaladas, no solo la exacta actual.
<code>-t</code> , <code>--time-limit=<tiempo></code>	No borra archivos modificados recientemente (ej. <code>1y</code> , <code>2w</code> , <code>3d</code>).
<code>-e</code> , <code>--exclude-file=<ruta></code>	Usa un archivo de exclusión personalizado (ver sección 8).
<code>-q</code> , <code>--quiet</code>	Salida mínima, solo errores.
<code>-C</code> , <code>--nocolor</code>	Desactiva colores en la salida.

Opciones exclusivas de `eclean-dist`

Opción	Qué hace
<code>-s</code> , <code>--size-limit=<tamaño></code>	No borra archivos más grandes que el tamaño indicado (ej. <code>50M</code>).
<code>-f</code> , <code>--fetch-restricted</code>	Con <code>--deep</code> : protege archivos marcados como "fetch-restricted" (los que hay que descargar manualmente, no vía mirror).
<code>--skip-vcs</code>	Protege fuentes de control de versiones (ebuilds "live") no instalados.

Opciones exclusivas de `eclean-pkg`

Opción	Qué hace
<code>--changed-deps</code>	Borra binpkgs cuyas dependencias del ebuild original ya cambiaron (quedaron desactualizados respecto al árbol actual).
<code>-i</code> , <code>--ignore-failure</code>	Ignora el error si no logra localizar <code>PKGDIR</code> .

FLUJO RECOMENDADO

1) Corre con `--pretend` para ver qué se borraría → 2) revisa la lista → 3) si todo se ve bien, vuelve a correr el comando sin `--pretend` .

```
# Paso 1: revisar qué se limpiaría, sin tocar nada
root # eclean-dist --pretend
```

```
# Paso 2: ejecutar de verdad
root # eclean-dist
```

8. Archivos de exclusión

Si quieres proteger paquetes específicos de la limpieza (por ejemplo, porque sabes que vas a reinstalar algo pronto), puedes crear un archivo de exclusión:

Comando	Archivo de exclusión usado por defecto
---------	--

<code>eclean-dist</code>	<code>/etc/eclean/distfiles.exclude</code>
<code>eclean-pkg</code>	<code>/etc/eclean/packages.exclude</code>

Formato del archivo (líneas de texto plano):

```
# Proteger toda una categoría
sys-apps

# Proteger un paquete específico (todas sus versiones)
app-shells/bash

# Excluir un paquete de una protección de categoría
!sys-apps/portage

# Proteger un archivo distfile específico por nombre
openoffice-i18n-3.0.tar.gz

# Proteger archivos que coincidan con una expresión regular
.*\sig$
```

9. Extra: otras herramientas de gentoolkit

Ya que instalaste `gentoolkit`, vienen incluidas otras utilidades que valen la pena conocer para el mantenimiento continuo del sistema:

Herramienta	Para qué sirve
<code>equery</code>	Consultar información de Portage: dependencias, metadatos y archivos instalados de un paquete.
<code>eread</code>	Mostrar los mensajes "elog" que Portage guarda durante instalaciones (avisos, notas post-instalación de cada paquete).
<code>revdep-rebuild</code>	Detecta binarios rotos por dependencias faltantes tras actualizaciones y los reconstruye.
<code>quickpkg</code>	Crea un binstrip de un paquete ya instalado, sin recompilarlo.

RELACIÓN CON OTROS COMANDOS DEL HANDBOOK

`eclean` limpia archivos *descargados/compilados* que ya no se usan. Es distinto de `emerge --depclean` (visto en capítulos anteriores), que desinstala **paquetes completos** que ya no son necesarios en el sistema. Ambos se complementan: primero `--depclean` para quitar paquetes sobrantes, luego `eclean-dist` / `eclean-pkg` para limpiar el caché de descargas.

10. Dónde se configura — resumen

Elemento	Ubicación
Directorio de tarballs fuente	<code>/var/cache/distfiles</code> (variable <code>DISTDIR</code> en <code>make.conf</code>)

Directorio de binpkgs	<code>/var/cache/binhost/gentoo</code> (variable <code>PKGDIR</code> en <code>make.conf</code>)
Instalar la herramienta	<code>emerge --ask app-portage/gentoolkit</code>
Limpiar tarballs fuente	<code>eclean-dist</code>
Limpiar binpkgs	<code>eclean-pkg</code>
Exclusión de distfiles	<code>/etc/eclean/distfiles.exclude</code>
Exclusión de binpkgs	<code>/etc/eclean/packages.exclude</code>

NOTA SOBRE OPENRC / SYSTEMD

Este capítulo no tiene pasos diferenciados por sistema de init — `eclean` es una herramienta de Portage que funciona igual sin importar si el sistema usa OpenRC o systemd. No requiere ningún servicio corriendo ni configuración de arranque.

Capítulo 50 — Administración de usuarios

Gentoo Handbook AMD64 · Perfil: OpenRC · Sistema con systemd omitido salvo notas comparativas

Para qué sirve este capítulo: hasta ahora todo el trabajo se ha hecho como `root`, la cuenta con permisos totales sobre el sistema. Este capítulo crea una cuenta de usuario normal para el uso diario y define cómo recuperar permisos de `root` solo cuando se necesiten, en vez de tenerlos siempre activos.

50.1 — Crear un usuario para uso diario

Trabajar como `root` de forma permanente es riesgoso: un comando mal escrito (por ejemplo un `rm -rf` con la ruta equivocada) puede dañar el sistema completo, porque `root` no tiene restricciones. Un usuario normal solo puede afectar sus propios archivos y lo que sus grupos le permitan.

Grupos relevantes

Grupo	Para qué sirve
<code>wheel</code>	El más importante: permite usar <code>su</code> para volverse <code>root</code> . Sin este grupo, el usuario no puede elevar privilegios con <code>su</code> .
<code>users</code>	Grupo genérico base para cuentas de usuario estándar.
<code>audio</code>	Acceso a dispositivos de sonido.
<code>video</code>	Acceso a hardware de video y aceleración gráfica.
<code>cdrom</code> / <code>usb</code>	Acceso directo a unidades ópticas o dispositivos USB.
<code>cron</code>	Permite programar tareas con cron (ver Cap. 37).

Comando: crear el usuario

```
# Ejemplo con un usuario llamado "larry"
root # useradd -m -G users,wheel,audio -s /bin/bash larry
root # passwd larry
```

▼ Ver opciones de `useradd` usadas y disponibles

Flag	Función
<code>-m</code>	Crea la carpeta home (<code>/home/larry</code>) junto con el usuario. Sin esta opción, el usuario no tiene home.
<code>-G</code> <code>grupo1,grupo2</code>	Agrega el usuario a grupos secundarios (separados por coma, sin espacios).

`-s /bin/bash`

Define la shell de login. Si se omite, puede quedar sin shell asignada o usar la que diga `/etc/default/useradd`.

`-u UID`

(No usada arriba) Fuerza un UID específico en vez de que el sistema asigne el siguiente disponible.

`-c`
`"comentario"`

(No usada arriba) Campo GECOS: nombre completo u otra nota descriptiva.

Pasos que ejecuta el proceso

1. `useradd` crea la entrada del usuario en `/etc/passwd` y su grupo primario en `/etc/group`.
2. Con `-m`, copia el contenido de `/etc/skel` a la nueva carpeta `/home/larry` y ajusta los permisos.
3. Con `-G`, añade al usuario como miembro secundario de cada grupo listado (edita `/etc/group`).
4. `passwd larry` pide y confirma la contraseña, y la guarda cifrada en `/etc/shadow`.

Buena práctica: usar una contraseña distinta y no relacionada con la de `root`, de al menos 16 caracteres.

50.2 — Elevar privilegios temporalmente

Una vez que ya no se trabaja como `root` por defecto, se necesita una forma de recuperar esos permisos solo cuando haga falta. Hay tres caminos habituales:

Opción 1: `su -` (incluida de fábrica)

```
user $ su -  
Password: # contraseña de root
```

Para qué sirve: abre una sesión completa como `root` (con el `-` además carga el entorno de `root`, como su `$PATH` y variables). Requiere conocer la contraseña de `root`. Es simple, pero todo o nada: no hay forma de limitar qué comandos se pueden correr.

Opción 2: `sudo` `app-admin/sudo`

Para qué sirve: ejecuta un comando puntual como `root` (u otro usuario) sin abrir una sesión aparte, pidiendo la **contraseña del propio usuario** (no la de `root`). Permite definir con precisión qué usuarios pueden ejecutar qué comandos.

```
# Instalación  
root # emerge --ask app-admin/sudo  
  
# Uso básico  
user $ sudo comando  
  
# Editar la configuración de forma segura (valida sintaxis antes de guardar)  
root # visudo
```


▼ Ver configuración y opciones comunes de sudo

Elemento	Descripción
<code>/etc/sudoers</code>	Archivo principal de reglas. Se edita solo con <code>visudo</code> para evitar dejarlo con errores de sintaxis que bloqueen el acceso.
<code>%wheel ALL=(ALL) ALL</code>	Regla típica: cualquier miembro del grupo <code>wheel</code> puede ejecutar cualquier comando como cualquier usuario.
<code>usuario ALL=(ALL) NOPASSWD: ALL</code>	Permite a ese usuario usar sudo sin pedir contraseña (usar con cuidado).
<code>sudo -i</code>	Abre una sesión interactiva como root, similar a <code>su -</code> .
<code>sudo -u usuario comando</code>	Ejecuta el comando como otro usuario, no necesariamente root.
<code>/etc/sudoers.d/</code>	Directorio para agregar reglas en archivos separados sin tocar el archivo principal.

Opción 3: `doas` `app-admin/doas`

Para qué sirve: alternativa minimalista a sudo, con muchas menos opciones de configuración (menor superficie de ataque) y sintaxis más simple. Buena opción para sistemas de un solo usuario.

```
# Instalación
root # emerge --ask app-admin/doas
```

```
# Configuración: crear el archivo (no existe por defecto)
root # nano -w /etc/doas.conf
```

```
# Ejemplo de regla en /etc/doas.conf: permite a "larry" usar doas
# pidiendo su propia contraseña
permit persist larry as root
```

persist evita que se pida contraseña en cada comando durante unos minutos, igual que hace sudo por defecto.

Comparativa rápida

Método	Contraseña que pide	Configuración	Granularidad
<code>su -</code>	La de root	Ninguna (viene listo)	Ninguna: todo o nada
<code>sudo</code>	La del usuario	<code>/etc/sudoers</code> vía <code>visudo</code>	Alta: por comando, usuario, grupo, host
<code>doas</code>	La del usuario	<code>/etc/doas.conf</code>	Media: reglas simples

Nota systemd: en sistemas con systemd como gestor de servicios existe además `run0`, integrado en systemd, que cumple un rol similar a `sudo` sin necesidad de instalar un paquete aparte. No aplica en esta instalación (perfil OpenRC).

50.3 — Deshabilitar el login de root RIESGO MEDIO

Advertencia del handbook: antes de deshabilitar el login de `root`, hay que confirmar que el usuario normal ya está en el grupo `wheel` y que `su`, `sudo` o `doas` funcionan correctamente. De lo contrario, el sistema queda sin forma de administrarse y se necesita un rescate desde el medio de instalación.

Comando: bloquear el login de root

```
root # passwd -l root
```

Para qué sirve: antepone un carácter inválido al hash de la contraseña en `/etc/shadow`, de forma que ningún login por contraseña puede tener éxito. La cuenta `root` sigue existiendo y sigue siendo accesible vía `su` desde una sesión ya elevada, o revirtiendo el bloqueo.

Alternativa: eliminar la contraseña por completo

```
root # passwd -d root
```

Para qué sirve: borra el hash de la contraseña de `root` en vez de invalidarlo. El resultado práctico es similar (no se puede iniciar sesión por contraseña), pero es un estado distinto a nivel de archivo: `-l` es reversible con `passwd -u root`; con `-d` hay que asignar una contraseña nueva desde cero.

▼ Ver otras opciones útiles de `passwd`

Flag	Función
<code>-l</code>	Bloquea (lock) la cuenta, invalidando el hash actual.
<code>-u</code>	Desbloquea una cuenta previamente bloqueada con <code>-l</code> .
<code>-d</code>	Elimina la contraseña de la cuenta.
<code>-e</code>	Fuerza que la contraseña expire, obligando a cambiarla en el próximo login.
<code>-S</code>	Muestra el estado de la cuenta (bloqueada, con contraseña, etc.) sin modificar nada.

Tabla resumen: ¿dónde se configura?

Elemento	Ubicación / comando
Cuentas de usuario y su grupo primario	<code>/etc/passwd</code> · gestionado con <code>useradd</code>
Membresía a grupos secundarios	<code>/etc/group</code> · gestionado con <code>useradd -G</code> o <code>usermod -aG</code>
Contraseñas cifradas y estado de la cuenta	<code>/etc/shadow</code> · gestionado con <code>passwd</code>
Reglas de sudo	<code>/etc/sudoers</code> (y <code>/etc/sudoers.d/</code>) · editar solo con <code>visudo</code>
Reglas de doas	<code>/etc/doas.conf</code>
Plantilla para nuevos home	<code>/etc/skel</code> (se copia al crear usuario con <code>-m</code>)

Limpieza de disco

Eliminación de artefactos de instalación tras un reboot exitoso

¿PARA QUÉ SIRVE ESTE CAPÍTULO?

Una vez que tu sistema Gentoo ya arrancó correctamente fuera del chroot, los archivos que usaste para *construir* el sistema (el stage3 y sus acompañantes de verificación) ya cumplieron su propósito. Este capítulo es simplemente recoger la basura que dejó el proceso de instalación en el directorio raíz.

1. ¿Qué son estos "artefactos de instalación"?

Cuando descargaste e instalaste el stage3 (capítulo de instalación del stage), dejaste varios archivos en `/`:

- `stage3-amd64-*.tar.xz` — el tarball con el sistema base que extrajiste.
- `*.DIGESTS` / `*.CONTENTS` — archivos con los checksums (SHA256/BLAKE2B) que usaste para verificar que el tarball no estuviera corrupto o alterado.
- `*.asc` — la firma PGP del tarball, usada para confirmar que el archivo lo publicó realmente el equipo de Gentoo.

Ninguno de estos archivos es necesario para que el sistema funcione — son evidencia del proceso de instalación, no parte del sistema instalado.

ANALOGÍA RÁPIDA

Es como cuando instalas un programa desde un `.exe` o `.deb` descargado: una vez instalado, el instalador ya no hace falta. Puedes borrarlo sin que el programa deje de funcionar.

2. Requisito previo IMPORTANTE

Este paso se hace **después** de reiniciar y confirmar que el sistema Gentoo bootea correctamente (capítulo 48, "Rebooting the system"). No lo hagas dentro del entorno live ni antes de comprobar que el arranque fue exitoso — si algo salió mal con GRUB o el kernel, es útil conservar el stage3 por si necesitas volver a chroot y reparar algo.

3. Verificar qué hay antes de borrar RIESGO BAJO

Es buena práctica ver primero qué archivos existen, en vez de borrar a ciegas:

```
# Lista archivos en la raíz que empiecen con "stage3"
root # ls -lh /stage3-*
```

- 1 **Ejecutas** `ls -lh /stage3-*`. El shell expande el patrón (*glob*) y te muestra cada archivo que

coincide, junto con su tamaño en formato legible (`-h`).

- 2 **Confirma visualmente** que ahí está el `.tar.xz` y sus archivos de verificación asociados (`.DIGESTS`, `.asc`, etc.).

4. Eliminar los artefactos

```
root # rm /stage3-*.tar.*
```

- 1 **El shell expande** `/stage3-*.tar.*` a todos los archivos en `/` que empiecen con `stage3-` y contengan `.tar.` en el nombre (esto cubre el `.tar.xz` mismo y variantes como `.tar.xz.asc` o `.tar.xz.sha256`).
- 2 `rm` **borra cada archivo** de la lista resultante, uno por uno, sin pedir confirmación (comportamiento por defecto de `rm` sin la flag `-i`).
- 3 **El espacio en disco se libera** inmediatamente — el stage3 suele pesar varios cientos de MB, así que es una limpieza notoria.

▼ Opciones útiles de `rm` si quieres más control

Opción	Qué hace
<code>-i</code>	Pregunta confirmación antes de borrar cada archivo.
<code>-v</code>	Modo verboso: imprime cada archivo que borra.
<code>-f</code>	Fuerza el borrado sin preguntar, ignora archivos inexistentes.

Ejemplo más seguro para primera vez: `rm -iv /stage3-*.tar.*`

5. Confirmar que se liberó espacio (opcional)

```
root # df -h /
```

`df -h` (*disk free*, formato legible) muestra el espacio usado y disponible en cada sistema de archivos montado. Correrlo antes y después del `rm` te deja ver la diferencia real.

6. Enriquecimiento: no lo confundas con la limpieza de Portage

Este capítulo borra **archivos sueltos de instalación**, algo que se hace una sola vez. Es distinto de la limpieza continua que hace Portage sobre paquetes ya emergidos (vista en el capítulo 49, "Portage maintenance"):

Comando	Qué limpia	Frecuencia

<code>rm</code> <code>/stage3-*.tar.*</code>	El tarball de instalación inicial y sus firmas/checksums.	Una sola vez, tras el primer reboot exitoso.
<code>emerge --depclean</code>	Paquetes huérfanos que ya no dependen de nada instalado.	Periódica, tras actualizaciones.
<code>eclean-dist</code>	Tarballs de código fuente viejos en <code>/var/cache/distfiles</code> (requiere <code>app-portage/gentoolkit</code>).	Periódica.
<code>eclean-pkg</code>	Paquetes binarios viejos en <code>/var/cache/binhost/gentoo</code> .	Periódica.

SYSTEMD VS OPENRC

Este capítulo no tiene diferencias entre init systems — `rm` y `df` son comandos de shell estándar, no dependen de OpenRC ni systemd. Como usas OpenRC, no aplica ninguna variante aquí.

Resumen: dónde se configura / dónde se ejecuta cada cosa

Acción	Ubicación / comando	Cuándo
Ver artefactos pendientes	<code>ls -lh /stage3-*</code>	Antes de borrar, para confirmar qué hay.
Borrar artefactos	<code>rm /stage3-*.tar.*</code> en <code>/</code>	Después del primer reboot exitoso al nuevo sistema.
Verificar espacio liberado	<code>df -h /</code>	Opcional, antes/después del <code>rm</code> .
Limpieza continua de paquetes	<code>emerge --depclean</code> , <code>eclean-dist</code> , <code>eclean-pkg</code>	Mantenimiento periódico (no es parte de este capítulo, ver Cap. 49).

Capítulo 52 — ¿A dónde ir desde aquí?

Handbook de Gentoo AMD64 · Cierre de la instalación y puntos de continuidad

Para qué sirve este capítulo: no instala ni configura nada. Es el mapa de salida: qué leer después, dónde preguntar cuando algo falle, y cómo participar en la comunidad. No hay comandos obligatorios aquí — el sistema ya está instalado y funcionando.

52.1 — Documentación adicional

El Handbook cubre solo lo mínimo para tener un sistema arrancando: partición, chroot, kernel, bootloader, red básica. Deliberadamente **no** cubre:

- Entornos de escritorio (DE) ni gestores de ventanas (WM)
- Hardening (endurecimiento de seguridad)
- Administración avanzada del sistema día a día

Todo eso vive en la **Gentoo Wiki**, fuera del Handbook.

Siguiente lectura recomendada

Working with Gentoo (Trabajando con Gentoo) — es la continuación natural del Handbook, y cubre lo que necesitarás en el día a día:

- Mantener el sistema actualizado (`emerge --sync` , `emerge -avuDN @world`)
- Instalar y desinstalar paquetes adicionales
- USE flags a fondo (ya viste lo básico en capítulos previos)
- OpenRC en profundidad: servicios, runlevels, dependencias

▼ Otros recursos útiles en la wiki (no forman parte del Handbook)

- **Índice de temas de documentación** — la wiki organiza artículos por categoría (redes, contenedores, seguridad, hardware, etc.)
- **Guía de localización** — útil para dejar el sistema completamente en español (fechas, mensajes, teclado). Relevante para tu configuración con locale `es_MX.UTF-8` .
- **Artículos "meta" de DE/WM** — resúmenes de alto nivel de entornos de escritorio (KDE Plasma, GNOME, XFCE) y gestores de ventanas (i3, Sway, Hyprland). Cada uno tiene pasos de

instalación propios y distintos entre sí.

- **Category:Meta** — artículos panorámicos sobre software disponible en el ecosistema Gentoo.

52.2 — Gentoo en línea

Código de conducta: todos los canales oficiales (foro, IRC, listas, Bugzilla) se rigen por el *Code of Conduct* de Gentoo. Participar es un privilegio, no un derecho — se espera buen trato incluso al reportar errores.

52.2.1 — Foro e IRC Comunidad

Para qué sirve: son los canales de soporte entre usuarios, en tiempo real (IRC) o asíncrono (foro). Antes de preguntar, siempre conviene buscar — es muy probable que otro usuario nuevo ya haya tenido tu mismo problema.

- **Foro:** forums.gentoo.org
- **IRC:** canal `#gentoo` en la red **Libera.Chat**

Cómo entrar al canal IRC (opciones)

- 1 **Vía navegador (sin instalar nada):** entra a web.libera.chat/#gentoo, elige un apodo (nickname) y te conecta directo al canal.
- 2 **Vía cliente IRC dedicado** (ej. `irssi`, `weechat`, `HexChat`): se instala con `emerge --ask net-irc/irssi` (o el cliente que prefieras), luego se conecta al servidor `irc.libera.chat` y se une al canal con `/join #gentoo`.
- 3 **Registro de nick (opcional pero recomendado):** en Libera.Chat se registra el nickname con NickServ para evitar que otros lo usen y para tener acceso a ciertos canales con restricciones.

52.2.2 — Listas de correo Comunidad

Para qué sirve: alternativa para quienes prefieren soporte por correo electrónico en vez de crear cuenta en foro o usar IRC. Cada lista cubre un tema específico (usuarios, desarrollo, anuncios, etc.) y requiere suscripción previa siguiendo las instrucciones de cada lista en el sitio oficial.

52.2.3 — Bugs Diagnóstico

Cuándo usarlo: es el último recurso, después de buscar en la wiki, el foro y preguntar en IRC/listas sin encontrar solución. En ese punto, es señal de que puede ser un error real del software o de la documentación.

Proceso para reportar un bug

- 1 Entra a bugs.gentoo.org (Bugzilla de Gentoo).
- 2 Busca si el problema ya fue reportado (evita duplicados).
- 3 Si no existe, crea una cuenta y abre un nuevo reporte, indicando: paquete/versión afectada, pasos para reproducir, logs relevantes (ej. salida de `emerge --info`).
- 4 Un desarrollador asignado dará seguimiento — el Handbook aclara explícitamente que no hay que temerles, generalmente son accesibles.

52.2.4 — Guía de desarrollo Opcional / avanzado

Para qué sirve: es para quien quiera ir más allá de usuario y contribuir a Gentoo mismo — no es necesaria para uso normal del sistema.

- Sitio: devmanual.gentoo.org
- Enseña a escribir **ebuilds** (las "recetas" que Portage usa para compilar paquetes)
- Explica **eclasses** (funciones reutilizables entre ebuilds)
- Define conceptos generales del desarrollo en Gentoo

▼ ¿Y systemd? — nota contextual (tu instalación usa OpenRC)

El capítulo 52 no menciona systemd directamente porque es puramente informativo sobre comunidad y documentación, no sobre configuración de sistema. Sin embargo, vale aclarar: los recursos de esta sección (foro, IRC, Bugzilla, listas de correo) son **idénticos** para instalaciones con OpenRC o con systemd — no hay diferencia de canal según el init system. La única diferencia sería, en la sección "Working with Gentoo" que sigue, algunos artículos estarán separados por init system (por ejemplo, gestión de servicios: `rc-update` en OpenRC vs `systemctl` en systemd). Como tu instalación usa **OpenRC**, esa será la rama de documentación relevante para ti.

Dónde se configura / dónde se usa cada cosa

Elemento	Dónde vive	Acción típica
Foro	forums.gentoo.org	Buscar antes de postear; crear cuenta solo si vas a participar

IRC	irc.libera.chat , canal <code>#gentoo</code>	Conectar vía web o cliente (<code>irssi</code>), <code>/join #gentoo</code>
Listas de correo	gentoo.org/get-involved/mailling-lists	Suscribirse a la lista temática deseada
Bugzilla	bugs.gentoo.org	Buscar bug existente → si no hay, crear reporte nuevo
Devmanual	devmanual.gentoo.org	Consulta al escribir ebuilds/eclases propios
Siguiente parte del Handbook	Handbook:AMD64/Working/Portage	Leer para gestión post-instalación (updates, USE, OpenRC)

Capítulo 53 — Reflexiones finales

INFORMATIVO

*Cierre oficial del Handbook. No contiene comandos ni configuración — es la despedida del proceso de instalación. Se enriquece este documento con el **mantenimiento de Portage** (Cap. 49), el paso técnico real que da seguimiento natural a esta etapa.*

¿Qué es este capítulo?

Para qué sirve

Es el epílogo del Handbook completo. No enseña nada técnico: agradece al usuario, invita a dar feedback sobre la documentación y despide el proceso de instalación. Si llegaste hasta aquí siguiendo la secuencia, significa que tu sistema Gentoo ya está instalado, arrancado (reboot) y con usuario creado.

El único contenido "accionable" es: si encuentras errores en el Handbook, reportarlos siguiendo las pautas de la sección "*How do I improve the Handbook?*" al inicio del documento. No hay archivos que tocar ni paquetes que instalar en este capítulo.

Sobre el mantenimiento de Portage

Puente hacia el resto del compendio

El paso técnico que sigue naturalmente al reboot -- limpiar la caché de Portage (`eclean-dist` , `eclean-pkg`) -- se cubre a fondo en el [Capítulo 49](#). Para el uso diario de `emerge` ya con el sistema arrancado (instalar, actualizar `@world` , resolver conflictos con `--autounmask` , `package.use/` , etc.), ver el [Apéndice: Gestión de paquetes post-instalación](#).

¿Aplica systemd aquí?

No. Tanto `eclean` como `emerge --depclean` son herramientas de **Portage**, el gestor de paquetes de Gentoo — funcionan igual sin importar si el sistema usa OpenRC o systemd como init. En tu caso, con **OpenRC** confirmado, no hay pasos alternativos que considerar en este punto.

Resumen: dónde se configura / dónde se ejecuta

Elemento	Ubicación / Comando	Notas
Caché de código fuente	<code>/var/cache/distfiles</code>	Limpiar con <code>eclean-dist</code>
Caché de binarios	<code>/var/cache/binhost/gentoo</code>	Limpiar con <code>eclean-pkg</code>
Instalar gentoolkit	<code>emerge --ask app-portage/gentoolkit</code>	Trae <code>eclean</code> , <code>equery</code> , <code>revdep-rebuild</code>
Quitar paquetes huérfanos	<code>emerge --ask --depclean</code>	Siempre probar antes con <code>--pretend</code>

Compendio personal — Instalación Gentoo AMD64 · OpenRC · GRUB · dracut · es_MX.UTF-8

Capítulo 53 (Closing thoughts) enriquecido con Capítulo 49 (Portage maintenance) del Handbook oficial.

Gestión de paquetes post-instalación

Portage · `emerge` y herramientas relacionadas — AMD64 · OpenRC

Contenido

1. ¿Qué es `emerge`?
2. Instalar paquetes
3. Actualizar el sistema (`@world`)
4. Sincronizar el árbol de ebuilds
5. Buscar paquetes
6. Desinstalar y limpiar
7. Conflictos y `--autounmask`
8. `package.use/` y `package.accept_keywords/`
9. Inspección: `equery`, `revdep-rebuild`
10. Dependencias circulares
11. Nota sobre `systemd`
12. Tabla resumen — dónde se configura

1. ¿Qué es `emerge`?

`emerge` es el gestor de paquetes de Portage: resuelve dependencias, descarga fuentes (o binarios), compila e instala. Es el único comando que necesitas para el 90% del mantenimiento diario del sistema.

Para qué sirve: es el punto de entrada único para instalar, actualizar, desinstalar, buscar y limpiar software. Internamente lee `/etc/portage/make.conf` en cada invocación, además de todo lo que haya bajo `/etc/portage/package.*/`.

Sintaxis general:

```
emerge [opciones] <categoría/paquete | @conjunto>
```

El paquete se identifica como `categoría/nombre` (ej. `www-client/firefox`). Si el nombre no es ambiguo, puedes omitir la categoría. Un `@conjunto` (set) agrupa varios paquetes — el más importante es `@world`, la lista de todo lo instalado explícitamente.

2. Instalar paquetes

▼ Ejemplo de instalación básica

```
# Instalar un navegador
root # emerge --ask www-client/firefox

# Proceso que sigue Portage al ejecutarse:
# 1. Lee make.conf y package.* para conocer USE flags, keywords, masks
# 2. Calcula el árbol de dependencias del paquete pedido
# 3. Muestra el plan: qué se instala, con qué flags, tamaño de descarga
# 4. Espera confirmación (por --ask)
# 5. Descarga fuentes/binarios, compila si aplica, e instala
```

Opción	Para qué sirve
<code>-a</code> / <code>--ask</code>	Muestra el plan de acción y pide confirmación antes de tocar el sistema. Úsalo siempre.
<code>-p</code> / <code>--pretend</code>	Simula: calcula el plan pero no ejecuta nada, ni siquiera pide confirmar.
<code>-v</code> / <code>--verbose</code>	Muestra detalle extra: qué USE flags quedan activos/inactivos por paquete.

Diferencia clave: `--pretend` nunca ejecuta nada aunque no lo combines con `--ask`. `--ask` sí ejecuta, pero solo después de tu confirmación.

3. Actualizar el sistema (@world) riesgo medio

Portage no tiene un mecanismo de "generaciones" atómicas: actualiza paquete por paquete sobre el sistema en vivo. El objetivo de actualización casi siempre es `@world`, el conjunto de paquetes instalados explícitamente (más el `@system` base).

▼ Ejemplo de actualización completa

```
root # emerge --ask --verbose --update --deep --newuse @world

# Proceso:
# 1. Toma @world como punto de partida
# 2. --deep expande la revisión a TODO el árbol de dependencias, no solo directas
# 3. Compara versiones instaladas vs disponibles en el árbol sincronizado
# 4. --newuse detecta cambios de USE flags en make.conf/package.use desde la última compilación
# 5. Construye el plan de recompilación/actualización y lo muestra (por -v)
# 6. Tras confirmar, compila/instala en orden de dependencias
```

Opción	Para qué sirve
<code>-u</code> / <code>--update</code>	Considera paquetes ya instalados como candidatos a actualizar (sin esto, solo instala lo que falta).
<code>-D</code> / <code>--deep</code>	Revisa dependencias de forma recursiva y completa, no solo el primer nivel.

<code>--newuse</code>	Recompila si cambiaron USE flags desde la última vez que se compiló el paquete.
<code>--changed-use</code>	Similar, pero solo si el cambio de USE afecta características ya compiladas (más selectivo, evita recompilaciones innecesarias).
<code>-g /</code> <code>--getbinpkg</code>	Prefiere descargar binarios de un binhost configurado en vez de compilar desde fuente.

4. Sincronizar el árbol de ebuilds

Antes de instalar o actualizar, tu copia local de "recetas" (ebuilds) debe estar actualizada.

▼ Ejemplo de sincronización

```
root # emerge --sync

# Proceso:
# 1. Lee sync-type y sync-uri en /etc/portage/repos.conf/gentoo.conf
# 2. Descarga los cambios más recientes (rsync/git/webrsync según config)
# 3. Verifica firmas OpenPGP del snapshot (si sync-openpgp-key-path está definido)
# 4. Actualiza /var/db/repos/gentoo con el árbol nuevo
# 5. Anuncia "news items" pendientes si los hay

# Terminal lenta (framebuffer/serial):
root # emerge --sync --quiet
```

▼ Configuración: /etc/portage/repos.conf/gentoo.conf

```
[DEFAULT]
main-repo = gentoo

[gentoo]
location = /var/db/repos/gentoo
sync-type = rsync
sync-uri = rsync://rsync.gentoo.org/gentoo-portage/
auto-sync = yes
```

Aquí se define el mecanismo de sincronización. Cambiar `sync-type` a `git` es común si prefieres seguir el repositorio oficial en GitHub en vez de rsync.

Leer noticias del repositorio: tras sincronizar, Portage puede avisar de "news items" — mensajes críticos (cambios que rompen algo, migraciones necesarias).

```
root # eselect news list    # ver disponibles
root # eselect news read   # leer
root # eselect news purge  # marcar como leídos
```

5. Buscar paquetes

```
root # emerge --search firefox          # por nombre
root # emerge --searchdesc "web browser" # también en descripción, más lento
```

Alternativa recomendada: `app-portage/eix` mantiene un índice propio y busca mucho más rápido que `emerge --search`. Vale la pena instalarlo temprano: `emerge --ask app-portage/eix`, y luego `eix <paquete>`.

6. Desinstalar y limpiar

▼ Desinstalar un paquete específico **riesgo medio**

```
root # emerge --ask --unmerge www-client/firefox
```

`-C` / `--unmerge`: elimina exactamente el paquete indicado, **sin** tocar sus dependencias huérfanas — esas se limpian aparte.

▼ Limpiar paquetes obsoletos/huérfanos **riesgo alto si no revisas antes**

```
# 1. Revisar SIEMPRE antes de ejecutar de verdad
root # emerge --ask --pretend --depclean

# 2. Si la lista se ve correcta, ejecutar
root # emerge --ask --depclean
```

`--depclean` elimina paquetes que ya no son necesarios: dependencias que quedaron huérfanas tras actualizaciones, o versiones antiguas de paquetes con slots múltiples (como el kernel).

Proteger un paquete de depclean: si Portage marca algo como huérfano pero lo usas manualmente fuera de @world, usa `emerge --noreplace app-misc/foo` para preservarlo sin agregarlo formalmente al conjunto mundial.

7. Conflictos de configuración: `--autounmask`

Cuando Portage no puede instalar algo con la configuración actual (falta un USE flag, requiere rama inestable, está enmascarado), no falla en silencio: te muestra exactamente qué cambio resolvería el conflicto.

▼ Ejemplo: conflicto de USE flag

```
root # emerge --ask media-video/mpv

# Salida típica de Portage:
# The following USE changes are necessary to proceed:
# #required by mpv (argument)
# >=media-video/mpv-0.38.0 -vulkan
```

Opción	Para qué sirve
<code>--autounmask</code>	Calcula y <i>propone</i> el cambio necesario. Activo por defecto. No modifica archivos.
<code>--autounmask-write</code>	Escribe la propuesta directamente en <code>/etc/portage/package.*/</code> . No vuelve a intentar el emerge por sí solo.
<code>--autounmask-keep-masks</code>	Restringe la propuesta a USE/keywords, sin proponer desenmascarar paquetes.

▼ Flujo completo para aplicar el cambio

```
root # emerge --ask --autounmask-write media-video/mpv
root # etc-update
root # emerge --ask media-video/mpv

# Proceso:
# 1. autounmask-write escribe el cambio propuesto (a veces como archivo ._cfg0000_*)
# 2. etc-update te muestra el diff y te deja aprobarlo o rechazarlo
# 3. Una vez aprobado, el archivo definitivo queda en package.use/ o package.accept_keywords/
# 4. Se reintenta el emerge, ahora sin conflicto
```

8. `package.use/` y `package.accept_keywords/`

`package.use/` — excepciones de USE flags por paquete

Directorio donde defines USE flags específicos de un paquete, sin tocar el `USE=` global de `make.conf`. No existen archivos por defecto — tú los creas.

▼ Ejemplo: `/etc/portage/package.use/mpv`

```
media-video/mpv -vulkan libplacebo
```

Sintaxis: `categoría/paquete flag1 -flag2 flag3`. Sin signo = activar; con `-` = desactivar. Puedes acotar por versión: `>=media-video/mpv-0.38.0`.

package.accept_keywords/ — permitir ramas inestables

Gentoo marca paquetes como estables (`amd64`) o inestables/testing (`~amd64`). Por defecto solo se instalan versiones estables. Si necesitas una versión más nueva que aún está en pruebas, la autorizas aquí — paquete por paquete, no globalmente.

▼ Ejemplo: /etc/portage/package.accept_keywords/mpv riesgo medio

```
media-video/mpv ~amd64
```

Esto autoriza instalar versiones `~amd64` (testing) solo de `mpv`, dejando el resto del sistema en la rama estable.

Para qué sirve separar en directorio: Portage lee *todos* los archivos dentro de `package.use/` y `package.accept_keywords/`, sin importar el nombre. La convención de un archivo por paquete (o por tema) es solo para mantener orden y facilitar revertir cambios individuales.

9. Herramientas de inspección

▼ equery (paquete gentoolkit) — consultar sin instalar

```
root # emerge --ask app-portage/gentoolkit

root # equery list '*'           # todo lo instalado
root # equery depends www-client/firefox # qué depende de firefox
root # equery uses www-client/firefox   # USE flags activos/inactivos
root # equery files www-client/firefox  # archivos instalados por el paquete
root # equery belongs /usr/bin/firefox   # a qué paquete pertenece un archivo
```

▼ revdep-rebuild — reparar enlaces rotos riesgo bajo

```
root # revdep-rebuild

# Para qué sirve: tras actualizar librerías compartidas, algunos binarios
# pueden quedar enlazados contra versiones viejas. Este comando detecta
# esos casos y recompila los paquetes afectados.
```

10. Dependencias circulares

A veces el paquete A necesita a B para compilarse, y B necesita a A. Portage resuelve esto rompiendo el ciclo con una compilación en dos pasadas: instala una versión mínima de uno de los

dos (a veces usando USE flags reducidos temporalmente), y luego reconstruye ambos ya con las dependencias completas satisfechas. Esto es automático — normalmente no necesitas intervenir, pero si ves mensajes de "circular dependencies" en un `--pretend`, es señal de que Portage ya está resolviendo esto internamente antes de compilar.

11. Nota sobre systemd

No aplica a tu instalación (OpenRC). Se incluye solo como referencia si en el futuro migras o comparas sistemas.

En un sistema con systemd como gestor de servicios, `emerge` funciona exactamente igual — la gestión de paquetes no cambia. Lo que cambia es cómo administras servicios:

Tarea	OpenRC (tu sistema)	systemd
Habilitar servicio al arrancar	<code>rc-update add sshd default</code>	<code>systemctl enable sshd</code>
Iniciar servicio ahora	<code>rc-service sshd start</code>	<code>systemctl start sshd</code>
Ver estado	<code>rc-service sshd status</code>	<code>systemctl status sshd</code>
Listar servicios habilitados	<code>rc-update show</code>	<code>systemctl list-unit-files --state=enabled</code>

Portage también expone el USE flag global `systemd` en `make.conf` para paquetes que ofrecen integración opcional (unidades `.service`, journal, etc.) — en tu configuración OpenRC ese flag debe permanecer desactivado.

12. Tabla resumen — dónde se configura

Archivo / directorio	Propósito
<code>/etc/portage/make.conf</code>	USE flags globales, opciones de compilación, mirrors, binhost. Se lee en cada <code>emerge</code> .
<code>/etc/portage/package.use/</code>	USE flags específicos por paquete. Un archivo por paquete/tema (convención, no obligatorio).
<code>/etc/portage/package.accept_keywords/</code>	Autoriza ramas inestables (<code>~amd64</code>) por paquete.
<code>/etc/portage/package.mask</code> / <code>package.unmask</code>	Bloquea o desbloquea manualmente versiones específicas de paquetes.
<code>/etc/portage/repos.conf/gentoo.conf</code>	Define cómo y desde dónde se sincroniza el árbol de ebuilds (<code>emerge --sync</code>).
<code>/var/db/repos/gentoo</code>	Copia local del árbol de ebuilds sincronizado.